Black Box as a Diagnostic Reasoning Benchmark for LLMs

Assessing Hidden State Identification Through Test Selection, Constraint Tracking, and Belief Updating

Todd R. Johnson

2026-03-08

Diagnostic reasoning—identifying hidden states from observable evidence under a known causal model—is fundamental to medical diagnosis, fault detection, and troubleshooting. This study uses the classic Black Box game as a benchmark to assess LLM capabilities in diagnostic reasoning. Black Box requires the core processes of diagnosis: generating hypotheses about hidden states, selecting informative tests, tracking constraints as evidence accumulates, and updating beliefs appropriately. Unlike medical benchmarks that may reward pattern matching to training data, Black Box provides out-of-distribution problems with verifiable ground truth. We present baseline results comparing Claude model variants (Haiku 4.5, Sonnet 4.5/4.6, Opus 4.5/4.6) across prompt conditions. Our findings reveal systematic limitations in the component processes of diagnostic reasoning that persist despite scaling and prompt engineering, with implications for the deployment of LLMs in medical diagnosis and other domains requiring hidden state identification.

Table of contents

1 Introduction

1.1 The Deployment Reality

AI systems are increasingly deployed for diagnostic tasks—identifying hidden states or conditions from observable evidence. Medical AI tools assist with diagnosis from imaging, lab results, and symptoms. Industrial systems perform fault diagnosis on complex machinery. Customer service agents troubleshoot technical problems through iterative questioning.

Yet deployment has outpaced our understanding of whether these systems actually perform diagnostic *reasoning* or sophisticated pattern matching. Wen et al. (2025) demonstrate that high performance on medical benchmarks—the multiple-choice question answering (MCQA) format derived from clinical licensing examinations—may reflect "context matching" rather than clinical reasoning. Their analysis invalidates three assumptions underlying MCQA-based assessment: that knowledge is applied rather than memorized, that semantic consistency leads to consistent answers, and that models can recognize situations with no valid answer.

If LLMs achieve high scores on medical board examinations through pattern matching rather than inference, benchmark performance provides false assurance about deployment readiness for genuine diagnostic tasks.

1.2 The Evaluation Crisis

The fundamental problem is that current benchmarks may not test diagnostic reasoning at all. Most medical AI benchmarks use static question-answering formats: given a case description, select the correct diagnosis from options. This tests recognition, not the iterative process of gathering evidence and refining hypotheses that characterizes actual diagnosis.

We need benchmarks that test the *process* of diagnostic reasoning, not just its end product. But creating such benchmarks in medical or industrial domains is difficult: cases are complex, ground truth is often uncertain, and training data contamination is hard to control. We need controlled environments where the component processes of diagnostic reasoning can be isolated and measured.

1.3 What is Diagnostic Reasoning?

Diagnostic reasoning is the cognitive process of identifying which hidden state (or states) is present based on observable evidence, given a known or partially known model of how hidden states produce observations.

The structure of diagnostic reasoning:

- **Given**: A causal model mapping hidden states to observations (diseases cause symptoms; faults cause error codes; atom configurations cause ray behaviors)
- **Task**: Identify which hidden state is present based on observations
- **Process**: Iteratively gather evidence and refine hypotheses until confident

Diagnostic reasoning is distinct from:

- **Causal structure discovery**: Learning *what causes what* (the causal graph itself)
- **Causal effect estimation**: Measuring *how much* an intervention affects an outcome
- **Mechanism discovery**: Understanding *how* a cause produces its effect

In diagnostic reasoning, we typically know (or assume) the causal structure—we know that certain diseases cause certain symptoms, or that certain faults produce certain error patterns. The challenge is identifying *which* specific state from a known set is present.

Component processes of diagnostic reasoning:

Process	Description
Hypothesis Generation	Generate candidate hidden states that could explain observations
Forward Reasoning	Predict what observations would occur under each hypothesis
Test Selection	Choose which observation to gather next to best discriminate among hypotheses
Constraint Tracking	Maintain which hidden states have been ruled in or out
Belief Updating	Revise hypothesis probabilities when new evidence arrives

Stopping Decision	Decide when evidence is sufficient to commit to a diagnosis

These processes must be *integrated*: test selection depends on current hypotheses; hypotheses depend on accumulated evidence; the decision to stop depends on confidence in the leading hypothesis. Failures in any component can cascade through the diagnostic process.

1.4 Black Box as a Diagnostic Instrument

We propose the Black Box game as a diagnostic instrument for understanding LLM reasoning limitations. Black Box is a classic deduction game designed by Eric Solomon in 1978 (Wikipedia contributors 2024). Players face an 8×8 grid containing four hidden "atoms" at unknown locations. By firing rays into the grid and observing their behavior—absorption, deflection, or reflection—players must infer the atom positions.

Black Box is not merely another game benchmark. It is a *minimal realization of diagnostic reasoning*—requiring all six component processes in an integrated task:

Diagnostic Process	Black Box Analog
Hypothesis Generation	Consider which atom configurations could explain observations
Forward Reasoning	Predict ray behavior under a hypothesized configuration
Test Selection	Choose which ray to fire next to discriminate among hypotheses
Constraint Tracking	Maintain which cells are ruled out by accumulated evidence
Belief Updating	Revise candidate configurations when new ray results arrive
Stopping Decision	Decide: fire more rays or commit to a guess?

This structural correspondence makes Black Box findings informative about diagnostic reasoning generally, while providing advantages that clinical or scientific domains lack:

1. **Out-of-distribution**: While the rules are simple and fully specifiable, any particular game configuration is almost certainly not in training data
2. **Ground truth**: We know the correct atom positions and can precisely measure both forward reasoning accuracy (Predict mode) and inverse reasoning accuracy (Play mode)
3. **Decomposable errors**: We can analyze extended thinking traces to categorize failures as occurring in experiment design, world model simulation, constraint tracking, or belief updating

1.5 Addressing the Spatial Reasoning Confound

A methodological concern is whether Black Box's spatial reasoning requirements confound interpretation. Many diagnostic reasoning problems—medical diagnosis, fault detection, troubleshooting—do not involve spatial relationships, so spatial failures on Black Box might not predict failures on non-spatial diagnostic tasks.

We address this through experimental design. Our two-mode structure allows partial decomposition:

- **Predict mode** tests forward reasoning (heavily spatial): given atom positions, trace the ray's path
- **Play mode** tests the full diagnostic loop: spatial reasoning is one input, but experiment design, abductive inference, and constraint tracking are the primary challenges

Comparing performance across modes provides diagnostic information: good Predict performance with poor Play performance would implicate non-spatial reasoning limitations (abduction, constraint tracking, experiment design), while poor performance on both would suggest spatial reasoning is at least partly responsible.

Additionally, we use *mean atoms per ray*—a measure of spatial complexity—as a moderator variable. If treatment effects (model, prompt style, extended thinking) are constant across spatial complexity, the limitations lie in inferential machinery rather than spatial processing. If effects are moderated by spatial complexity, spatial reasoning contributes to observed failures.

Finally, we analyze extended thinking traces to categorize errors:

- **Spatial errors**: Ray path traced incorrectly
- **Constraint errors**: Failed to track ruled-out positions
- **Experiment design errors**: Fired uninformative rays
- **Belief updating errors**: Did not revise hypotheses when evidence contradicted them

Black Box alone cannot fully separate spatial from non-spatial limitations. However, our findings can be triangulated with non-spatial benchmarks (ZebraLogic for constraint satisfaction, LogicGame for rule-based planning) to identify common factors. If LLMs fail on Black Box *and* on non-spatial reasoning benchmarks, the shared limitation is inferential machinery, not spatial processing specifically.

1.6 Research Questions

This study addresses the following questions:

1. **Diagnostic Reasoning Capability**: Can LLMs perform the integrated reasoning loop required for diagnostic inference on novel problems?

2. **Forward vs. Inverse Reasoning**: Do limitations arise primarily in world model simulation (Predict mode) or in the inferential machinery of hypothesis generation and updating (Play mode)?
3. **Decomposition of Failures**: What proportion of errors reflect spatial reasoning failures versus non-spatial failures (constraint tracking, experiment design, belief updating)?
4. **Prompt and Scaffold Effects**: Do augmented instructions, visualizations, or extended thinking improve performance, and do these effects interact with spatial complexity?
5. **Scaling Effects**: Do larger models show improved diagnostic reasoning, and if so, is improvement sufficient to approach human-level competence?
6. **Implications for Deployment**: What do the observed limitations suggest about the readiness of LLMs for medical diagnosis, fault detection, and other diagnostic tasks?

2 Literature Review

The case for Black Box as a diagnostic instrument rests on documented limitations in each component capability required for diagnostic reasoning. If LLMs struggle with constraint satisfaction, spatial reasoning, world model simulation, and abductive inference *in isolation*, we should expect compounding failures when these capabilities must be integrated. The literature reviewed below provides this supporting evidence, organized by the component capabilities Black Box requires.

A comprehensive survey by Giadikiaroglou et al. (2024) provides a useful taxonomy for understanding LLM puzzle-solving capabilities. They distinguish between *rule-based puzzles* (with explicit victory conditions and game rules, including deterministic puzzles like Sudoku and Chess, and stochastic puzzles like Minesweeper and Poker) and *rule-less puzzles* (requiring world knowledge and inference, such as riddles and commonsense reasoning). Black Box falls squarely in the rule-based deterministic category, making findings from this literature directly relevant. The survey notes that advanced prompting methods like Tree-of-Thoughts and Everything-of-Thoughts yield improvements of 26-70% over basic Chain-of-Thought on deterministic puzzles, yet "significant challenges in complex puzzle scenarios" persist, revealing gaps between LLM capabilities and human-like reasoning.

2.1 Constraint Satisfaction and LLMs

Constraint satisfaction problems (CSPs) require finding values for variables that satisfy a set of constraints. Classic examples include Sudoku, scheduling problems, and logic puzzles. Recent research has systematically evaluated LLM performance on such tasks.

The ZebraLogic benchmark (Lin et al. 2024) evaluates LLMs on logic grid puzzles (also known as Zebra puzzles), a well-established CSP format used in standardized tests like the LSAT.

Results indicate that while LLMs show some capability on simple puzzles, performance degrades substantially as problem complexity increases. The benchmark reveals that LLMs lack several abilities required for complex logical reasoning, including counterfactual thinking, reflective reasoning, and compositional generalization.

Bonlarron, Régin, and De Maria (2024) demonstrate that LLMs "excel at generating fluent text but struggle to enforce external constraints because they generate tokens sequentially without explicit control mechanisms." Their GenCP framework addresses this by combining LLM predictions with constraint programming (CP) reasoning, treating text generation as a CSP. This hybrid approach significantly outperforms pure LLM methods, suggesting that constraint satisfaction may require symbolic reasoning components that LLMs lack.

Michailidis, Tsouros, and Guns (2024) evaluated LLMs on translating natural language problem descriptions into formal constraint models. They found that zero-shot prompting yields essentially no success, and even few-shot prompting produces inconsistent results. This aligns with Freuder (2024)'s observation that successful constraint satisfaction often requires iterative refinement through dialogue, not single-shot generation.

The LogicGame benchmark (Gui et al. 2024) directly evaluates rule-based reasoning—the ability to understand, execute, and plan according to predefined rules. Using game scenarios at varying difficulty levels, they found that even the best-performing model (claude-3.5-sonnet) achieves only 19.15% overall accuracy, with performance falling below 20% on complex tasks requiring multi-step planning. Critically, they observed that while execution tasks improve modestly with few-shot examples, planning tasks often *deteriorate* with additional demonstrations, and any improvements diminish substantially at higher difficulty levels. This suggests that LLMs struggle not just with constraint tracking but with the multi-step planning required to design informative experiments.

The implications for Black Box are direct: tracking which cell locations are ruled out by each ray observation is fundamentally a constraint satisfaction problem. If LLMs struggle with CSPs generally, they should struggle with the constraint-tracking aspect of Black Box.

2.2 Spatial Reasoning Limitations

Spatial reasoning—the ability to mentally represent and manipulate spatial relationships—is a well-documented weakness of current LLMs.

The PLUGH benchmark (Tikhonov et al. 2024) evaluates spatial understanding and reasoning using text adventure game scenarios that require building mental maps from textual descriptions. Results show that while some commercial LLMs exhibit moderate reasoning abilities, "all models still have significant room for improvement." The benchmark identifies typical failure modes including difficulty maintaining consistent spatial representations across multiple textual updates.

Chen et al. (2024) conducted an empirical analysis of spatial reasoning in large multimodal models, finding that even GPT-4o achieves only 27.5% accuracy on questions requiring reasoning from a human's viewpoint in an image. Performance degrades further on multi-hop spatial reasoning questions that require integrating multiple spatial relationships.

Bos (2024) provides a practitioner's perspective on LLM spatial reasoning, noting that "all of the LLMs failed immediately on the easiest problems" involving mental box folding—a standard spatial reasoning task in human cognitive assessments. Interestingly, LLMs performed better on reasoning about objects in photographs than on abstract spatial diagrams, suggesting reliance on statistical patterns from image-caption training data rather than genuine spatial reasoning.

A synthesis of spatial reasoning research (Quan et al. 2025) notes that "empirical assessments consistently show a characteristic pattern: models exhibit moderate competence in simple, direct spatial tasks but rapidly deteriorate as the problem scale or compositional/geometric complexity increases, with performance losses ranging from 42% to over 80% as task complexity grows."

The BALROG benchmark (Paglieri et al. 2024) evaluates LLMs and vision language models on diverse game environments ranging from tasks "solvable by non-experts in seconds to extremely difficult challenges like NetHack requiring years to master." Their findings reveal that current models achieve only partial success on simpler games but "struggle significantly with more challenging tasks." Most striking is their discovery of "severe deficiencies in vision-based decision-making"—several models actually perform *worse* when given visual representations of environments compared to text-only descriptions. This counterintuitive result suggests that visual processing may introduce noise rather than helpful spatial information.

J. Wang et al. (2025) addresses a related challenge: tasks requiring both semantic understanding and adherence to visual grid constraints. Using crossword puzzles as a benchmark, they demonstrate that "mathematical reasoning training doesn't necessarily generalize to tasks requiring linguistic and spatial integration." This finding is particularly relevant to Black Box, which similarly requires integrating rule-based reasoning (ray physics) with spatial constraint tracking (grid positions).

For Black Box, ray tracing requires understanding how rays move through 2D space and how they deflect upon encountering atoms. The documented limitations in spatial reasoning suggest LLMs will struggle with accurate ray path simulation.

2.3 Reasoning Complexity and Performance Collapse

A critical question for evaluating LLM reasoning is how performance scales with problem complexity. Shojaee et al. (2025) provide a systematic investigation using controllable puzzle environments that allow precise manipulation of compositional complexity while maintaining consistent logical structures.

Their findings reveal three distinct performance regimes. In low-complexity tasks, standard models surprisingly outperform reasoning models (those using extended thinking or chain-of-thought), suggesting that extended reasoning can introduce unnecessary complexity for simple problems. In medium-complexity tasks, extended thinking provides clear advantages, demonstrating the intended benefit of reasoning-augmented models. However, in high-complexity tasks, both standard and reasoning models exhibit complete accuracy collapse—a fundamental ceiling that persists regardless of model capability or available computational resources.

Particularly striking is their observation that reasoning models exhibit *declining effort* at higher complexity levels despite adequate token budgets. Rather than leveraging available computational resources more intensively when problems become harder, models show inconsistent reasoning patterns across similar problems. This suggests that current reasoning mechanisms are not genuinely algorithmic but rather approximate heuristics that fail when problems exceed certain complexity thresholds.

The PuzzlePlex benchmark (Authors 2025b) extends this analysis to multi-turn adversarial environments, evaluating LLM reasoning and planning across 24 diverse puzzles including both deterministic and stochastic games. Their findings are sobering: "LLMs still significantly lag behind even simple heuristics in puzzles." This result suggests that the limitations are not merely about single-step reasoning accuracy but extend to the sequential decision-making and strategic planning that characterizes game play. For tasks requiring iterative hypothesis refinement based on accumulated evidence—like Black Box—this predicts systematic failures in experiment design and belief updating.

For Black Box, these findings predict a non-linear relationship between game difficulty (determined by atom configurations, number of atoms, and observation ambiguity) and model performance. Simple configurations may be solved adequately, but complex configurations requiring deep inference chains should trigger the same "reasoning collapse" observed across other puzzle domains.

2.4 World Models and Mental Simulation

A "world model" refers to an internal representation that can be used to simulate how states evolve over time. The question of whether LLMs possess genuine world models—versus sophisticated pattern matching—is actively debated.

Harig et al. (2025) investigated this question using mechanical reasoning about pulley systems. Their findings are striking: while LLMs showed some ability to estimate mechanical advantage, they appeared to use a "pulley counting heuristic" rather than genuine simulation. The best model (GPT-4o) achieved only 26.1% accuracy on exact predictions. When problems were modified to be superficially similar but mechanically different, performance collapsed, suggesting "abilities are often fragile and dependent on superficial features rather than deep understanding."

R. Wang et al. (2024) directly tested whether LLMs can serve as text-based world simulators. Using a benchmark of 76,369 state transitions from text adventure games, they found that GPT-4's best single-step accuracy was 59.9%. Because simulation errors accumulate, after 10 steps the expected accuracy drops to less than 1% (0.599^{10}). They conclude that "LLMs are not yet able to reliably act as text world simulators."

This literature suggests that LLMs may appear to reason about world dynamics but are actually relying on heuristic shortcuts that fail when problems require genuine simulation. For Black Box's "Predict" mode—where the model must trace a ray's path given known atom positions—we should expect systematic errors reflecting heuristic approximation rather than accurate simulation.

However, recent work suggests that targeted post-training on decision-making games can substantially improve LLM reasoning in those specific domains. Y. Wang et al. (2025) developed data synthesis strategies for training LLMs on Doudizhu (a Chinese card game) and Go, demonstrating that game-based training data can complement traditional code and mathematics datasets for reasoning enhancement. Their "Mastermind" models achieved 90% action accuracy matching expert data and 98% similarity in thought processes for Doudizhu. Critically, this approach uses hierarchical training pipelines that decompose complex decisions into subtasks—possible action prediction, opponent strategy modeling, and final action selection—rather than relying on direct policy learning.

For our purposes, this work highlights an important distinction: Black Box is designed as an *out-of-distribution* benchmark that tests zero-shot reasoning capabilities on novel configurations. The success of task-specific training reported by Y. Wang et al. (2025) does not contradict predictions of poor zero-shot performance; rather, it suggests that observed limitations may reflect training distribution gaps rather than fundamental architectural constraints. Whether fine-tuning on Black Box traces would improve performance—and whether such improvement would generalize to other spatial reasoning tasks—remains an empirical question addressed in our future directions.

2.5 Abductive Reasoning and Hypothesis Generation

Abductive reasoning—inferring the most plausible explanation for observations—is the core inferential process in diagnostic reasoning. A growing body of research examines LLM capabilities in this domain.

Yang et al. (2025) provides a comprehensive framework for understanding LLM hypothesis discovery, organizing methods according to Peirce's reasoning paradigm: abduction (hypothesis generation), deduction (hypothesis application), and induction (hypothesis validation). They note that "most existing work assessing LLM reasoning centers almost exclusively on multi-step deductive tasks, leaving abduction and induction, the engines of hypothesis discovery, largely unexplored."

Z. Wang et al. (2025) directly tests whether LLMs follow Occam's Razor—the principle of preferring simpler explanations. Using a synthetic benchmark (InAbHyD) designed to test inductive and abductive reasoning, they find that "current LLMs can't generate high-quality hypotheses that match human-level intelligence." LLMs frequently produce overly complex explanations when simpler ones suffice, failing a fundamental principle of diagnostic reasoning.

Liu, Neubig, and Andreas (2024) characterizes the current state as "an incomplete loop," arguing that while LLMs can perform some aspects of each reasoning type, they fail to integrate abduction, deduction, and induction into a coherent reasoning cycle. This fragmentation prevents genuine hypothesis-driven inquiry.

Rodriguez et al. (2025) argues from a theoretical perspective that LLMs face fundamental limitations in abductive reasoning: "(1) They cannot work with counterfactuals, relying only on correlational datasets. (2) They are unable to perform true abductive reasoning." While LLMs may appear to generate hypotheses, this reflects parameter-controlled variation rather than genuine inference to the best explanation.

For Black Box, the inverse problem—inferring atom locations from ray observations—is quintessentially abductive. The literature predicts systematic failures in generating parsimonious hypotheses and in appropriately updating beliefs as evidence accumulates.

2.6 Test Selection in Diagnostic Reasoning

A distinguishing feature of diagnostic reasoning is *active* test selection—choosing which observations to gather based on current hypotheses to maximally discriminate among candidate diagnoses. Most LLM benchmarks present static problems where all information is provided upfront. Few benchmarks test the ability to strategically select tests and iteratively refine hypotheses.

Auto-Bench (Authors 2025a) evaluates LLMs on a task structurally similar to diagnosis: discovering hidden graph structures through iterative experimentation. The task proceeds in cycles: the model receives previous observations, proposes a hypothesis about the hidden structure, and designs an intervention to test that hypothesis. An oracle executes the intervention and returns new observations, continuing until the hypothesis matches ground truth or a limit is reached. This directly parallels diagnostic reasoning: observe effects, hypothesize hidden states, select tests to discriminate. The key finding is that LLMs struggle with strategic test selection—they often propose uninformative experiments that fail to disambiguate among hypotheses.

This finding is particularly relevant for Black Box. Effective play requires selecting rays that maximally constrain the hypothesis space— firing into regions where different candidate configurations would produce different outcomes. Random or poorly-chosen rays waste resources and may never converge on the correct answer. The documented failures in Auto-Bench predict that LLMs will struggle with strategic ray selection in Black Box's Play mode.

2.7 Base Rate Neglect in Clinical Diagnosis

A particularly consequential manifestation of abductive reasoning failure is base rate neglect—the tendency to overweight salient observations while underweighting prior probabilities. Omar et al. (2025) provide the first large-scale demonstration of this phenomenon in LLMs applied to clinical diagnosis.

Using 300 synthetic clinical vignettes tested across 20 LLMs with approximately 1.8 million responses, they found that models systematically selected rare "zebra" diagnoses linked to salient but non-decisive cues (such as travel history or animal exposure) more often than epidemiologically correct diagnoses (49.8% vs 37.7%). This pattern persisted even when models were explicitly asked to identify the "most likely" diagnosis. Critically, prompts that explicitly invoked base rates and epidemiology reduced but did not eliminate this saliency bias.

The implications are profound: LLMs that achieve high performance on medical board examinations—demonstrating factual knowledge of disease prevalence and diagnostic criteria—nevertheless fail to appropriately weight this knowledge when salient distractors are present. As the authors note, this represents "associative pattern matching rather than stable probabilistic reasoning." Training on medical literature, which overrepresents rare and dramatic conditions in case reports and teaching materials, may systematically bias models toward zebra diagnoses.

For Black Box, this finding suggests that even when LLMs possess accurate knowledge of ray physics, they may be systematically distracted by salient features of observations (e.g., absorption events, unusual deflection patterns) at the expense of systematic constraint tracking and hypothesis evaluation. The base rate neglect documented in clinical diagnosis may manifest as premature hypothesis commitment or failure to consider the full space of configurations consistent with accumulated evidence.

2.8 Implications for Diagnostic AI Systems

The limitations documented above converge on a concerning picture for LLM deployment in diagnostic contexts.

The LLM-Modulo framework (Kambhampati et al. 2024; Valmeekam et al. 2023) proposes a pragmatic response: accept that LLMs cannot plan or reason reliably, but use them as components within hybrid systems that include external verifiers and symbolic reasoners. Under this view, the LLM's role is translation and suggestion, not autonomous diagnostic reasoning. A physician might use an LLM to suggest differential diagnoses, but should not rely on it to track constraints or select optimal tests.

This matters because AI diagnostic systems are already being deployed. Medical AI tools assist with diagnosis from imaging, lab results, and symptoms. Industrial systems perform fault diagnosis on complex machinery. The failure modes documented throughout this review—failure

to track constraints, inability to select informative tests, base rate neglect, and premature hypothesis commitment—map directly onto the requirements of diagnostic reasoning.

Diagnostic benchmarks that can characterize *when* and *why* these failures occur are needed before broader deployment in high-stakes diagnostic domains.

2.9 Summary: Predictions for Diagnostic Reasoning in Black Box

Based on the documented limitations in component processes of diagnostic reasoning, we derive the following predictions:

1. **Forward reasoning (Predict mode)** will show substantial errors, consistent with documented failures in spatial reasoning and world model simulation.
2. **Inverse reasoning (Play mode)** will show even greater errors, as it compounds spatial reasoning with abductive inference and constraint satisfaction.
3. **Prompt engineering** will provide modest improvements but will not overcome fundamental limitations.
4. **Scaling** (larger models) will show improvement but not to human-level performance.
5. **Error patterns** will reveal heuristic approximation rather than systematic reasoning—e.g., over-reliance on surface features, failure to track constraints, non-parsimonious hypotheses.
6. **Tool augmentation** (e.g., giving the LLM a ray-tracing verifier) will help with verification but not with hypothesis generation, as the limitation is in search/inference, not just simulation.
7. **Uninformative test selection**: Following findings from Auto-Bench (Authors 2025a), models will frequently choose rays that fail to disambiguate among hypotheses—firing into already-constrained regions or missing opportunities for informative observations. Strategic test selection requires anticipating which observations would most reduce uncertainty, a capability that current models lack.
8. **Complexity-dependent collapse**: Following Shojaee et al. (2025), performance will show a non-linear pattern—adequate performance on simple configurations but catastrophic failure on complex configurations requiring extended inference chains, regardless of available thinking budget.
9. **Saliency-driven distraction**: Following Omar et al. (2025), models will be systematically distracted by salient observations (e.g., absorption events, dramatic deflections) and may anchor prematurely on hypotheses suggested by striking but non-decisive evidence, even when simpler explanations are more consistent with the full observation set.

10. **Spatial-inferential decomposition**: If spatial reasoning is the primary bottleneck, we expect: (a) poor Predict mode performance,

    (b) treatment effects moderated by spatial complexity (mean atoms per ray), and (c) error traces dominated by ray path miscalculations. If inferential machinery is the primary bottleneck, we expect:
    (c) relatively better Predict than Play performance, (b) treatment effects constant across spatial complexity, and (c) error traces showing constraint tracking failures, uninformative experiment choices, and belief updating errors.

11. **Convergent validity**: If Black Box limitations reflect general diagnostic reasoning failures rather than spatial-specific failures, we expect correlations with findings from non-spatial benchmarks (ZebraLogic, LogicGame). Models that perform poorly on constraint satisfaction and rule-based planning should also perform poorly on Black Box's non-spatial components.

3 Methods

3.1 The Black Box Game Implementation

We implemented Black Box as a web-based application using React, allowing both human play and automated LLM interaction through the Anthropic API. The implementation follows standard Black Box rules:

- **Grid**: 8×8 cells, with 4 randomly placed atoms (non-adjacent placement enforced)
- **Rays**: Fired from any of 32 edge positions (8 per side)
- **Ray behavior**:
  - Direct hit on atom: **Absorbed** (marked "H" for hit)
  - Atom diagonally adjacent to entry cell: **Reflected** (marked "R")
  - Atom diagonally adjacent to path: **Deflected** 90° away from atom
  - Atoms on both sides of path: **Reflected** (reverses direction)
  - Clear path: Ray exits at opposite edge

3.2 Task Modes

3.2.1 Predict Mode (Forward Reasoning)

In Predict mode, the LLM is shown: - The 8×8 grid with atom positions marked - A ray entry point (e.g., "NORTH-4")

The LLM must predict the ray's outcome: exit position, absorption, or reflection. This tests world model accuracy—can the LLM correctly simulate ray physics?

3.2.2 Play Mode (Inverse Reasoning)

In Play mode, the LLM: - Receives the game rules - Chooses which rays to fire (up to 20) - Receives feedback after each ray (exit position, absorption, or reflection) - Must guess all 4 atom positions

This tests the full reasoning pipeline: experiment design, observation integration, constraint tracking, and abductive inference.

3.3 Prompt Conditions

We developed two prompt styles with optional visualization augmentation:

3.3.1 Baseline (Human-Equivalent)

Comprehensive instructions adapted from the traditional Emacs Black Box rules, comparable to what a human player would receive. The prompt includes:

- Complete rules with ASCII diagram examples showing deflection, reflection, and absorption
- Scoring information (rays cost points, missed atoms cost 5 points each)
- JSON response format specification

Key excerpt:

```
Your opponent has hidden 4 balls within this box. By shooting rays into
the box and observing where they emerge, it is possible to deduce the
positions of the hidden balls.

There are three possible outcomes for each ray you send into the box:
- DETOUR: The ray is deflected and emerges somewhere other than where
  you sent it in.
- REFLECTION (R): The ray is reflected and emerges in the same place
  it was sent in.
- HIT (H): The ray strikes a ball directly and is absorbed.
```

3.3.2 Augmented

Extended instructions including:

- Explicit coordinate system explanation
- Detailed deflection geometry with worked examples
- Strategy guidance (triangulation, multiple rays)
- Common error warnings
- Explicit note that single observations are typically ambiguous

3.3.3 Visualization Options

Either prompt style can be combined with:

- **Include Visualization**: Text-based board visualization showing current state. **Note**: Include Visualization was enabled for all experimental runs in both Predict and Play modes and is therefore not a varying factor in the analyses.
- **VoT (Visualization of Thought) Options**:
  - Grid State (Play only): Shows current board state in prompt
  - Ray Trace: Shows ray path visualization
  - Hypothesis (Play only): Shows marked hypothesis positions

3.4 Models

We evaluated five Claude model variants spanning two generations:

- **Claude Haiku 4.5** (`claude-haiku-4-5-20251001`): Smallest, fastest
- **Claude Sonnet 4.5** (`claude-sonnet-4-5-20250929`): Mid-tier
- **Claude Opus 4.5** (`claude-opus-4-5-20251101`): Largest 4.5 variant
- **Claude Sonnet 4.6** (`claude-sonnet-4-6`): Mid-tier, next generation
- **Claude Opus 4.6** (`claude-opus-4-6`): Largest, most capable

The inclusion of both 4.5 and 4.6 generation models enables within-family comparison of how architectural and training improvements affect diagnostic reasoning capabilities. All models were accessed through the Anthropic API with extended thinking enabled to capture reasoning processes.

3.5 Experimental Design

The experiment uses a factorial design with factors that differ slightly between Predict and Play modes. Ten fixed atom configurations ensure reproducibility across conditions.

3.5.1 Predict Mode Factors

Factor	Levels	Description
Model	Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	Model variant
Prompt Style	Baseline, Augmented	Human-equivalent vs. detailed strategy guidance
Extended Thinking	Yes (with budget), No	Enable extended thinking with token budget
VoT Ray Trace	Yes, No	Include ray trace visualization in prompt
Atom Configuration	10 fixed configurations	Predetermined atom placements for reproducibility

Model	Prompt	Extended Thinking	Vot (None, B Ray Trace)
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**No**	**None**
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**baseline**	**No**	**VoT B Ray Trace Visualization**
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**baseline**	**10K**	**None**
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**baseline**	**10K**	**VoT B Ray Trace Visualization**
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**augmented**	**No**	**None**
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**augmented**	**No**	**VoT B Ray Trace Visualization**

Model	Prompt	Extended Thinking	Vot (None, B Ray Trace)
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**augmented**	**10K**	**None**
***Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6**	**augmented**	**10K**	**VoT B Ray Trace Visualization**

3.5.2 Play Mode Factors

Play Mode Experiment Log

Table 5: Play Mode Experiment Log

Model	Prompt	Extended Thinking	VoT (None, A, B, or C)
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**No**	**None**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**No**	**VoT A Grid State Tracking**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**No**	**VoT B Ray Trace Visualization**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**No**	**VoT C Hypothesis Verification**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**10K**	**None**

Model	Prompt	Extended Thinking	VoT (None, A, B, or C)
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**10K**	**VoT A Grid State Tracking**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**10K**	**VoT B Ray Trace Visualization**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**baseline**	**10K**	**VoT C Hypothesis Verification**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**No**	**None**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**No**	**VoT A Grid State Tracking**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**No**	**VoT B Ray Trace Visualization**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**No**	**VoT C Hypothesis Verification**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**10K**	**None**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**10K**	**VoT A Grid State Tracking**
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**10K**	**VoT B Ray Trace Visualization**

Model	Prompt	Extended Thinking	VoT (None, A, B, or C)
Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	**augmented**	**10K**	**VoT C Hypothesis Verification**

Play mode includes all Predict mode factors plus additional factors for hypothesis management:

Factor	Levels	Description
Model	Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6	Model variant
Prompt Style	Baseline, Augmented	Human-equivalent vs. detailed strategy guidance
Include Visualization	Yes (held constant)	Text board visualization in prompt (enabled for all runs)
Extended Thinking	Yes (with budget), No	Enable extended thinking with token budget
Allow Hypotheses	Yes, No	Enable mark/unmark/check actions for hypothesis tracking
VoT Condition	None, A: Grid State, B: Ray Trace, C: Hypothesis	Visualization of Thought prompt addition (mutually exclusive): A shows current grid state, B includes ray trace visualization, C shows hypothesized atom positions
Atom Configuration	10 fixed configurations	Predetermined atom placements for reproducibility

3.5.3 Atom Configurations

Ten fixed atom configurations are used for reproducible experiments:

Config	Pattern	Positions
1	Spread	(2,3), (3,6), (6,2), (7,7)
2	Cluster in corner	(1,1), (1,3), (2,2), (5,6)
3	Diagonal	(2,2), (4,4), (6,6), (8,8)
4	Edge-heavy	(1,4), (4,8), (8,5), (5,1)
5	Central cluster	(3,4), (4,3), (4,5), (5,4)
6	L-shape	(2,2), (2,3), (2,4), (4,2)
7	Corners	(1,1), (1,8), (8,1), (8,8)
8	Asymmetric	(2,7), (3,2), (6,5), (7,3)
9	Row cluster	(4,2), (4,4), (4,6), (4,8)
10	Mixed	(1,5), (3,3), (5,7), (8,2)

3.5.4 Predict Mode Procedure

For each experimental condition (model × prompt style × visualization × thinking × VoT):

1. For each of the 10 fixed atom configurations:
    a. For each of 32 possible ray entry points:
        - Present atom positions and ray entry to LLM
        - Record predicted outcome (exit position, absorption, or reflection)
        - Compare to ground-truth ray trace
    b. Compute accuracy metrics for this configuration
2. Aggregate accuracy across configurations

3.5.5 Play Mode Procedure

For each experimental condition (model × prompt style × visualization × thinking × hypotheses × VoT options):

1. For each of the 10 fixed atom configurations:
    a. Initialize game with hidden atom positions
    b. LLM selects rays iteratively (max 20)
    c. After each ray, provide outcome feedback
    d. If hypotheses enabled, LLM can mark/unmark suspected positions
    e. LLM submits final guess of 4 atom positions (via guess or check action)
    f. Record: rays used, invalid moves, atoms correct, reasoning trace, thinking
2. Aggregate performance metrics across configurations

3.6 Dependent Variables

3.6.1 Predict Mode

- **Accuracy**: Proportion of rays with correct outcome prediction
- **Error types**: Absorption/deflection/reflection confusion patterns
- **Response time**: API response latency (ms)
- **Token usage**: Input and output tokens per prediction

3.6.2 Play Mode

- **Atoms correct**: 0-4 atoms in correct positions
- **Rays used**: Number of rays before guessing
- **Invalid moves**: Attempts to fire into used positions
- **Efficiency**: Atoms correct / rays used
- **Score**: Game score (lower is better: ray costs + 5 $\times$ missed atoms)
- **Response time**: Total API response time (ms)
- **Token usage**: Total input and output tokens

3.6.3 Process Measures (from Extended Thinking)

- **Thinking traces**: Full extended thinking content when enabled
- **Reasoning explanations**: Model's stated reasoning for each action
- **Hypothesis tracking**: Marked positions over time (when enabled)
- **Constraint tracking accuracy**: Quality of constraint maintenance
- **Hypothesis revision patterns**: How beliefs change with new evidence

3.7 Analysis Plan

3.7.1 Statistical Approach

Unlike most LLM benchmarks that report only accuracy rankings, we employ factorial ANOVA with effect size estimation to quantify the unique contribution of each experimental factor. This approach allows us to determine not just *which* conditions produce better performance, but *how much* each factor contributes and whether effects depend on problem complexity (interaction effects). While less common in LLM evaluation, this statistical framework is standard in experimental psychology and provides stronger inferences about the mechanisms underlying diagnostic reasoning failures.

We complement ANOVA with regression analysis to provide interpretable effect magnitudes:

- **ANOVA**: Tests omnibus hypotheses (does model matter? does prompt style matter?) and interaction effects
- **Regression**: Quantifies effect magnitudes (how much does Opus improve over Haiku?) and allows inclusion of continuous covariates (spatial complexity)
- **Effect sizes**: Standardized metrics (², Cohen's d, odds ratios) allow comparison across studies and DVs

3.7.2 Primary Analyses

1. **Predict Mode**: Logistic regression with Type III ANOVA on prediction accuracy (binary outcome)
2. **Play Mode**: Linear regression with Type III ANOVA on atoms correct (continuous outcome)
3. **Scaling analysis**: Linear trend across model sizes
4. **Effect modification**: Test whether treatment effects vary with spatial complexity (mean atoms per ray)

3.7.3 Effect Size Interpretation Guidelines

Following Cohen (1988), we interpret effect sizes using conventional benchmarks:

Measure	Small	Medium	Large	Notes
² (Eta-squared)	0.01	0.06	0.14	Total variance explained
²p (Partial eta-squared)	0.01	0.06	0.14	Variance explained controlling for other factors
² (Omega-squared)	0.01	0.06	0.14	Unbiased estimate (preferred over ²)
Cohen's d	0.20	0.50	0.80	Standardized mean difference
Cohen's f	0.10	0.25	0.40	Effect size for ANOVA (f = √[²/(1- ²)])
Cramér's V	0.10	0.30	0.50	Association strength for categorical variables

Measure	Small	Medium	Large	Notes
Odds Ratio	1.5	3.0	9.0	OR = 1 means no effect; > 1 increases odds

Note: These are heuristics, not absolute cutoffs. Practical significance depends on context and cost-benefit considerations.

3.7.4 Qualitative Analysis

Systematic coding of extended thinking traces to identify: - Constraint tracking errors - Spatial reasoning errors - Hypothesis generation patterns - Response to contradictory evidence

3.7.5 Spatial-Inferential Decomposition Analysis

To address the concern that spatial reasoning requirements may confound interpretation for non-spatial diagnostic tasks, we employ three complementary analytical strategies:

1. Mode Comparison

Comparing Predict mode (forward reasoning, heavily spatial) to Play mode (full diagnostic loop) performance provides diagnostic information:

- *Good Predict + Poor Play*: Implicates non-spatial limitations (abductive inference, constraint tracking, experiment design)
- *Poor Predict + Poor Play*: Implicates spatial/world model limitations as at least partly responsible
- *Performance delta* (Play accuracy conditional on Predict accuracy): Isolates the contribution of inferential machinery

2. Spatial Complexity Moderation

We use two complementary measures of spatial complexity:

- **Predict mode**: *Cells traveled*—the total number of cells a ray traverses (entry cell + internal path + exit cell). This measures path length complexity.
- **Play mode**: *Mean atoms per ray*—the average number of atoms affecting each ray's path in a game. This measures interaction complexity.

Moderation analysis tests whether treatment effects (model, prompt style, extended thinking) interact with spatial complexity:

- *Constant effects across complexity*: Limitations lie in inferential machinery, not spatial processing
- *Effects moderated by complexity*: Spatial reasoning contributes to observed failures

3. Error Taxonomy from Thinking Traces

We classify errors from extended thinking traces into categories:

Error Type	Spatial?	Description
Ray path error	Yes	Path traced incorrectly through grid
Deflection geometry error	Yes	Misunderstood deflection rules
Constraint tracking error	No	Failed to update ruled-out positions
Experiment design error	No	Fired uninformative/redundant rays
Belief updating error	No	Did not revise hypothesis when contradicted
Premature commitment	No	Anchored on hypothesis before sufficient evidence

The proportion of spatial vs. non-spatial errors indicates which component capabilities drive overall performance.

4 Results

4.1 Predict Mode Results

```
# Load all Predict mode experiment JSON files from Experiment 1/Predict/
predict_json_files <- list.files(
  path = "Experiment 1/Predict",
  pattern = "\\.json$",
  full.names = TRUE
)

# Helper function to count cells traveled by a ray
# Entry cell (outside box) + internal path cells + exit cell (outside box)
count_ray_cells <- function(rayResult) {
```

```
  if (is.null(rayResult) || is.null(rayResult$path)) {
    return(NA_real_)
  }

  path <- rayResult$path
  if (!is.matrix(path) || nrow(path) == 0) {
    return(NA_real_)
  }

  # Start with entry cell (outside box) + all internal path cells
  cells <- 1 + nrow(path)

  # Add exit cell (outside box) for rays that exit (not absorbed)
  # Absorbed rays have absorbed=TRUE and no exit field
  # Deflected/reflected rays have absorbed=FALSE and an exit field
  if (!is.null(rayResult$absorbed) && !rayResult$absorbed) {
    cells <- cells + 1
  }

  return(cells)
}

# Helper function: count atoms adjacent (within 1 cell) to any cell in ray path
# Note: jsonlite parses arrays as matrices, so atom_config is 4x2 and path is Nx2
count_atoms_affecting <- function(path, atom_config) {
  if (is.null(path) || length(path) == 0) return(0)
  if (!is.matrix(path)) return(0)

  affected <- 0
  n_atoms <- nrow(atom_config)
  n_cells <- nrow(path)

  for (i in seq_len(n_atoms)) {
    atom_row <- atom_config[i, 1]
    atom_col <- atom_config[i, 2]
    for (j in seq_len(n_cells)) {
      cell_row <- path[j, 1]
      cell_col <- path[j, 2]
      if (abs(atom_row - cell_row) <= 1 && abs(atom_col - cell_col) <= 1) {
        affected <- affected + 1
        break  # Count each atom only once per ray
      }
```

```
    }
  }
  affected
}

# Function to extract results from a single Predict JSON file
extract_predict_results <- function(json_file) {
  data <- jsonlite::fromJSON(json_file, simplifyDataFrame = FALSE)

  # Extract results array
  results <- data$results

  # Convert each result to prediction-level rows
  map_dfr(results, function(r) {
    # Each result has a predictions array
    predictions <- r$predictions

    if (is.null(predictions) || length(predictions) == 0) {
      return(tibble())
    }

    # Extract each prediction
    map_dfr(predictions, function(p) {
      # Calculate path complexity metrics
      cells_traveled <- count_ray_cells(p$rayResult)
      atoms_affecting <- count_atoms_affecting(p$rayResult$path, r$atomConfig)

      tibble(
        experiment_id = r$experimentId,
        model = r$model,
        model_name = r$modelName,
        prompt_style = r$promptStyle,
        prompt_condition = r$promptCondition,
        config_index = r$configIndex,
        include_visualization = r$includeVisualization,
        enable_thinking = r$enableThinking,
        thinking_budget = r$thinkingBudget,
        vot_ray_trace = r$votRayTrace,
        ray_entry = p$rayEntry,
        predicted = p$predicted,
        actual = p$actual,
        correct = p$correct,
```

```
        reasoning = p$reasoning,
        response_time_ms = p$responseTimeMs,
        input_tokens = p$inputTokens,
        output_tokens = p$outputTokens,
        cells_traveled = cells_traveled,
        atoms_affecting = atoms_affecting,
        source_file = basename(json_file)
      )
    })
  })
}

# Load all results into a single long-format tibble
predict_data <- map_dfr(predict_json_files, extract_predict_results)
```

Data loaded: 18800 predictions from 24 JSON files.

- **Models**: Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6
- **Prompt styles**: augmented, baseline
- **Extended thinking**: FALSE, TRUE
- **VoT ray trace**: FALSE, TRUE
- **Configurations tested**: 10
- **Cells traveled range**: 2–13
- **Atoms affecting range**: 0–3

```
### Data Completeness Check

**Note on sample size**: Fewer than 32 predictions per configuration is expected due to ray s


::: {.cell}

```{.r .cell-code}
predictions_per_config <- predict_data |>
  group_by(model_name, prompt_style, enable_thinking, vot_ray_trace, config_index) |>
  summarise(n_predictions = n(), .groups = "drop")

# Create matrix: rows = experimental conditions, columns = configurations
# Since all models should have same N per config, take first model's values
predictions_matrix <- predictions_per_config |>
  filter(model_name == first(model_name)) |>
```
```

```
  mutate(
    condition = paste0(
      substr(prompt_style, 1, 3), "_",
      ifelse(enable_thinking, "think", "no"), "_",
      ifelse(vot_ray_trace, "vot", "no")
    )
  ) |>
  select(condition, config_index, n_predictions) |>
  pivot_wider(names_from = config_index, values_from = n_predictions, names_prefix = "cfg_")

kable(predictions_matrix,
      caption = "Predictions per Configuration by Experimental Condition",
      col.names = c("Condition", paste0("Config ", 0:9)))
```

Table 10: Predictions per Configuration by Experimental Condition

Condition	Config 0	Config 1	Config 2	Config 3	Config 4	Config 5	Config 6	Config 7	Config 8	Config 9
aug_no_no	46	44	50	56	36	42	48	46	50	52
aug_no_vot	46	44	50	56	36	42	48	46	50	52
aug_think_no	46	44	50	56	36	42	48	46	50	52
aug_think_vot	46	44	50	56	36	42	48	46	50	52
bas_no_no	46	44	50	56	36	42	48	46	50	52
bas_no_vot	46	44	50	56	36	42	48	46	50	52
bas_think_no	46	44	50	56	36	42	48	46	50	52
bas_think_vot	46	44	50	56	36	42	48	46	50	52

```
# Check for consistent N across models within each condition
consistency <- predictions_per_config |>
  group_by(prompt_style, enable_thinking, vot_ray_trace, config_index) |>
  summarise(
    n_unique = n_distinct(n_predictions),
    min_n = min(n_predictions),
    max_n = max(n_predictions),
    .groups = "drop"
  )

# Check for duplicate runs (same condition run multiple times)
runs_per_condition <- predict_data |>
  group_by(prompt_style, enable_thinking, vot_ray_trace, model_name, config_index) |>
  summarise(n_runs = n_distinct(source_file), .groups = "drop")
```

:::

Consistency: All models have consistent N per configuration within each condition.

Duplicate runs: No duplicate runs detected.

```
# Define expected full factorial design
expected_predict_models <- c("Haiku 4.5", "Sonnet 4.5", "Opus 4.5", "Sonnet 4.6", "Opus 4.6")
expected_predict_design <- expand_grid(
  model_name = expected_predict_models,
  prompt_style = c("baseline", "augmented"),
  enable_thinking = c(FALSE, TRUE),
  vot_ray_trace = c(FALSE, TRUE),
  config_index = 0:9
)

# What's actually present
actual_predict_conditions <- predict_data |>
  distinct(model_name, prompt_style, enable_thinking, vot_ray_trace, config_index)

# Find missing cells
missing_predict <- anti_join(
  expected_predict_design, actual_predict_conditions,
  by = c("model_name", "prompt_style", "enable_thinking", "vot_ray_trace", "config_index")
)

# Summarise missing by condition (collapse config indices)
missing_predict_summary <- missing_predict |>
  group_by(model_name, prompt_style, enable_thinking, vot_ray_trace) |>
  summarise(
    n_missing_configs = n(),
    missing_configs = paste(config_index, collapse = ", "),
    .groups = "drop"
  ) |>
  arrange(model_name, prompt_style, enable_thinking, vot_ray_trace)

# Condition-level coverage matrix (how many of 10 configs present per condition)
predict_coverage <- expected_predict_design |>
  distinct(model_name, prompt_style, enable_thinking, vot_ray_trace) |>
  left_join(
    predict_data |>
      group_by(model_name, prompt_style, enable_thinking, vot_ray_trace) |>
      summarise(configs_present = n_distinct(config_index),
                total_predictions = n(), .groups = "drop"),
```

```
    by = c("model_name", "prompt_style", "enable_thinking", "vot_ray_trace")
  ) |>
  replace_na(list(configs_present = 0, total_predictions = 0)) |>
  mutate(
    condition = paste0(
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      ifelse(vot_ray_trace, "VoT", "No-VoT")
    )
  )

# Pivot to show models as columns
predict_coverage_matrix <- predict_coverage |>
  select(model_name, condition, configs_present) |>
  pivot_wider(names_from = model_name, values_from = configs_present) |>
  arrange(condition)

if (nrow(missing_predict) > 0) {
  cat("**Missing experimental runs detected.**\n\n")
  kable(predict_coverage_matrix,
        caption = "Predict Mode: Configurations Present (out of 10) by Model and Condition",
        format = "pipe") |> print()
  cat("\n\n**Missing conditions detail:**\n\n")
  kable(missing_predict_summary,
        col.names = c("Model", "Prompt", "Thinking", "VoT Ray", "Missing (n)", "Config Indic
        caption = "Predict Mode: Missing Experimental Runs",
        format = "pipe") |> print()
} else {
  cat("All", nrow(expected_predict_design), "expected predict-mode cells are present",
      "(5 models x 2 prompts x 2 thinking x 2 VoT x 10 configs).\n")
}
```

```
All 400 expected predict-mode cells are present (5 models x 2 prompts x 2 thinking x 2 VoT x
```

```
# Check for unexpected model names not in the design
unexpected_models_predict <- setdiff(unique(predict_data$model_name), expected_predict_models
if (length(unexpected_models_predict) > 0) {
  cat("\n**Unexpected model names in data:**", paste(unexpected_models_predict, collapse = "
}
```

All expected predict-mode experimental runs are present.

```r
# Summary statistics by all experimental factors
predict_summary <- predict_data |>
  group_by(model_name, prompt_style, enable_thinking, vot_ray_trace) |>
  summarise(
    n = n(),
    n_correct = sum(correct),
    accuracy = mean(correct),
    accuracy_se = sqrt(accuracy * (1 - accuracy) / n),
    mean_response_time = mean(response_time_ms, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(across(where(is.numeric) & !c(n, n_correct), \(x) round(x, 3)))

kable(predict_summary,
      col.names = c("Model", "Prompt", "Thinking", "VoT Ray", "N", "Correct",
                    "Accuracy", "SE", "Response Time (ms)"),
      caption = "Predict Mode Performance by Experimental Condition")
```

Table 11: Predict Mode Performance by Experimental Condition

Model	Prompt	Thinking	VoT Ray	N	Correct	Accuracy	SE	Response Time (ms)
Haiku 4.5	augmented	FALSE	FALSE	470	352	0.749	0.020	10740.27
Haiku 4.5	augmented	FALSE	TRUE	470	384	0.817	0.018	12758.47
Haiku 4.5	augmented	TRUE	FALSE	470	426	0.906	0.013	20242.76
Haiku 4.5	augmented	TRUE	TRUE	470	450	0.957	0.009	23141.35
Haiku 4.5	baseline	FALSE	FALSE	470	270	0.574	0.023	11026.42
Haiku 4.5	baseline	FALSE	TRUE	470	248	0.528	0.023	11719.38
Haiku 4.5	baseline	TRUE	FALSE	470	288	0.613	0.022	39523.49
Haiku 4.5	baseline	TRUE	TRUE	470	366	0.779	0.019	66692.71
Opus 4.5	augmented	FALSE	FALSE	470	384	0.817	0.018	10909.51
Opus 4.5	augmented	FALSE	TRUE	470	388	0.826	0.018	12618.99
Opus 4.5	augmented	TRUE	FALSE	470	440	0.936	0.011	20568.46

Model	Prompt	Thinking	VoT Ray	N	Correct	Accuracy	SE	Response Time (ms)
Opus 4.5	augmented	TRUE	TRUE	470	440	0.936	0.011	23033.41
Opus 4.5	baseline	FALSE	FALSE	470	266	0.566	0.023	11015.83
Opus 4.5	baseline	FALSE	TRUE	470	248	0.528	0.023	11500.29
Opus 4.5	baseline	TRUE	FALSE	470	298	0.634	0.022	40646.76
Opus 4.5	baseline	TRUE	TRUE	470	368	0.783	0.019	69663.18
Opus 4.6	augmented	FALSE	FALSE	470	352	0.749	0.020	10819.92
Opus 4.6	augmented	FALSE	TRUE	470	380	0.809	0.018	12121.75
Opus 4.6	augmented	TRUE	FALSE	470	436	0.928	0.012	21100.77
Opus 4.6	augmented	TRUE	TRUE	470	448	0.953	0.010	24972.77
Opus 4.6	baseline	FALSE	FALSE	470	278	0.591	0.023	10535.14
Opus 4.6	baseline	FALSE	TRUE	470	250	0.532	0.023	11202.86
Opus 4.6	baseline	TRUE	FALSE	470	354	0.753	0.020	67770.07
Opus 4.6	baseline	TRUE	TRUE	470	362	0.770	0.019	66409.90
Sonnet 4.5	augmented	FALSE	FALSE	470	366	0.779	0.019	10893.54
Sonnet 4.5	augmented	FALSE	TRUE	470	400	0.851	0.016	12927.71
Sonnet 4.5	augmented	TRUE	FALSE	470	448	0.953	0.010	20636.58
Sonnet 4.5	augmented	TRUE	TRUE	470	450	0.957	0.009	23033.25
Sonnet 4.5	baseline	FALSE	FALSE	470	278	0.591	0.023	11023.54
Sonnet 4.5	baseline	FALSE	TRUE	470	240	0.511	0.023	12235.34
Sonnet 4.5	baseline	TRUE	FALSE	470	292	0.621	0.022	41686.14
Sonnet 4.5	baseline	TRUE	TRUE	470	370	0.787	0.019	68985.51
Sonnet 4.6	augmented	FALSE	FALSE	470	390	0.830	0.017	10247.92
Sonnet 4.6	augmented	FALSE	TRUE	470	374	0.796	0.019	17046.00
Sonnet 4.6	augmented	TRUE	FALSE	470	442	0.940	0.011	20267.15

Model	Prompt	Thinking	VoT Ray	N	Correct	Accuracy	SE	Response Time (ms)
Sonnet 4.6	augmented	TRUE	TRUE	470	454	0.966	0.008	23906.89
Sonnet 4.6	baseline	FALSE	FALSE	470	276	0.587	0.023	10784.63
Sonnet 4.6	baseline	FALSE	TRUE	470	254	0.540	0.023	10592.90
Sonnet 4.6	baseline	TRUE	FALSE	470	368	0.783	0.019	65685.45
Sonnet 4.6	baseline	TRUE	TRUE	470	354	0.753	0.020	65603.62

4.1.1 Predict Mode Leaderboard

Top 10 configurations by overall accuracy. Format matches typical LLM benchmark reporting for cross-study comparison.

```
leaderboard_predict <- predict_data |>
  mutate(
    condition = paste0(
      model_name, " | ",
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      ifelse(vot_ray_trace, "VoT", "No-VoT")
    )
  ) |>
  group_by(condition, model_name, prompt_style, enable_thinking, vot_ray_trace) |>
  summarise(
    n = n(),
    accuracy = mean(correct),
    ci_lower = accuracy - 1.96 * sqrt(accuracy * (1 - accuracy) / n),
    ci_upper = accuracy + 1.96 * sqrt(accuracy * (1 - accuracy) / n),
    .groups = "drop"
  ) |>
  arrange(desc(accuracy)) |>
  mutate(
    rank = row_number(),
    accuracy_pct = paste0(round(accuracy * 100, 1), "%"),
    ci_95 = paste0("[", round(ci_lower * 100, 1), "%, ", round(ci_upper * 100, 1), "%]")
  ) |>
```

```
  select(rank, condition, accuracy_pct, ci_95, n) |>
  head(10)

kable(leaderboard_predict,
      col.names = c("Rank", "Configuration", "Accuracy", "95% CI", "N"),
      caption = "Predict Mode Leaderboard (Top 10 Configurations)")
```

Table 12: Predict Mode Leaderboard (Top 10 Configurations)

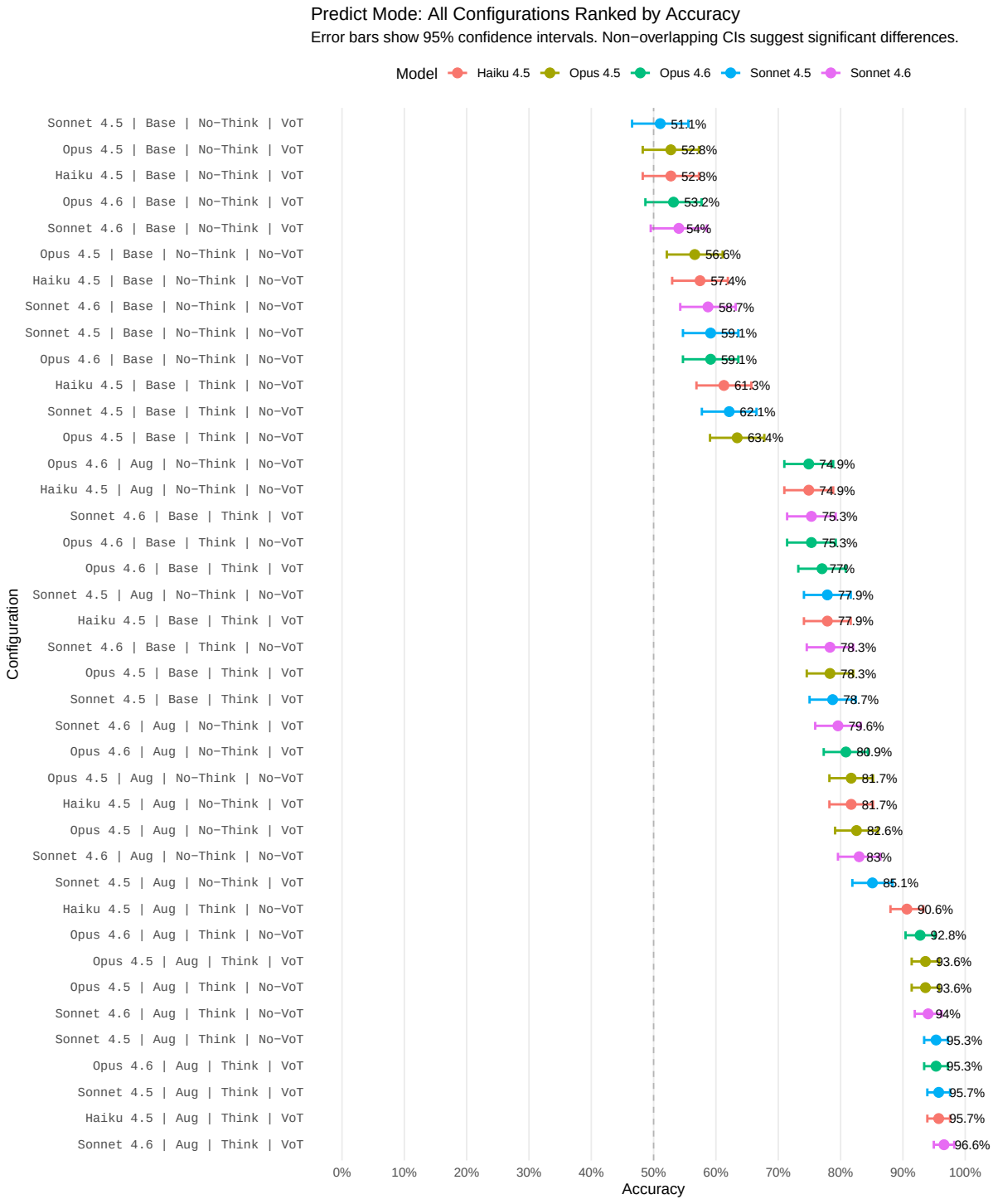
Rank	Configuration	Accuracy	95% CI	N
1	Sonnet 4.6 | Aug | Think | VoT	96.6%	[95%, 98.2%]	470
2	Haiku 4.5 | Aug | Think | VoT	95.7%	[93.9%, 97.6%]	470
3	Sonnet 4.5 | Aug | Think | VoT	95.7%	[93.9%, 97.6%]	470
4	Opus 4.6 | Aug | Think | VoT	95.3%	[93.4%, 97.2%]	470
5	Sonnet 4.5 | Aug | Think | No-VoT	95.3%	[93.4%, 97.2%]	470
6	Sonnet 4.6 | Aug | Think | No-VoT	94%	[91.9%, 96.2%]	470
7	Opus 4.5 | Aug | Think | No-VoT	93.6%	[91.4%, 95.8%]	470
8	Opus 4.5 | Aug | Think | VoT	93.6%	[91.4%, 95.8%]	470
9	Opus 4.6 | Aug | Think | No-VoT	92.8%	[90.4%, 95.1%]	470
10	Haiku 4.5 | Aug | Think | No-VoT	90.6%	[88%, 93.3%]	470

4.1.2 Forest Plot: All Configurations

Visual comparison of all experimental factor combinations. Non-overlapping confidence intervals suggest statistically significant differences.

```
# Calculate accuracy and 95% CI for all configurations
forest_data <- predict_data |>
  mutate(
    # Create readable configuration labels
    config_label = paste0(
      model_name, " | ",
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      ifelse(vot_ray_trace, "VoT", "No-VoT")
    )
  ) |>
  group_by(config_label, model_name, prompt_style, enable_thinking, vot_ray_trace) |>
  summarise(
    n = n(),
```

```
    accuracy = mean(correct),
    # 95% CI for proportion using normal approximation
    ci_lower = accuracy - 1.96 * sqrt(accuracy * (1 - accuracy) / n),
    ci_upper = accuracy + 1.96 * sqrt(accuracy * (1 - accuracy) / n),
    .groups = "drop"
  ) |>
  # Order by accuracy (highest first)
  arrange(desc(accuracy)) |>
  mutate(
    # Create ordered factor for plotting
    config_label = factor(config_label, levels = config_label)
  )

# Forest plot
p_forest <- ggplot(forest_data, aes(x = accuracy, y = config_label)) +
  geom_point(aes(color = model_name), size = 3) +
  geom_errorbarh(aes(xmin = ci_lower, xmax = ci_upper, color = model_name),
                 height = 0.3, linewidth = 0.8) +
  geom_vline(xintercept = 0.5, linetype = "dashed", color = "gray50", alpha = 0.5) +
  geom_text(aes(label = paste0(round(accuracy * 100, 1), "%")),
            hjust = -0.3, size = 3) +
  labs(
    title = "Predict Mode: All Configurations Ranked by Accuracy",
    subtitle = "Error bars show 95% confidence intervals. Non-overlapping CIs suggest signif
    x = "Accuracy",
    y = "Configuration",
    color = "Model"
  ) +
  scale_x_continuous(
    limits = c(0, 1),
    labels = scales::percent,
    breaks = seq(0, 1, 0.1)
  ) +
  theme_minimal(base_size = 11) +
  theme(
    axis.text.y = element_text(size = 9, family = "mono"),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank(),
    legend.position = "top"
  )

print(p_forest)
```



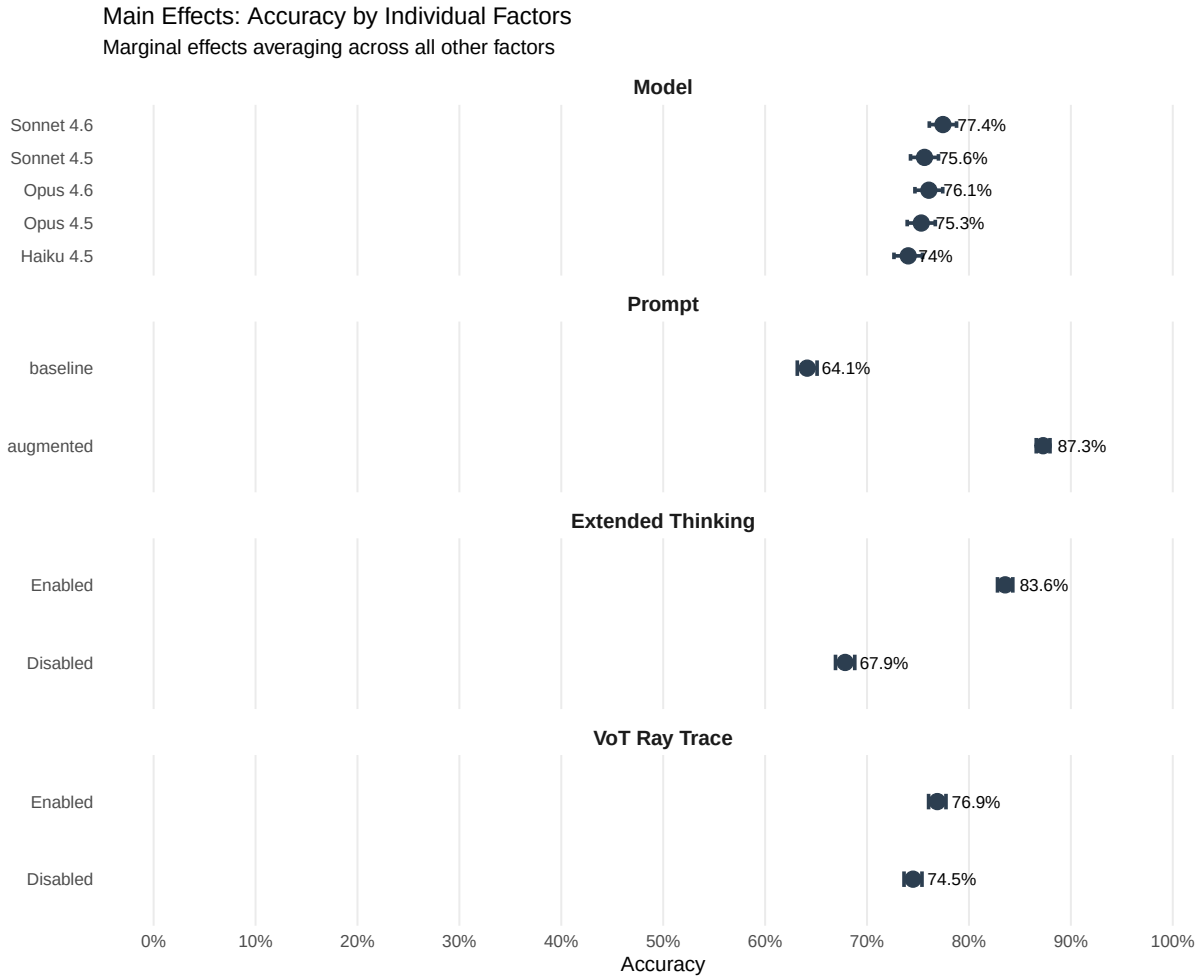
Total configurations: 40 **Best performing**: Sonnet 4.6 | Aug | Think | VoT **Accuracy**: 96.6% (95% CI: [95%, 98.2%])

4.1.3 Factor-Level Comparisons

Shows accuracy by individual factors, collapsing across other dimensions (marginal effects).

```
# Calculate marginal effects for each factor
factor_data <- list(
  Model = predict_data |>
    group_by(model_name) |>
    summarise(
      n = n(),
      accuracy = mean(correct),
      ci_lower = accuracy - 1.96 * sqrt(accuracy * (1 - accuracy) / n),
      ci_upper = accuracy + 1.96 * sqrt(accuracy * (1 - accuracy) / n)
    ) |>
    mutate(factor = "Model", level = model_name),

  Prompt = predict_data |>
    group_by(prompt_style) |>
    summarise(
      n = n(),
      accuracy = mean(correct),
      ci_lower = accuracy - 1.96 * sqrt(accuracy * (1 - accuracy) / n),
      ci_upper = accuracy + 1.96 * sqrt(accuracy * (1 - accuracy) / n)
    ) |>
    mutate(factor = "Prompt", level = prompt_style),

  Thinking = predict_data |>
    group_by(enable_thinking) |>
    summarise(
      n = n(),
      accuracy = mean(correct),
      ci_lower = accuracy - 1.96 * sqrt(accuracy * (1 - accuracy) / n),
      ci_upper = accuracy + 1.96 * sqrt(accuracy * (1 - accuracy) / n)
    ) |>
    mutate(factor = "Extended Thinking", level = ifelse(enable_thinking, "Enabled", "Disable

  VoT = predict_data |>
    group_by(vot_ray_trace) |>
    summarise(
      n = n(),
      accuracy = mean(correct),
      ci_lower = accuracy - 1.96 * sqrt(accuracy * (1 - accuracy) / n),
      ci_upper = accuracy + 1.96 * sqrt(accuracy * (1 - accuracy) / n)
```

```
    ) |>
    mutate(factor = "VoT Ray Trace", level = ifelse(vot_ray_trace, "Enabled", "Disabled"))
) |>
  bind_rows() |>
  mutate(factor = factor(factor, levels = c("Model", "Prompt", "Extended Thinking", "VoT Ray

# Factor comparison plot
p_factors <- ggplot(factor_data, aes(x = accuracy, y = level)) +
  geom_point(size = 4, color = "#2C3E50") +
  geom_errorbarh(aes(xmin = ci_lower, xmax = ci_upper),
                 height = 0.2, linewidth = 1, color = "#2C3E50") +
  geom_text(aes(label = paste0(round(accuracy * 100, 1), "%")),
            hjust = -0.3, size = 3.5) +
  facet_wrap(~factor, ncol = 1, scales = "free_y") +
  labs(
    title = "Main Effects: Accuracy by Individual Factors",
    subtitle = "Marginal effects averaging across all other factors",
    x = "Accuracy",
    y = NULL
  ) +
  scale_x_continuous(
    limits = c(0, 1),
    labels = scales::percent,
    breaks = seq(0, 1, 0.1)
  ) +
  theme_minimal(base_size = 12) +
  theme(
    strip.text = element_text(face = "bold", size = 12),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank()
  )

print(p_factors)
```




```
# Accuracy by configuration to assess difficulty variation
predict_by_config <- predict_data |>
  group_by(config_index, model_name) |>
  summarise(
    accuracy = mean(correct),
    .groups = "drop"
  ) |>
  pivot_wider(names_from = model_name, values_from = accuracy) |>
  mutate(across(where(is.numeric), \(x) round(x, 3)))

kable(predict_by_config,
      caption = "Prediction Accuracy by Configuration and Model")
```

Table 13: Prediction Accuracy by Configuration and Model

config_index	Haiku 4.5	Opus 4.5	Opus 4.6	Sonnet 4.5	Sonnet 4.6
0	0.636	0.614	0.652	0.663	0.641
1	0.761	0.750	0.784	0.773	0.778
2	0.760	0.820	0.775	0.785	0.790
3	0.719	0.692	0.714	0.705	0.732
4	0.729	0.785	0.792	0.771	0.819
5	0.804	0.845	0.810	0.839	0.821
6	0.844	0.833	0.828	0.844	0.839
7	0.630	0.696	0.717	0.685	0.755
8	0.775	0.765	0.785	0.785	0.825
9	0.745	0.750	0.764	0.731	0.760

4.1.4 ANOVA: Prediction Accuracy

This analysis uses mixed-effects logistic regression to test whether experimental factors significantly affect prediction accuracy, while accounting for clustering within configurations and individual rays.

Model specification: - **Outcome**: Correct prediction (binary: 0/1) - **Fixed effects**: Model, prompt style, extended thinking, VoT ray trace, and their interactions - **Covariates**: cells_traveled (path length), atoms_affecting (atom density) - **Random effects**: Nested structure with config_index (configuration-level) and ray_id nested within config (ray-level)

Why nested random effects?

- **Configuration-level (config_index)**: Accounts for overall difficulty due to specific atom placements; allows generalization beyond these 10 configurations
- **Ray-level (ray_id:config_index)**: Accounts for individual ray difficulty; handles correlation when the same ray is tested across multiple experimental conditions
- **Why nested?**: ray_id "NORTH_3" represents different physical rays across configurations (different atom placements), so must be nested, not crossed

Covariates (cells_traveled, atoms_affecting) are **NOT confounders** (same rays tested across all conditions) but **ARE precision variables** that explain variance and improve statistical power.

```
# Convert factors for ANOVA
predict_data <- predict_data |>
  mutate(
    model_name = factor(model_name, levels = order_model_names(model_name)),
    prompt_style = factor(prompt_style, levels = c("baseline", "augmented")),
```

```
    enable_thinking = factor(enable_thinking, levels = c(FALSE, TRUE)),
    vot_ray_trace = factor(vot_ray_trace, levels = c(FALSE, TRUE)),
    config_index = factor(config_index),
    # Create unique ray identifier (nested within config)
    ray_id = paste(toupper(ray_entry$side), ray_entry$pos, sep = "_")
  )

# Build model formula dynamically based on available factor levels
has_model_var <- length(unique(predict_data$model_name)) > 1
has_prompt_var <- length(unique(predict_data$prompt_style)) > 1
has_thinking_var <- length(unique(predict_data$enable_thinking)) > 1
has_vot_var <- length(unique(predict_data$vot_ray_trace)) > 1

# Build formula components
formula_parts <- c()
if (has_model_var) formula_parts <- c(formula_parts, "model_name")
if (has_prompt_var) formula_parts <- c(formula_parts, "prompt_style")
if (has_thinking_var) formula_parts <- c(formula_parts, "enable_thinking")

# Add interactions only if all three main effects are present
if (has_model_var && has_prompt_var && has_thinking_var) {
  main_formula <- "model_name * prompt_style * enable_thinking"
} else {
  main_formula <- paste(formula_parts, collapse = " + ")
}

# Add VoT if it varies
if (has_vot_var) {
  full_formula <- paste0("correct ~ ", main_formula, " + vot_ray_trace + cells_traveled + at
} else {
  full_formula <- paste0("correct ~ ", main_formula, " + cells_traveled + atoms_affecting +
}

model_predict <- glmer(as.formula(full_formula),
                       family = binomial(link = "logit"),
                       data = predict_data,
                       control = glmerControl(optimizer = "bobyqa"))
```

Model formula: correct ~ model_name * prompt_style * enable_thinking + vot_ray_trace + cells_traveled + atoms_affecting + (1 | config_index/ray_id)

```
Analysis of Deviance Table (Type III Wald chisquare tests)
```

```
Response: correct
                                        Chisq Df Pr(>Chisq)
(Intercept)                           53.1963  1  3.018e-13 ***
model_name                             1.2163  4  0.8753999
prompt_style                          39.9128  1  2.656e-10 ***
enable_thinking                       49.3321  1  2.161e-12 ***
vot_ray_trace                         22.8939  1  1.712e-06 ***
cells_traveled                       656.5486  1  < 2.2e-16 ***
atoms_affecting                      220.6100  1  < 2.2e-16 ***
model_name:prompt_style                8.2388  4  0.0832128 .
model_name:enable_thinking            16.1965  4  0.0027665 **
prompt_style:enable_thinking          12.8438  1  0.0003386 ***
model_name:prompt_style:enable_thinking  3.4216  4  0.4898995
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.1.4.1 Odds Ratios with 95% Confidence Intervals

Interpretation: OR > 1 means the factor increases odds of correct prediction; OR < 1 means it decreases odds. These are conditional odds ratios (controlling for random effects).

Table 14: Odds Ratios for Prediction Accuracy (Fixed Effects). * indicates p < 0.05.

Term	Odds Ratio	95% CI Lower	95% CI Upper	p-value	Sig
model_nameSonnet 4.5	1.000	0.807	1.239	0.9998	
model_nameSonnet 4.6	1.088	0.878	1.348	0.4430	
model_nameOpus 4.5	0.988	0.798	1.224	0.9134	
model_nameOpus 4.6	1.062	0.857	1.316	0.5841	
prompt_styleaugmented	2.074	1.654	2.601	<0.001	*
enable_thinkingTRUE	2.270	1.806	2.853	<0.001	*
vot_ray_traceTRUE	1.223	1.126	1.328	<0.001	*
cells_traveled	0.810	0.797	0.823	<0.001	*
atoms_affecting	0.623	0.585	0.663	<0.001	*
model_nameSonnet 4.5:prompt_styleaugmented	1.348	0.973	1.868	0.0729	
model_nameSonnet 4.6:prompt_styleaugmented	1.179	0.851	1.633	0.3219	
model_nameOpus 4.5:prompt_styleaugmented	1.387	1.001	1.923	0.0495	*

Term	Odds Ratio	95% CI Lower	95% CI Upper	p-value	Sig
model_nameOpus 4.6:prompt_styleaugmented	0.956	0.694	1.317	0.7826	
model_nameSonnet 4.5:enable_thinkingTRUE	1.064	0.769	1.472	0.7081	
model_nameSonnet 4.6:enable_thinkingTRUE	1.676	1.193	2.354	0.0029	*
model_nameOpus 4.5:enable_thinkingTRUE	1.129	0.815	1.564	0.4642	
model_nameOpus 4.6:enable_thinkingTRUE	1.648	1.175	2.311	0.0038	*
prompt_styleaugmented:enable_thinkingTRUE	2.038	1.381	3.007	<0.001	*
model_nameSonnet 4.5:prompt_styleaugmented:enable_thinkingTRUE	1.081	0.605	1.932	0.7934	
model_nameSonnet 4.6:prompt_styleaugmented:enable_thinkingTRUE	0.685	0.381	1.231	0.2056	
model_nameOpus 4.5:prompt_styleaugmented:enable_thinkingTRUE	0.789	0.448	1.390	0.4124	
model_nameOpus 4.6:prompt_styleaugmented:enable_thinkingTRUE	0.731	0.413	1.292	0.2808	

4.1.4.2 Random Effects Variance Components

Configuration-level random effects (config_index): - Accounts for overall difficulty due to specific atom placements - Generalizes inference beyond these 10 configurations - Captures configuration-specific effects (e.g., dense vs sparse atom patterns)

Ray-level random effects (ray_id:config_index): - Accounts for individual ray difficulty within each configuration - Handles correlation: Same ray tested across multiple conditions - Allows each ray to have its own baseline difficulty - Essential when same rays appear in multiple experimental conditions

```
Groups              Name        Std.Dev.
ray_id:config_index (Intercept) 0.58436
config_index        (Intercept) 0.87481
```

Intraclass Correlation Coefficient (ICC): 0.252 — Proportion of variance due to clustering at both configuration and ray-within-config levels.

Model Fit (Pseudo R^2, Nakagawa & Schielzeth): - **Marginal R^2** (fixed effects only): 0.217 - **Conditional R^2** (fixed + random effects): 0.414

4.1.4.3 Cramér's V (Effect Size for Categorical Associations)

Interpretation guidelines: 0.10=small, 0.30=medium, 0.50=large

Table 15: Association Strength with Prediction Accuracy

Factor	Cramér's V
Model	0.021
Prompt Style	0.270
Extended Thinking	0.183
VoT Ray Trace	0.027

4.1.4.4 Regression Coefficients for Main Effects

Key findings from mixed-effects logistic regression (fixed effects only).

Table 16: Regression Coefficients for Main Effects (Fixed Effects)

Term	(log-odds)	OR	OR 95% CI Lower	OR 95% CI Upper	p-value
model_nameSonnet 4.5	0.000	1.000	0.807	1.239	1.000
model_nameSonnet 4.6	0.084	1.088	0.878	1.348	0.443
model_nameOpus 4.5	-0.012	0.988	0.798	1.224	0.913
model_nameOpus 4.6	0.060	1.062	0.857	1.316	0.584
prompt_styleaug-mented	0.730	2.074	1.654	2.601	<0.001
enable_think-ingTRUE	0.820	2.270	1.806	2.853	<0.001
vot_ray_traceTRUE	0.201	1.223	1.126	1.328	<0.001
cells_traveled	-0.211	0.810	0.797	0.823	<0.001
atoms_affecting	-0.473	0.623	0.585	0.663	<0.001

4.1.5 Effect Modification by Path Complexity

Does path complexity (measured as total cells traveled: entry + internal + exit) moderate treatment effects? If treatment effects are constant across spatial complexity levels, limitations lie in inferential machinery rather than spatial processing. If effects are moderated by complexity, spatial reasoning contributes to observed failures.

```r
# Only run if we have variation in factors and valid cells_traveled data
run_effect_mod <- has_model_var || has_prompt_var || has_thinking_var

if (run_effect_mod) {
  # Build interaction formula
  interaction_parts <- c()
  if (has_model_var) interaction_parts <- c(interaction_parts, "model_name * cells_traveled")
  if (has_prompt_var) interaction_parts <- c(interaction_parts, "prompt_style * cells_travele
  if (has_thinking_var) interaction_parts <- c(interaction_parts, "enable_thinking * cells_t

  # Add nested random effects
  pred_mod_formula <- paste0("correct ~ ", paste(interaction_parts, collapse = " + "), " + (

  model_predict_mod <- glmer(as.formula(pred_mod_formula),
                             family = binomial(link = "logit"),
                             data = predict_data,
                             control = glmerControl(optimizer = "bobyqa"))
}
```

Model formula: correct ~ model_name * cells_traveled + prompt_style * cells_traveled + enable_thinking * cells_traveled + (1 | config_index/ray_id)

```
Analysis of Deviance Table (Type III Wald chisquare tests)

Response: correct
                                Chisq Df Pr(>Chisq)
(Intercept)                   33.7796  1  6.172e-09 ***
model_name                     6.9791  4   0.136997
cells_traveled               191.0674  1  < 2.2e-16 ***
prompt_style                  16.1232  1  5.935e-05 ***
enable_thinking                9.4523  1   0.002109 **
model_name:cells_traveled      5.2759  4   0.260137
cells_traveled:prompt_style   26.1911  1  3.093e-07 ***
cells_traveled:enable_thinking 57.0123  1  4.331e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation of interactions: - **No interaction**: Treatment effect is constant across simple and complex paths - **Positive interaction**: Treatment effect increases with path complexity - **Negative interaction**: Treatment effect decreases with path complexity

Table 17: Path Complexity Interaction Effects

Interaction Term	(log-odds)	OR	OR 95% CI Lower	OR 95% CI Upper	p-value
model_nameSonnet 4.5:cells_traveled	-0.027	0.973	0.928	1.020	0.253
model_nameSonnet 4.6:cells_traveled	-0.008	0.992	0.946	1.041	0.752
model_nameOpus 4.5:cells_traveled	-0.019	0.981	0.936	1.028	0.425
model_nameOpus 4.6:cells_traveled	0.024	1.024	0.977	1.073	0.318
cells_traveled:prompt_styleaugmented	0.081	1.084	1.051	1.119	<0.001
cells_traveled:enable_thinkingTRUE	0.121	1.128	1.093	1.164	<0.001

Path Complexity Distribution (cells traveled):

Statistic	Value
Min	2
Q1	4
Median	6
Q3	9
Max	13
Mean	6.49
SD	2.9

4.1.6 Effect Modification by Atom Density

Does atom density (number of atoms adjacent to ray path) moderate treatment effects?

```
run_effect_mod_atoms <- has_model_var || has_prompt_var || has_thinking_var

if (run_effect_mod_atoms) {
  # Build interaction formula
  interaction_parts <- c()
  if (has_model_var) interaction_parts <- c(interaction_parts, "model_name * atoms_affecting
  if (has_prompt_var) interaction_parts <- c(interaction_parts, "prompt_style * atoms_affecti
  if (has_thinking_var) interaction_parts <- c(interaction_parts, "enable_thinking * atoms_a
```

```
  # Add nested random effects
  pred_mod_formula_atoms <- paste0("correct ~ ", paste(interaction_parts, collapse = " + "),

  model_predict_mod_atoms <- glmer(as.formula(pred_mod_formula_atoms),
                                   family = binomial(link = "logit"),
                                   data = predict_data,
                                   control = glmerControl(optimizer = "bobyqa"))
}
```

Model formula: correct ~ model_name * atoms_affecting + prompt_style * atoms_affecting + enable_thinking * atoms_affecting + (1 | config_index/ray_id)

Mixed-effects logistic regression tests whether treatment effects vary with atom density.

```
Analysis of Deviance Table (Type III Wald chisquare tests)

Response: correct
                                  Chisq Df Pr(>Chisq)
(Intercept)                      1.1755  1  0.2782769
model_name                       0.8500  4  0.9316218
atoms_affecting                 12.3463  1  0.0004419 ***
prompt_style                  1124.3690  1  < 2.2e-16 ***
enable_thinking                 79.3057  1  < 2.2e-16 ***
model_name:atoms_affecting       3.2405  4  0.5184134
atoms_affecting:prompt_style   320.0841  1  < 2.2e-16 ***
atoms_affecting:enable_thinking  55.2104  1  1.083e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation of interactions: - **No interaction**: Treatment effect is constant across low and high atom density - **Positive interaction**: Treatment effect increases with more atoms affecting path - **Negative interaction**: Treatment effect decreases with more atoms affecting path

Table 19: Atom Density Interaction Effects

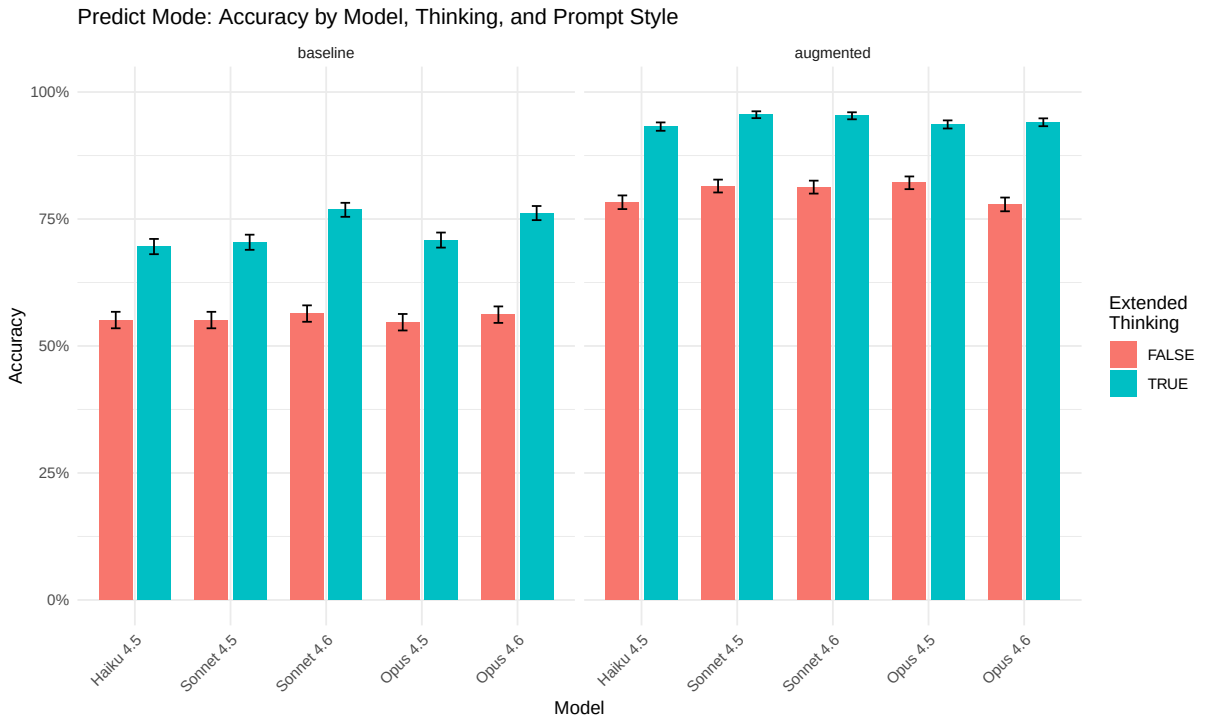
Interaction Term	(log-odds)	OR	OR 95% CI Lower	OR 95% CI Upper	p-value
model_nameSonnet 4.5:atoms_affecting	0.104	1.109	0.943	1.305	0.2101

Interaction Term	(log-odds)	OR	OR 95% CI Lower	OR 95% CI Upper	p-value
model_nameSonnet 4.6:atoms_affecting	0.145	1.156	0.980	1.363	0.0851
model_nameOpus 4.5:atoms_affecting	0.068	1.071	0.911	1.259	0.4077
model_nameOpus 4.6:atoms_affecting	0.083	1.087	0.924	1.279	0.3156
atoms_affecting:prompt_styleaugmented	-1.021	0.360	0.322	0.403	<0.001
atoms_affecting:enable_thinkingTRUE	0.414	1.512	1.356	1.687	<0.001

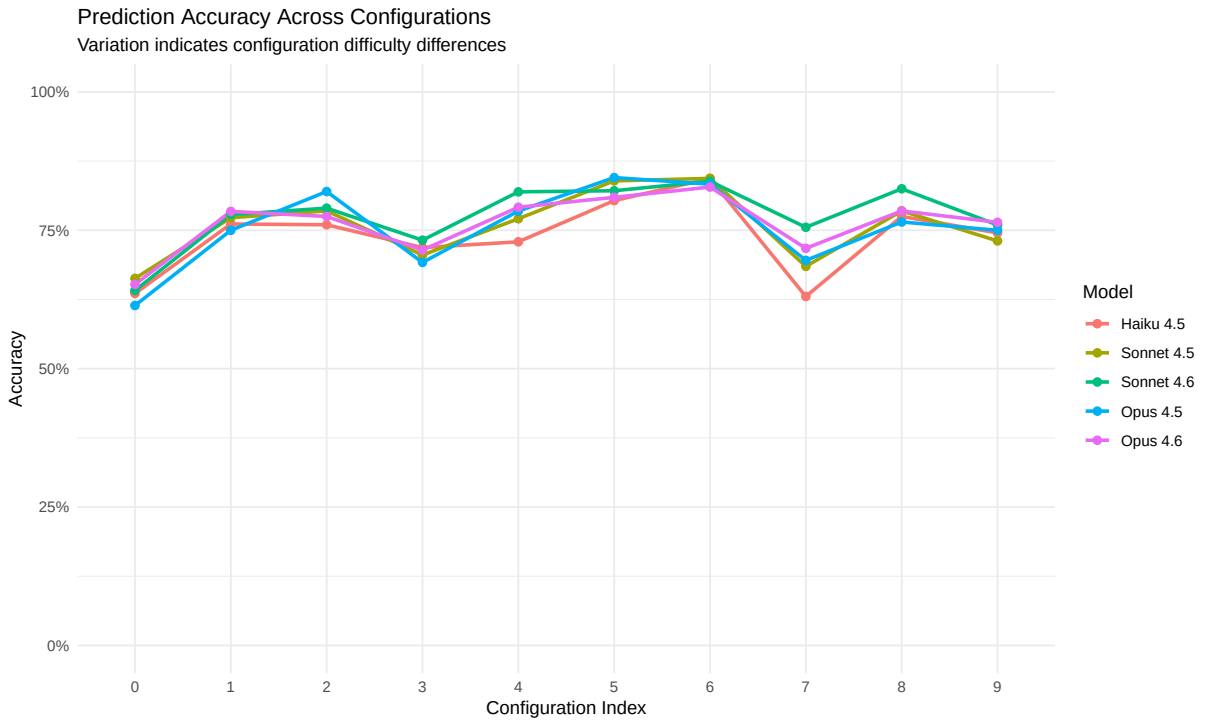
Atom Density Distribution:

Statistic	Value
Min	0
Q1	1
Median	1
Q3	1
Max	3
Mean	0.97
SD	0.71

```
# Accuracy by model and condition
p1_pred <- predict_data |>
  group_by(model_name, prompt_style, enable_thinking) |>
  summarise(accuracy = mean(correct),
            se = sqrt(accuracy * (1-accuracy) / n()),
            .groups = "drop") |>
  ggplot(aes(x = model_name, y = accuracy, fill = enable_thinking)) +
  geom_col(position = position_dodge(0.8), width = 0.7) +
  geom_errorbar(aes(ymin = accuracy - se, ymax = accuracy + se),
                position = position_dodge(0.8), width = 0.2) +
  facet_wrap(~prompt_style) +
  labs(title = "Predict Mode: Accuracy by Model, Thinking, and Prompt Style",
       x = "Model", y = "Accuracy", fill = "Extended\nThinking") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(p1_pred)
```

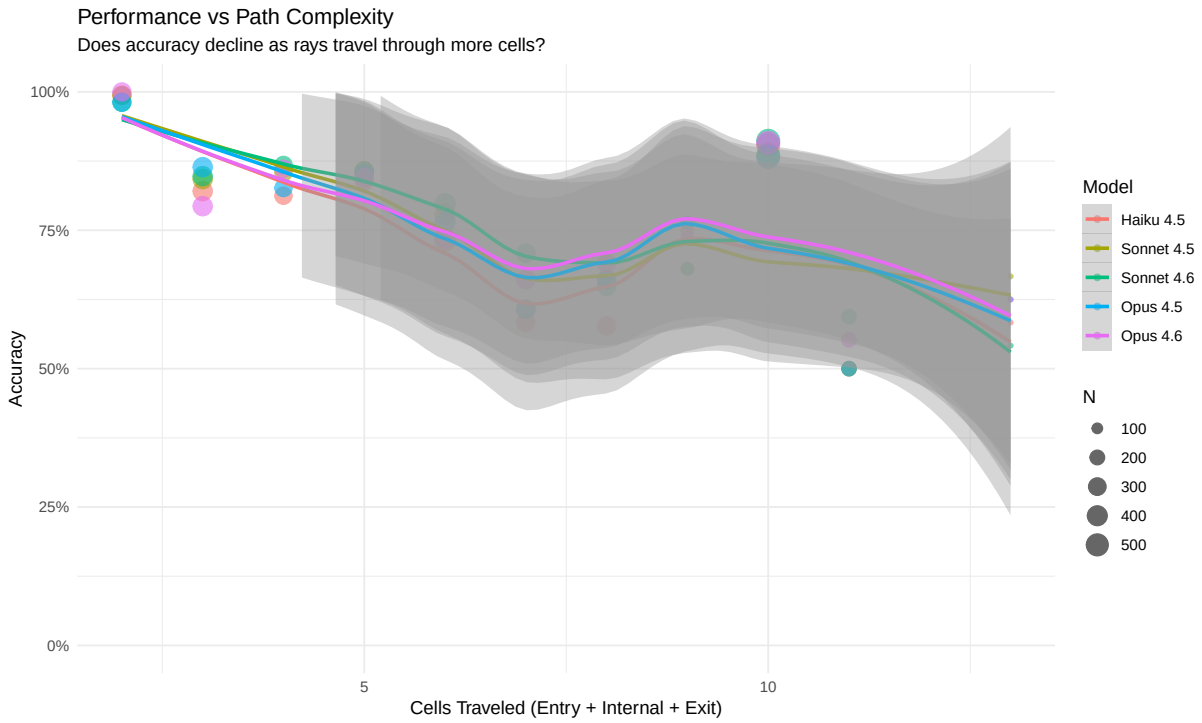



```
# Accuracy by configuration (difficulty analysis)
p2_pred <- predict_data |>
  group_by(config_index, model_name) |>
  summarise(accuracy = mean(correct), .groups = "drop") |>
  ggplot(aes(x = config_index, y = accuracy, color = model_name, group = model_name)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(title = "Prediction Accuracy Across Configurations",
       subtitle = "Variation indicates configuration difficulty differences",
       x = "Configuration Index", y = "Accuracy", color = "Model") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  theme_minimal()
print(p2_pred)
```




```
# VoT Ray Trace effect
p3_pred <- predict_data |>
  group_by(model_name, vot_ray_trace) |>
  summarise(accuracy = mean(correct),
            se = sqrt(accuracy * (1-accuracy) / n()),
            .groups = "drop") |>
  ggplot(aes(x = model_name, y = accuracy, fill = vot_ray_trace)) +
  geom_col(position = position_dodge(0.8), width = 0.7) +
  geom_errorbar(aes(ymin = accuracy - se, ymax = accuracy + se),
                position = position_dodge(0.8), width = 0.2) +
  labs(title = "Effect of VoT Ray Trace Visualization",
       x = "Model", y = "Accuracy", fill = "VoT Ray\nTrace") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(p3_pred)
```

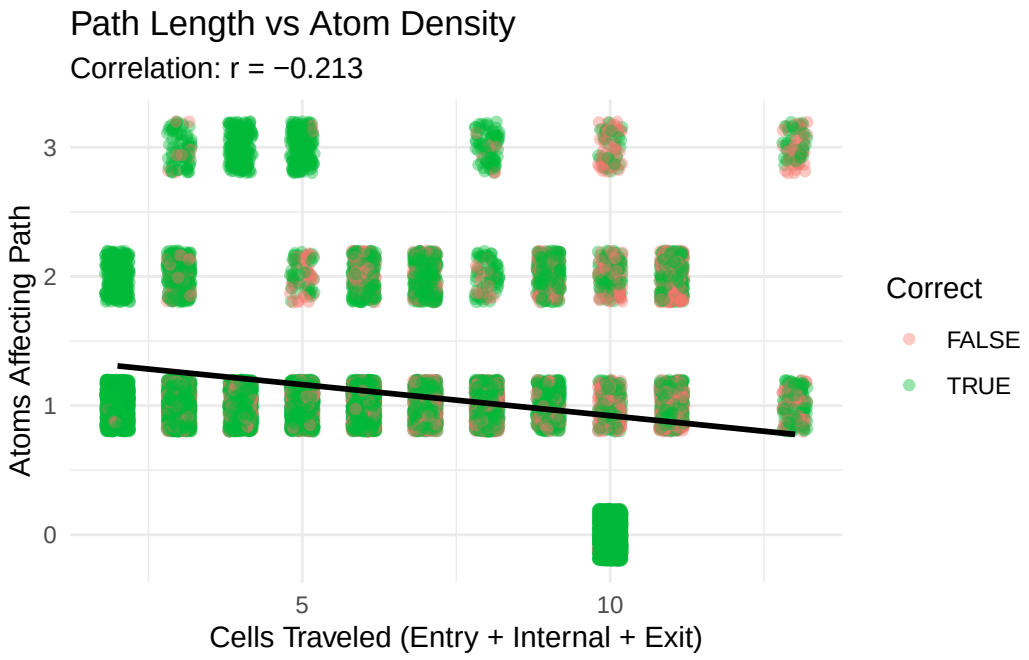

```
# Path complexity vs accuracy
p4_pred <- predict_data |>
  filter(!is.na(cells_traveled)) |>
  group_by(cells_traveled, model_name) |>
  summarise(accuracy = mean(correct),
            n = n(),
            .groups = "drop") |>
  ggplot(aes(x = cells_traveled, y = accuracy, color = model_name)) +
  geom_point(aes(size = n), alpha = 0.6) +
  geom_smooth(method = "loess", se = TRUE, linewidth = 1) +
  labs(title = "Performance vs Path Complexity",
       subtitle = "Does accuracy decline as rays travel through more cells?",
       x = "Cells Traveled (Entry + Internal + Exit)",
       y = "Accuracy",
       color = "Model",
       size = "N") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  theme_minimal()
print(p4_pred)
```

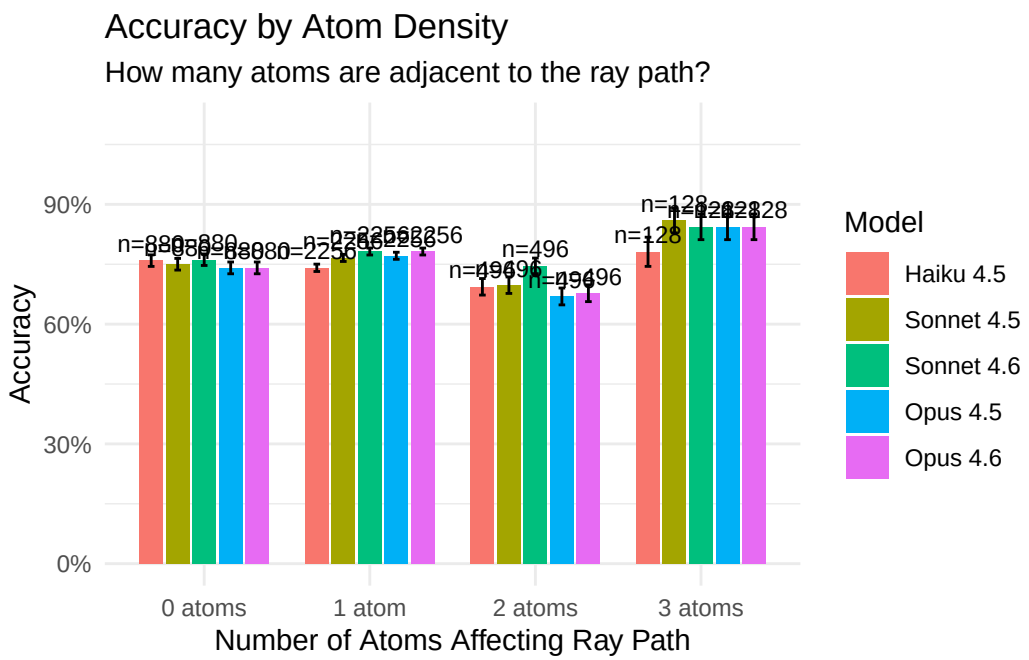


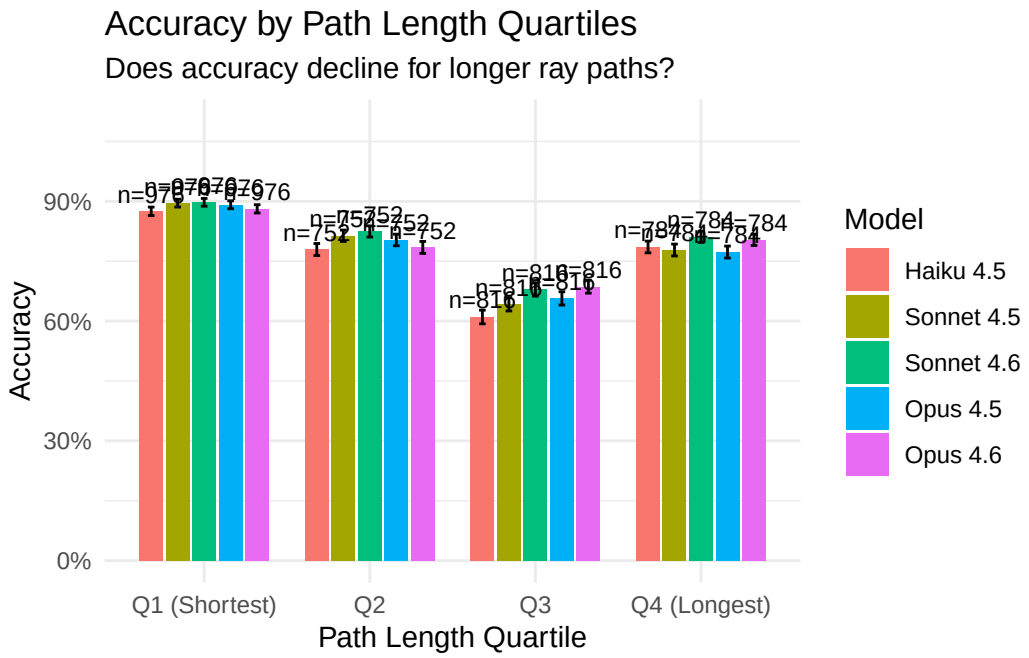
4.1.7 Ray Path Complexity Analysis

```
# 1. Correlation between complexity measures
complexity_cor <- cor(predict_data$cells_traveled, predict_data$atoms_affecting,
                      use = "complete.obs")
```

Correlation between cells_traveled and atoms_affecting: **-0.213**


Path Length vs Atom Density
Correlation: r = −0.213
Atoms Affecting Path
0
1
2
3
5
10
Cells Traveled (Entry + Internal + Exit)
Correct
FALSE
TRUE


Accuracy by Atom Density
How many atoms are adjacent to the ray path?
Accuracy
0%
30%
60%
90%
0 atoms
1 atom
2 atoms
3 atoms
Number of Atoms Affecting Ray Path
Model
Haiku 4.5
Sonnet 4.5
Sonnet 4.6
Opus 4.5
Opus 4.6




```
# Analyze error patterns by comparing predicted vs actual
# This helps identify systematic errors (e.g., absorption vs deflection confusion)

# First, extract the type of result from predicted and actual
predict_errors <- predict_data |>
  filter(!correct) |>
  mutate(
    predicted_type = case_when(
      grepl("absorbed|HIT", predicted, ignore.case = TRUE) ~ "absorbed",
      grepl("reflected|REFLECTION", predicted, ignore.case = TRUE) ~ "reflected",
      TRUE ~ "detour"
    ),
    actual_type = case_when(
      grepl("absorbed|HIT", actual, ignore.case = TRUE) ~ "absorbed",
      grepl("reflected|REFLECTION", actual, ignore.case = TRUE) ~ "reflected",
      TRUE ~ "detour"
    )
  )

# Error confusion matrix
error_matrix <- predict_errors |>
  count(actual_type, predicted_type) |>
  pivot_wider(names_from = predicted_type, values_from = n, values_fill = 0)
```

```
kable(error_matrix, caption = "Prediction Errors: Actual vs Predicted Outcome Type (Rows = A
```

Table 21: Prediction Errors: Actual vs Predicted Outcome Type (Rows = Actual, Columns = Predicted)

actual_type	detour	reflected	absorbed
absorbed	770	62	0
detour	1972	42	472
reflected	856	0	394

```
# Error rates by model
error_by_model <- predict_errors |>
  count(model_name, actual_type, predicted_type) |>
  group_by(model_name) |>
  mutate(prop = n / sum(n)) |>
  ungroup()

error_by_model |>
  group_by(model_name) |>
  slice_max(n, n = 3) |>
  mutate(error_pattern = paste(actual_type, "->", predicted_type)) |>
  select(model_name, error_pattern, n, prop) |>
  mutate(prop = round(prop, 3)) |>
  kable(col.names = c("Model", "Error Pattern (Actual → Predicted)", "Count", "Proportion"),
        caption = "Most Common Error Types by Model (Top 3 per Model)")
```

Table 22: Most Common Error Types by Model (Top 3 per Model)

Model	Error Pattern (Actual → Predicted)	Count	Proportion
Haiku 4.5	detour -> detour	450	0.461
Haiku 4.5	reflected -> detour	180	0.184
Haiku 4.5	absorbed -> detour	174	0.178
Sonnet 4.5	detour -> detour	400	0.437
Sonnet 4.5	reflected -> detour	166	0.181
Sonnet 4.5	absorbed -> detour	152	0.166
Sonnet 4.6	detour -> detour	368	0.434
Sonnet 4.6	reflected -> detour	162	0.191
Sonnet 4.6	absorbed -> detour	124	0.146
Opus 4.5	detour -> detour	376	0.405

Model	Error Pattern (Actual → Predicted)	Count	Proportion
Opus 4.5	reflected -> detour	180	0.194
Opus 4.5	absorbed -> detour	166	0.179
Opus 4.6	detour -> detour	378	0.420
Opus 4.6	reflected -> detour	168	0.187
Opus 4.6	absorbed -> detour	154	0.171

4.1.8 Budget Tag Hallucination Artifact

During experiment runs, we observed that models occasionally hallucinate XML-like `<budget:...>` tags in their visible response text—fabricating token usage metadata that mimics internal API bookkeeping. These tags (e.g., `<budget:used_tokens>`, `<budget:token_budget>`, `<budget:token_usage>`) are not present in any prompt or API parameter passed to the model; they appear to be confabulated from the model's awareness of its extended thinking budget context.

Prevalence: 54 of 18800 predictions (0.29%) contained hallucinated budget tags.

Key patterns:

- **Exclusive to extended thinking**: All 54 instances occurred with extended thinking enabled; no instances were found in non-thinking runs.
- **100% incorrect**: Every prediction containing a budget tag hallucination was incorrect (0/54 correct).
- **Not present in Play mode**: No budget tag hallucinations were found in Play mode data, suggesting the multi-turn conversational structure may suppress this artifact.

Table 23: Budget Tag Hallucinations by Experimental Condition

Model	Prompt	Thinking	Count
Opus 4.6	augmented	TRUE	16
Opus 4.5	augmented	TRUE	10
Haiku 4.5	augmented	TRUE	8
Sonnet 4.5	augmented	TRUE	8
Sonnet 4.6	augmented	TRUE	8
Sonnet 4.5	baseline	TRUE	2
Opus 4.5	baseline	TRUE	2

The perfect correlation with incorrect predictions suggests these hallucinations are a signal of reasoning process failure: when the model generates metadata artifacts instead of reasoning

about ray physics, it invariably produces an incorrect prediction. While the overall prevalence is low (0.29%), this artifact represents a qualitatively distinct failure mode—the model confabulating system-level metadata rather than engaging with the task.

4.2 Play Mode Results

```
# Load all Play mode experiment JSON files from Experiment 1/Play/
play_json_files <- list.files(
  path = "Experiment 1/Play",
  pattern = "\\.json$",
  full.names = TRUE
)

# Function to extract results from a single JSON file
extract_play_results <- function(json_file) {
  data <- jsonlite::fromJSON(json_file, simplifyDataFrame = FALSE)

  # Extract results array
  results <- data$results

  # Convert each result to a tibble row
  map_dfr(results, function(r) {
    # Calculate mean complexity measures for each ray fired
    ray_complexity <- lapply(r$raySequence, function(ray) {
      if (!is.null(ray$action) && ray$action == "fire" &&
          !is.null(ray$rayResult) && !is.null(ray$rayResult$path)) {
        list(
          atoms = count_atoms_affecting(ray$rayResult$path, r$atomConfig),
          cells = count_ray_cells(ray$rayResult)
        )
      } else {
        list(atoms = NA_real_, cells = NA_real_)
      }
    })

    mean_atoms <- mean(sapply(ray_complexity, function(x) x$atoms), na.rm = TRUE)
    mean_cells <- mean(sapply(ray_complexity, function(x) x$cells), na.rm = TRUE)
    if (is.nan(mean_atoms)) mean_atoms <- NA_real_
    if (is.nan(mean_cells)) mean_cells <- NA_real_

    tibble(
```

```
      experiment_id = r$experimentId,
      start_time = r$startTime,
      end_time = r$endTime,
      model = r$model,
      model_name = r$modelName,
      prompt_style = r$promptStyle,
      prompt_condition = r$promptCondition,
      mode = r$mode,
      config_index = r$configIndex,
      atom_config = list(r$atomConfig),
      include_visualization = r$includeVisualization,
      allow_hypotheses = r$allowHypotheses,
      enable_thinking = r$enableThinking,
      thinking_budget = r$thinkingBudget,
      vot_grid_state = r$votGridState,
      vot_ray_trace = r$votRayTrace,
      vot_hypothesis = r$votHypothesis,
      rays_used = r$raysUsed,
      atoms_correct = r$atomsCorrect,
      atoms_missed = r$atomsMissed,
      score = r$score,
      invalid_moves = r$invalidMoves,
      hypothesis_actions = r$hypothesisActions,
      total_api_calls = r$totalApiCalls,
      total_input_tokens = r$totalInputTokens,
      total_output_tokens = r$totalOutputTokens,
      total_response_time_ms = r$totalResponseTimeMs,
      mean_atoms_per_ray = mean_atoms,
      mean_cells_per_ray = mean_cells,
      source_file = basename(json_file)
    )
  })
}

# Load all results into a single long-format tibble
play_data <- map_dfr(play_json_files, extract_play_results)

# Validate: allowHypotheses should be TRUE for all play experiments
hyp_check <- play_data |>
  filter(!allow_hypotheses) |>
  group_by(source_file) |>
  summarise(n_false = n(), .groups = "drop")
```

```r
if (nrow(hyp_check) > 0) {
  warning("Files with allowHypotheses = false detected:\n",
          paste0("  ", hyp_check$source_file, " (", hyp_check$n_false, " configs)\n", collaps
          "Total configs with allowHypotheses = false: ", sum(hyp_check$n_false))
}

# Create single VoT condition factor from mutually exclusive binary flags
play_data <- play_data |>
  mutate(
    vot_condition = case_when(
      vot_grid_state ~ "A: Grid State",
      vot_ray_trace ~ "B: Ray Trace",
      vot_hypothesis ~ "C: Hypothesis",
      TRUE ~ "None"
    ),
    vot_condition = factor(vot_condition, levels = c("None", "A: Grid State", "B: Ray Trace"
  )
```

Data loaded: 800 experiment results from 37 JSON files.

- **Models**: Haiku 4.5, Sonnet 4.5, Opus 4.5, Sonnet 4.6, Opus 4.6
- **Prompt styles**: augmented, baseline
- **VoT conditions**: None, A: Grid State, B: Ray Trace, C: Hypothesis
- **Configurations tested**: 10
- **Mean atoms per ray range**: 0–2
- **Mean cells per ray range**: 2–10

Data Completeness Check

::: {.cell}

```r
# Define expected full factorial design
expected_play_models <- c("Haiku 4.5", "Sonnet 4.5", "Opus 4.5", "Sonnet 4.6", "Opus 4.6")
expected_play_design <- expand_grid(
  model_name = expected_play_models,
  prompt_style = c("baseline", "augmented"),
  enable_thinking = c(FALSE, TRUE),
  vot_condition = factor(
```

```
    c("None", "A: Grid State", "B: Ray Trace", "C: Hypothesis"),
    levels = c("None", "A: Grid State", "B: Ray Trace", "C: Hypothesis")
  ),
  config_index = 0:9
)

# What's actually present
actual_play_conditions <- play_data |>
  distinct(model_name, prompt_style, enable_thinking, vot_condition, config_index)

# Find missing cells
missing_play <- anti_join(
  expected_play_design, actual_play_conditions,
  by = c("model_name", "prompt_style", "enable_thinking", "vot_condition", "config_index")
)

# Summarise missing by condition (collapse config indices)
missing_play_summary <- missing_play |>
  group_by(model_name, prompt_style, enable_thinking, vot_condition) |>
  summarise(
    n_missing_configs = n(),
    missing_configs = paste(config_index, collapse = ", "),
    .groups = "drop"
  ) |>
  arrange(model_name, prompt_style, enable_thinking, vot_condition)

# Condition-level coverage matrix (how many of 10 configs present per condition)
play_coverage <- expected_play_design |>
  distinct(model_name, prompt_style, enable_thinking, vot_condition) |>
  left_join(
    play_data |>
      group_by(model_name, prompt_style, enable_thinking, vot_condition) |>
      summarise(configs_present = n_distinct(config_index),
                total_games = n(), .groups = "drop"),
    by = c("model_name", "prompt_style", "enable_thinking", "vot_condition")
  ) |>
  replace_na(list(configs_present = 0, total_games = 0)) |>
  mutate(
    condition = paste0(
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      vot_condition
    )
```

```
  )

# Pivot to show models as columns
play_coverage_matrix <- play_coverage |>
  select(model_name, condition, configs_present) |>
  pivot_wider(names_from = model_name, values_from = configs_present) |>
  arrange(condition)

if (nrow(missing_play) > 0) {
  cat("**Missing experimental runs detected.**\n\n")
  kable(play_coverage_matrix,
        caption = "Play Mode: Configurations Present (out of 10) by Model and Condition",
        format = "pipe") |> print()
  cat("\n\n**Missing conditions detail:**\n\n")
  kable(missing_play_summary,
        col.names = c("Model", "Prompt", "Thinking", "VoT", "Missing (n)", "Config Indices"),
        caption = "Play Mode: Missing Experimental Runs",
        format = "pipe") |> print()
} else {
  cat("All", nrow(expected_play_design), "expected play-mode cells are present",
      "(5 models x 2 prompts x 2 thinking x 4 VoT x 10 configs).\n")
}
```

```
All 800 expected play-mode cells are present (5 models x 2 prompts x 2 thinking x 4 VoT x 10
```

```
# Check for unexpected model names
unexpected_models_play <- setdiff(unique(play_data$model_name), expected_play_models)
if (length(unexpected_models_play) > 0) {
  cat("\n**Unexpected model names in data:**", paste(unexpected_models_play, collapse = ", ")
}

# Check for duplicate runs (same condition run multiple times)
play_duplicates <- play_data |>
  group_by(model_name, prompt_style, enable_thinking, vot_condition, config_index) |>
  summarise(n_runs = n(), .groups = "drop") |>
  filter(n_runs > 1)
if (nrow(play_duplicates) > 0) {
  cat("\n**Duplicate runs detected (same condition × config run multiple times):**\n")
  kable(play_duplicates,
        col.names = c("Model", "Prompt", "Thinking", "VoT", "Config", "Runs"),
        format = "pipe") |> print()
}
```

:::

All expected play-mode experimental runs are present.

```
# Summary statistics by model, prompt, thinking, and VoT condition
play_summary <- play_data |>
  group_by(model_name, prompt_style, enable_thinking, vot_condition) |>
  summarise(
    n = n(),
    atoms_correct_mean = mean(atoms_correct),
    atoms_correct_sd = sd(atoms_correct),
    score_mean = mean(score),
    score_sd = sd(score),
    mean_atoms_ray = mean(mean_atoms_per_ray, na.rm = TRUE),
    rays_used_mean = mean(rays_used),
    .groups = "drop"
  ) |>
  mutate(across(where(is.numeric) & !n, \(x) round(x, 2)))

kable(play_summary,
      col.names = c("Model", "Prompt", "Thinking", "VoT", "N",
                    "Atoms Correct (M)", "Atoms Correct (SD)",
                    "Score (M)", "Score (SD)",
                    "Atoms/Ray", "Rays Used"))
```

Model	Prompt	Thinking	VoT	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Atoms/Ray	Rays Used
Haiku 4.5	augmented	FALSE	None	10	1.0	1.25	29.5	8.44	1.04	9.9
Haiku 4.5	augmented	FALSE	A: Grid State	10	0.8	1.03	34.5	8.63	1.01	12.8
Haiku 4.5	augmented	FALSE	B: Ray Trace	10	0.8	1.32	31.4	8.92	1.00	10.7
Haiku 4.5	augmented	FALSE	C: Hypothesis	10	0.9	1.37	30.7	7.15	1.03	10.7
Haiku 4.5	augmented	TRUE	None	10	1.9	1.73	26.4	10.81	0.95	11.5
Haiku 4.5	augmented	TRUE	A: Grid State	10	1.7	1.34	26.5	11.74	0.96	10.9

Model	Prompt	Thinking	VoT	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Atoms/Ray	Rays Used
Haiku 4.5	augmented	TRUE	B: Ray Trace	10	1.7	1.42	26.0	9.61	1.01	10.4
Haiku 4.5	augmented	TRUE	C: Hypothesis	10	1.8	1.48	26.7	10.03	0.98	11.0
Haiku 4.5	baseline	FALSE	None	10	0.9	1.37	27.7	7.65	1.09	8.4
Haiku 4.5	baseline	FALSE	A: Grid State	10	0.6	1.26	27.3	7.48	1.16	7.6
Haiku 4.5	baseline	FALSE	B: Ray Trace	10	0.8	1.32	27.3	4.69	1.05	8.1
Haiku 4.5	baseline	FALSE	C: Hypothesis	10	1.0	1.49	24.9	7.32	1.18	7.5
Haiku 4.5	baseline	TRUE	None	10	0.8	1.32	29.9	7.69	0.92	9.7
Haiku 4.5	baseline	TRUE	A: Grid State	10	0.5	1.08	30.0	7.56	1.03	8.6
Haiku 4.5	baseline	TRUE	B: Ray Trace	10	0.9	1.37	28.2	7.60	1.01	8.8
Haiku 4.5	baseline	TRUE	C: Hypothesis	10	0.8	1.03	31.2	4.87	1.04	10.9
Opus 4.5	augmented	FALSE	None	10	1.3	1.34	29.5	8.44	1.04	11.2
Opus 4.5	augmented	FALSE	A: Grid State	10	1.2	1.55	28.4	10.51	1.04	10.4
Opus 4.5	augmented	FALSE	B: Ray Trace	10	1.2	1.32	29.8	9.04	0.99	10.9
Opus 4.5	augmented	FALSE	C: Hypothesis	10	1.0	1.15	30.8	9.03	1.05	11.0
Opus 4.5	augmented	TRUE	None	10	1.6	1.35	27.2	7.15	1.08	10.8
Opus 4.5	augmented	TRUE	A: Grid State	10	2.0	1.56	24.2	10.39	1.00	9.9
Opus 4.5	augmented	TRUE	B: Ray Trace	10	1.9	1.52	23.5	9.41	1.13	9.9

Model	Prompt	Thinking	VoT	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Atoms/Ray	Rays Used
Opus 4.5	augmented	TRUE	C: Hypothesis	10	1.7	1.34	28.6	12.11	0.96	12.3
Opus 4.5	baseline	FALSE	None	10	0.8	1.03	27.9	6.26	1.04	8.5
Opus 4.5	baseline	FALSE	A: Grid State	10	1.3	1.25	28.3	6.91	0.98	10.0
Opus 4.5	baseline	FALSE	B: Ray Trace	10	0.8	1.32	28.6	7.66	1.16	8.8
Opus 4.5	baseline	FALSE	C: Hypothesis	10	0.8	1.23	28.0	5.75	1.02	8.5
Opus 4.5	baseline	TRUE	None	10	0.7	1.49	28.7	8.64	0.95	8.6
Opus 4.5	baseline	TRUE	A: Grid State	10	1.0	1.33	29.2	7.76	0.94	9.7
Opus 4.5	baseline	TRUE	B: Ray Trace	10	1.0	1.49	26.8	8.68	0.94	7.9
Opus 4.5	baseline	TRUE	C: Hypothesis	10	1.0	0.94	28.7	5.44	1.03	9.8
Opus 4.6	augmented	FALSE	None	10	1.4	1.35	33.1	10.31	1.10	14.5
Opus 4.6	augmented	FALSE	A: Grid State	10	1.5	1.51	32.6	9.42	1.08	15.0
Opus 4.6	augmented	FALSE	B: Ray Trace	10	0.7	1.25	37.7	10.25	0.99	15.3
Opus 4.6	augmented	FALSE	C: Hypothesis	10	1.2	1.55	36.2	11.25	1.00	15.6
Opus 4.6	augmented	TRUE	None	10	2.4	1.35	23.3	9.80	1.06	11.1
Opus 4.6	augmented	TRUE	A: Grid State	10	2.5	1.27	25.5	10.08	1.01	12.8
Opus 4.6	augmented	TRUE	B: Ray Trace	10	3.1	0.74	21.3	5.60	0.98	11.8
Opus 4.6	augmented	TRUE	C: Hypothesis	10	1.9	1.73	30.1	12.57	1.03	13.8

Model	Prompt	Thinking	VoT	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Atoms/Ray	Rays Used
Opus 4.6	baseline	FALSE	None	10	0.6	1.26	32.4	9.13	0.99	10.8
Opus 4.6	baseline	FALSE	A: Grid State	10	1.6	1.58	28.3	10.06	1.08	11.9
Opus 4.6	baseline	FALSE	B: Ray Trace	10	0.9	0.74	31.2	6.23	1.05	11.0
Opus 4.6	baseline	FALSE	C: Hypothesis	10	1.2	1.32	34.3	10.99	1.09	14.4
Opus 4.6	baseline	TRUE	None	10	1.6	1.35	26.4	7.96	1.07	10.2
Opus 4.6	baseline	TRUE	A: Grid State	10	1.3	1.34	27.7	7.86	1.10	10.1
Opus 4.6	baseline	TRUE	B: Ray Trace	10	1.5	1.35	25.6	7.37	0.97	9.5
Opus 4.6	baseline	TRUE	C: Hypothesis	10	1.5	1.35	26.7	8.43	1.11	10.1
Sonnet 4.5	augmented	FALSE	None	10	1.2	1.14	29.3	7.89	1.00	10.5
Sonnet 4.5	augmented	FALSE	A: Grid State	10	1.4	1.51	31.0	13.01	0.96	12.5
Sonnet 4.5	augmented	FALSE	B: Ray Trace	10	1.1	1.37	32.1	9.75	1.01	11.9
Sonnet 4.5	augmented	FALSE	C: Hypothesis	10	1.2	1.40	31.3	10.38	1.01	12.0
Sonnet 4.5	augmented	TRUE	None	10	2.0	1.15	24.6	9.17	0.99	10.5
Sonnet 4.5	augmented	TRUE	A: Grid State	10	1.2	1.32	30.5	9.82	1.00	11.1
Sonnet 4.5	augmented	TRUE	B: Ray Trace	10	1.7	1.42	26.7	7.90	0.92	11.1

Model	Prompt	Thinking	VoT	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Atoms/Ray	Rays Used
Sonnet 4.5	augmented	TRUE	C: Hypothesis	10	1.4	1.35	28.9	9.12	1.00	11.2
Sonnet 4.5	baseline	FALSE	None	10	0.7	1.16	29.4	6.43	1.09	9.3
Sonnet 4.5	baseline	FALSE	A: Grid State	10	1.1	1.20	28.5	8.81	1.01	10.2
Sonnet 4.5	baseline	FALSE	B: Ray Trace	10	0.8	1.40	25.8	7.05	1.01	7.1
Sonnet 4.5	baseline	FALSE	C: Hypothesis	10	1.3	1.70	26.2	8.39	1.04	9.0
Sonnet 4.5	baseline	TRUE	None	10	1.0	1.49	25.9	9.24	0.99	7.7
Sonnet 4.5	baseline	TRUE	A: Grid State	10	0.8	1.14	30.4	7.20	0.97	10.2
Sonnet 4.5	baseline	TRUE	B: Ray Trace	10	0.9	0.99	27.8	6.30	0.91	8.6
Sonnet 4.5	baseline	TRUE	C: Hypothesis	10	1.1	1.10	27.9	9.42	1.00	9.6
Sonnet 4.6	augmented	FALSE	None	10	1.3	1.49	34.9	9.02	1.01	14.9
Sonnet 4.6	augmented	FALSE	A: Grid State	10	1.5	1.51	33.4	10.47	1.06	15.4
Sonnet 4.6	augmented	FALSE	B: Ray Trace	10	1.2	1.55	35.8	9.03	1.01	15.7
Sonnet 4.6	augmented	FALSE	C: Hypothesis	10	1.7	0.95	33.5	6.62	0.99	15.5

Model	Prompt	Thinking	VoT	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Atoms/Ray	Rays Used
Sonnet 4.6	augmented	TRUE	None	10	2.6	1.26	24.5	9.23	1.02	13.2
Sonnet 4.6	augmented	TRUE	A: Grid State	10	2.4	1.17	24.3	8.04	1.04	11.9
Sonnet 4.6	augmented	TRUE	B: Ray Trace	10	3.0	0.94	20.9	7.37	0.98	11.5
Sonnet 4.6	augmented	TRUE	C: Hypothesis	10	2.4	1.71	27.9	11.83	1.00	14.2
Sonnet 4.6	baseline	FALSE	None	10	0.8	1.32	35.5	7.28	1.02	13.5
Sonnet 4.6	baseline	FALSE	A: Grid State	10	0.9	1.10	36.7	6.68	1.00	14.9
Sonnet 4.6	baseline	FALSE	B: Ray Trace	10	0.9	1.29	32.3	11.15	0.97	12.0
Sonnet 4.6	baseline	FALSE	C: Hypothesis	10	1.1	1.29	35.1	10.57	1.01	14.3
Sonnet 4.6	baseline	TRUE	None	10	1.0	1.49	25.3	7.23	1.07	7.3
Sonnet 4.6	baseline	TRUE	A: Grid State	10	1.3	1.49	27.5	9.26	1.04	10.1
Sonnet 4.6	baseline	TRUE	B: Ray Trace	10	1.2	1.55	25.9	9.24	0.96	8.3
Sonnet 4.6	baseline	TRUE	C: Hypothesis	10	1.3	1.16	29.3	6.91	0.91	10.7

```r
# Summary by VoT condition
play_summary_vot <- play_data |>
  group_by(vot_condition) |>
  summarise(
    n = n(),
    atoms_correct_mean = mean(atoms_correct),
    atoms_correct_sd = sd(atoms_correct),
    score_mean = mean(score),
    score_sd = sd(score),
    rays_used_mean = mean(rays_used),
    mean_atoms_ray = mean(mean_atoms_per_ray, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(across(where(is.numeric) & !n, \(x) round(x, 2)))

kable(play_summary_vot,
      col.names = c("VoT Condition", "N",
                    "Atoms Correct (M)", "Atoms Correct (SD)",
                    "Score (M)", "Score (SD)", "Rays Used", "Atoms/Ray"))
```

VoT Condition	N	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)	Rays Used	Atoms/Ray
None	200	1.28	1.40	28.57	8.71	10.61	1.02
A: Grid State	200	1.33	1.37	29.24	9.34	11.30	1.02
B: Ray Trace	200	1.30	1.41	28.24	8.95	10.46	1.00
C: Hypothesis	200	1.31	1.35	29.85	9.24	11.61	1.02

4.2.1 Play Mode Leaderboard

Top 10 configurations by atoms correct. Format matches typical LLM benchmark reporting for cross-study comparison.

```r
leaderboard_play <- play_data |>
  mutate(
    condition = paste0(
      model_name, " | ",
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
```

```
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      vot_condition
    )
  ) |>
  group_by(condition, model_name, prompt_style, enable_thinking, vot_condition) |>
  summarise(
    n = n(),
    atoms_correct_mean = mean(atoms_correct),
    atoms_correct_se = sd(atoms_correct) / sqrt(n),
    score_mean = mean(score),
    perfect_games = sum(atoms_correct == 4),
    .groups = "drop"
  ) |>
  arrange(desc(atoms_correct_mean), score_mean) |>
  mutate(
    rank = row_number(),
    atoms_ci = paste0(round(atoms_correct_mean, 2), " ± ", round(atoms_correct_se, 2)),
    perfect_pct = paste0(round(perfect_games / n * 100, 1), "%")
  ) |>
  select(rank, condition, atoms_ci, score_mean, perfect_pct, n) |>
  head(10)

kable(leaderboard_play,
      col.names = c("Rank", "Configuration", "Atoms Correct (M ± SE)", "Score (M)", "Perfect
      caption = "Play Mode Leaderboard (Top 10 Configurations)")
```

Table 26: Play Mode Leaderboard (Top 10 Configurations)

Rank	Configuration	Atoms Correct (M ± SE)	Score (M)	Perfect Games	N
1	Opus 4.6 | Aug | Think | B: Ray Trace	3.1 ± 0.23	21.3	30%	10
2	Sonnet 4.6 | Aug | Think | B: Ray Trace	3 ± 0.3	20.9	30%	10
3	Sonnet 4.6 | Aug | Think | None	2.6 ± 0.4	24.5	30%	10
4	Opus 4.6 | Aug | Think | A: Grid State	2.5 ± 0.4	25.5	30%	10
5	Opus 4.6 | Aug | Think | None	2.4 ± 0.43	23.3	30%	10
6	Sonnet 4.6 | Aug | Think | A: Grid State	2.4 ± 0.37	24.3	30%	10
7	Sonnet 4.6 | Aug | Think | C: Hypothesis	2.4 ± 0.54	27.9	40%	10
8	Opus 4.5 | Aug | Think | A: Grid State	2 ± 0.49	24.2	20%	10
9	Sonnet 4.5 | Aug | Think | None	2 ± 0.37	24.6	10%	10
10	Opus 4.5 | Aug | Think | B: Ray Trace	1.9 ± 0.48	23.5	20%	10

4.2.2 ANOVA: Play Mode Performance

Full factorial ANOVA using mixed-effects models with configuration-level random effects. Covariates (mean_atoms_per_ray, mean_cells_per_ray) capture ray difficulty.

```
# Convert factors
play_data <- play_data |>
  mutate(
    model_name = factor(model_name, levels = order_model_names(model_name)),
    prompt_style = factor(prompt_style, levels = c("baseline", "augmented")),
    enable_thinking = factor(enable_thinking, levels = c(FALSE, TRUE))
  )

# Build model formula dynamically
has_model_var <- length(unique(play_data$model_name)) > 1
has_prompt_var <- length(unique(play_data$prompt_style)) > 1
has_thinking_var <- length(unique(play_data$enable_thinking)) > 1
has_vot_var <- length(unique(play_data$vot_condition)) > 1

# Build main formula
formula_parts <- c()
if (has_model_var) formula_parts <- c(formula_parts, "model_name")
if (has_prompt_var) formula_parts <- c(formula_parts, "prompt_style")
if (has_thinking_var) formula_parts <- c(formula_parts, "enable_thinking")

if (has_model_var && has_prompt_var && has_thinking_var) {
  main_formula <- "model_name * prompt_style * enable_thinking"
} else {
  main_formula <- paste(formula_parts, collapse = " + ")
}

# Add VoT condition factor
if (has_vot_var) {
  atoms_formula <- paste0("atoms_correct ~ ", main_formula, " + vot_condition",
                          " + mean_cells_per_ray + mean_atoms_per_ray + (1 | config_index)")
} else {
  atoms_formula <- paste0("atoms_correct ~ ", main_formula,
                          " + mean_cells_per_ray + mean_atoms_per_ray + (1 | config_index)")
}

model_atoms <- lmer(as.formula(atoms_formula), data = play_data)
```

4.2.2.1 Model 1: Atoms Correct (0-4)

Testing which factors affect the number of atoms correctly identified.

Model formula: atoms_correct ~ model_name * prompt_style * enable_thinking + vot_condition + mean_cells_per_ray + mean_atoms_per_ray + (1 | config_index)

```
Analysis of Deviance Table (Type III Wald chisquare tests)

Response: atoms_correct
                                          Chisq Df Pr(>Chisq)
(Intercept)                             36.7258  1  1.360e-09 ***
model_name                               1.5088  4  0.8250739
prompt_style                             0.0006  1  0.9806273
enable_thinking                          0.9931  1  0.3189900
vot_condition                            0.3893  3  0.9424481
mean_cells_per_ray                      18.5125  1  1.688e-05 ***
mean_atoms_per_ray                      21.4161  1  3.697e-06 ***
model_name:prompt_style                  4.6153  4  0.3290917
model_name:enable_thinking               7.8392  4  0.0976474 .
prompt_style:enable_thinking            14.7514  1  0.0001227 ***
model_name:prompt_style:enable_thinking  4.2548  4  0.3726239
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Groups       Name        Std.Dev.
 config_index (Intercept) 0.82034
 Residual                 0.82211
```

Random Effects — Configuration (config_index) variance accounts for config-level difficulty and allows generalization beyond these 10 configurations.

ICC: 0.499 — Proportion of variance due to configuration clustering

Model Fit: - **Marginal R^2** (fixed effects only): 0.201 - **Conditional R^2** (fixed + random effects): 0.6

```
# Model 2: Score (main effects with config random effects)
if (has_vot_var) {
  score_formula <- paste0("score ~ ", main_formula, " + vot_condition",
                          " + mean_cells_per_ray + mean_atoms_per_ray + (1 | config_index)")
} else {
  score_formula <- paste0("score ~ ", main_formula,
```

```
                          " + mean_cells_per_ray + mean_atoms_per_ray + (1 | config_index)")
}

model_score <- lmer(as.formula(score_formula), data = play_data)
```

4.2.2.2 Model 2: Score (Lower is Better)

Testing which factors affect game score.

Model formula: score ~ model_name * prompt_style * enable_thinking + vot_condition + mean_cells_per_ray + mean_atoms_per_ray + (1 | config_index)

```
Analysis of Deviance Table (Type III Wald chisquare tests)

Response: score
                                          Chisq Df Pr(>Chisq)
(Intercept)                             28.0836  1  1.162e-07 ***
model_name                              60.7431  4  2.025e-12 ***
prompt_style                            11.7363  1  0.0006129 ***
enable_thinking                          5.6520  1  0.0174357 *
vot_condition                           14.1810  3  0.0026689 **
mean_cells_per_ray                      89.3372  1  < 2.2e-16 ***
mean_atoms_per_ray                       7.2678  1  0.0070203 **
model_name:prompt_style                  8.5635  4  0.0729855 .
model_name:enable_thinking              54.4989  4  4.137e-11 ***
prompt_style:enable_thinking            19.4394  1  1.038e-05 ***
model_name:prompt_style:enable_thinking  8.4127  4  0.0775787 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Groups       Name        Std.Dev.
 config_index (Intercept) 6.7338
 Residual                 5.3369
```

ICC: 0.614

Model Fit: - **Marginal R^2**: 0.168 - **Conditional R^2**: 0.679

```
### Effect Modification by Ray Complexity (Play Mode)
```

Does ray complexity (mean atoms per ray) moderate treatment effects?

::: {.cell}

```{.r .cell-code}
run_play_effect_mod <- has_model_var || has_prompt_var || has_thinking_var || has_vot_var

if (run_play_effect_mod) {
  # Build interaction formula with random effect
  interaction_parts <- c()
  if (has_model_var) interaction_parts <- c(interaction_parts, "model_name * mean_atoms_per_r
  if (has_prompt_var) interaction_parts <- c(interaction_parts, "prompt_style * mean_atoms_pe
  if (has_thinking_var) interaction_parts <- c(interaction_parts, "enable_thinking * mean_ato
  if (has_vot_var) interaction_parts <- c(interaction_parts, "vot_condition * mean_atoms_per_

  atoms_mod_formula <- paste0("atoms_correct ~ ",
                              paste(interaction_parts, collapse = " + "),
                              " + (1 | config_index)")
  score_mod_formula <- paste0("score ~ ",
                              paste(interaction_parts, collapse = " + "),
                              " + (1 | config_index)")

  model_atoms_mod <- lmer(as.formula(atoms_mod_formula), data = play_data)
  model_score_mod <- lmer(as.formula(score_mod_formula), data = play_data)
}
```

:::

Atoms Correct Model: atoms_correct ~ model_name * mean_atoms_per_ray + prompt_style * mean_atoms_per_ray + enable_thinking * mean_atoms_per_ray + vot_condition * mean_atoms_per_ray + (1 | config_index)

```
Analysis of Deviance Table (Type III Wald chisquare tests)

Response: atoms_correct
                                 Chisq Df Pr(>Chisq)
(Intercept)                     5.5252  1  0.0187446 *
model_name                      8.7995  4  0.0663112 .
mean_atoms_per_ray              3.0049  1  0.0830152 .
prompt_style                   14.1938  1  0.0001649 ***
enable_thinking                 4.9454  1  0.0261605 *
vot_condition                   1.0567  3  0.7875389
```

```
model_name:mean_atoms_per_ray       2.4541  4  0.6528718
mean_atoms_per_ray:prompt_style     1.3903  1  0.2383533
mean_atoms_per_ray:enable_thinking  0.0283  1  0.8664139
mean_atoms_per_ray:vot_condition    1.3588  3  0.7152269
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Score Model: score ~ model_name * mean_atoms_per_ray + prompt_style * mean_atoms_per_ray + enable_thinking * mean_atoms_per_ray + vot_condition * mean_atoms_per_ray + (1 | config_index)

```
Analysis of Deviance Table (Type III Wald chisquare tests)

Response: score
                                    Chisq Df Pr(>Chisq)
(Intercept)                      131.6743  1  < 2.2e-16 ***
model_name                         3.8517  4    0.42645
mean_atoms_per_ray                18.7534  1  1.488e-05 ***
prompt_style                      16.0010  1  6.331e-05 ***
enable_thinking                    4.6384  1    0.03126 *
vot_condition                      0.5511  3    0.90754
model_name:mean_atoms_per_ray      8.0824  4    0.08860 .
mean_atoms_per_ray:prompt_style   17.9663  1  2.248e-05 ***
mean_atoms_per_ray:enable_thinking  0.4668  1    0.49447
mean_atoms_per_ray:vot_condition   0.3837  3    0.94359
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.2.3 Regression Analysis: Play Mode

Regression complements ANOVA by providing interpretable effect magnitudes. Coefficients show the expected change in outcome per unit change in predictor.

```r
# Build formula with factors that have variation
regression_parts <- formula_parts  # From earlier: model_name, prompt_style, enable_thinking
if (has_vot_var) regression_parts <- c(regression_parts, "vot_condition")
regression_parts <- c(regression_parts, "mean_atoms_per_ray", "mean_cells_per_ray")

atoms_reg_formula <- paste0("atoms_correct ~ ", paste(regression_parts, collapse = " + "),
                            " + (1 | config_index)")
```

```
lmer_atoms <- lmer(as.formula(atoms_reg_formula), data = play_data)

# Tidy summary (fixed effects only)
atoms_reg_summary <- tidy(lmer_atoms, effects = "fixed", conf.int = TRUE) |>
  mutate(across(c(estimate, conf.low, conf.high, statistic), ~round(., 3)),
         p.value = format.pval(p.value, digits = 3, eps = 0.001)) |>
  select(term, estimate, std.error, statistic, p.value, conf.low, conf.high)

kable(atoms_reg_summary,
      col.names = c("Term", "  (Coefficient)", "SE", "t-statistic", "p-value", "95% CI Lower"
      caption = "Regression Coefficients: Atoms Correct (Fixed Effects)")
```

Table 27: Regression Coefficients: Atoms Correct (Fixed Effects)

Term	(Coefficient)	SE	t-statistic	p-value	95% CI Lower	95% CI Upper
(Intercept)	2.227	0.4264751	5.222	<0.001	1.371	3.083
model_nameSonnet 4.5	0.130	0.0957763	1.354	0.1762	-0.058	0.318
model_nameSonnet 4.6	0.483	0.0955754	5.057	<0.001	0.296	0.671
model_nameOpus 4.5	0.162	0.0955281	1.696	0.0903	-0.026	0.350
model_nameOpus 4.6	0.519	0.0955337	5.435	<0.001	0.332	0.707
prompt_styleaug-mented	0.591	0.0604765	9.771	<0.001	0.472	0.710
enable_think-ingTRUE	0.447	0.0609174	7.338	<0.001	0.327	0.567
vot_conditionA: Grid State	0.050	0.0853837	0.585	0.5584	-0.118	0.218
vot_conditionB: Ray Trace	0.024	0.0855034	0.283	0.7771	-0.144	0.192
vot_conditionC: Hypothesis	0.032	0.0853850	0.380	0.7039	-0.135	0.200
mean_atoms_per_ray	-0.688	0.1764329	-3.900	<0.001	-1.034	-0.342
mean_cells_per_ray	-0.165	0.0364162	-4.536	<0.001	-0.237	-0.094

```
r2_atoms_reg <- performance::r2_nakagawa(lmer_atoms)

# Model for Score
```

```
score_reg_formula <- paste0("score ~ ", paste(regression_parts, collapse = " + "),
                            " + (1 | config_index)")

lmer_score <- lmer(as.formula(score_reg_formula), data = play_data)

score_reg_summary <- tidy(lmer_score, effects = "fixed", conf.int = TRUE) |>
  mutate(across(c(estimate, conf.low, conf.high, statistic), ~round(., 3)),
         p.value = format.pval(p.value, digits = 3, eps = 0.001)) |>
  select(term, estimate, std.error, statistic, p.value, conf.low, conf.high)

kable(score_reg_summary,
      col.names = c("Term", "  (Coefficient)", "SE", "t-statistic", "p-value", "95% CI Lower"
      caption = "Regression Coefficients: Score (Fixed Effects)")
```

Table 28: Regression Coefficients: Score (Fixed Effects)

Term	(Coefficient)	SE	t-statistic	p-value	95% CI Lower	95% CI Upper
(Intercept)	21.152	3.1408966	6.734	<0.001	14.767	27.538
model_nameSonnet 4.5	-0.632	0.6375563	-0.992	0.3217	-1.884	0.619
model_nameSonnet 4.6	1.232	0.6362162	1.937	0.0532	-0.017	2.481
model_nameOpus 4.5	-0.867	0.6359000	-1.363	0.1732	-2.115	0.381
model_nameOpus 4.6	0.858	0.6359414	1.349	0.1776	-0.390	2.106
prompt_styleaug-mented	0.313	0.4025775	0.779	0.4365	-0.477	1.104
enable_think-ingTRUE	-4.008	0.4055416	-9.884	<0.001	-4.804	-3.212
vot_conditionA: Grid State	0.659	0.5683714	1.160	0.2465	-0.456	1.775
vot_conditionB: Ray Trace	-0.618	0.5691713	-1.085	0.2782	-1.735	0.500
vot_conditionC: Hypothesis	1.307	0.5683804	2.299	0.0218	0.191	2.422
mean_atoms_per_ray	-4.154	1.1798731	-3.521	<0.001	-6.471	-1.838
mean_cells_per_ray	2.172	0.2427849	8.944	<0.001	1.695	2.648

```
r2_score_reg <- performance::r2_nakagawa(lmer_score)

# Standardized coefficients for comparison
std_coef_atoms <- standardize_parameters(lmer_atoms, method = "refit")
std_coef_score <- standardize_parameters(lmer_score, method = "refit")
```

Model Fit — Atoms Correct: - Marginal R^2: 0.158 - Conditional R^2: 0.572

Model Fit — Score: - Marginal R^2: 0.117 - Conditional R^2: 0.649

4.2.3.1 Standardized Coefficients (Beta Weights)

Allows comparison of relative importance across fixed effects (based on fixed effects only).

Atoms Correct (Top 10):

```
# Standardization method: refit

Parameter                    | Std. Coef. |         95% CI
-----------------------------------------------------------
prompt style [augmented]     |       0.43 | [ 0.34,  0.51]
model name [Opus 4.6]        |       0.38 | [ 0.24,  0.51]
model name [Sonnet 4.6]      |       0.35 | [ 0.21,  0.49]
enable thinking [TRUE]       |       0.32 | [ 0.24,  0.41]
mean atoms per ray           |      -0.14 | [-0.22, -0.07]
mean cells per ray           |      -0.12 | [-0.17, -0.07]
model name [Opus 4.5]        |       0.12 | [-0.02,  0.25]
model name [Sonnet 4.5]      |       0.09 | [-0.04,  0.23]
vot conditionA × Grid State  |       0.04 | [-0.09,  0.16]
vot conditionC × Hypothesis  |       0.02 | [-0.10,  0.14]
```

Score (Top 10):

```
# Standardization method: refit

Parameter                    | Std. Coef. |         95% CI
-----------------------------------------------------------
enable thinking [TRUE]       |      -0.44 | [-0.53, -0.35]
mean cells per ray           |       0.24 | [ 0.19,  0.29]
vot conditionC × Hypothesis  |       0.14 | [ 0.02,  0.27]
model name [Sonnet 4.6]      |       0.14 | [ 0.00,  0.27]
mean atoms per ray           |      -0.13 | [-0.21, -0.06]
```

```
model name [Opus 4.5]       |      -0.10 | [-0.23,  0.04]
model name [Opus 4.6]       |       0.09 | [-0.04,  0.23]
vot conditionA × Grid State |       0.07 | [-0.05,  0.20]
model name [Sonnet 4.5]     |      -0.07 | [-0.21,  0.07]
vot conditionB × Ray Trace  |      -0.07 | [-0.19,  0.06]
```

4.2.4 Interpretation: Why Configuration Random Effects?

Play mode data structure: - Unit of analysis: Entire games (one game per config × condition) - LLM chooses which rays to fire (endogenous ray selection) - Different conditions may fire different rays - Multiple games per configuration (clustering)

Why config-level random effects?

1. Games within same config share atom placement
2. Some configs are inherently harder/easier (random variation)
3. Generalizes inference beyond these 10 specific configurations
4. Accounts for correlation among games within same config

Why NOT ray-level random effects (unlike Predict mode)?

1. Rays are not predetermined—LLM chooses them
2. Different conditions may fire different rays
3. Ray selection is part of the strategy (endogenous)
4. No repeated measures at individual ray level

Covariates (mean_cells_per_ray, mean_atoms_per_ray): - **NOT confounders**: Ray choice is endogenous to experimental condition - **ARE precision variables**: Describe difficulty of rays LLM chose to fire - **Improve model fit**: Explain variance, increase power

Impact of ignoring config-level clustering: - Underestimates standard errors - Inflates Type I error (false positives) - Invalid inference (cannot generalize beyond these 10 configs)

````
::: {.cell}

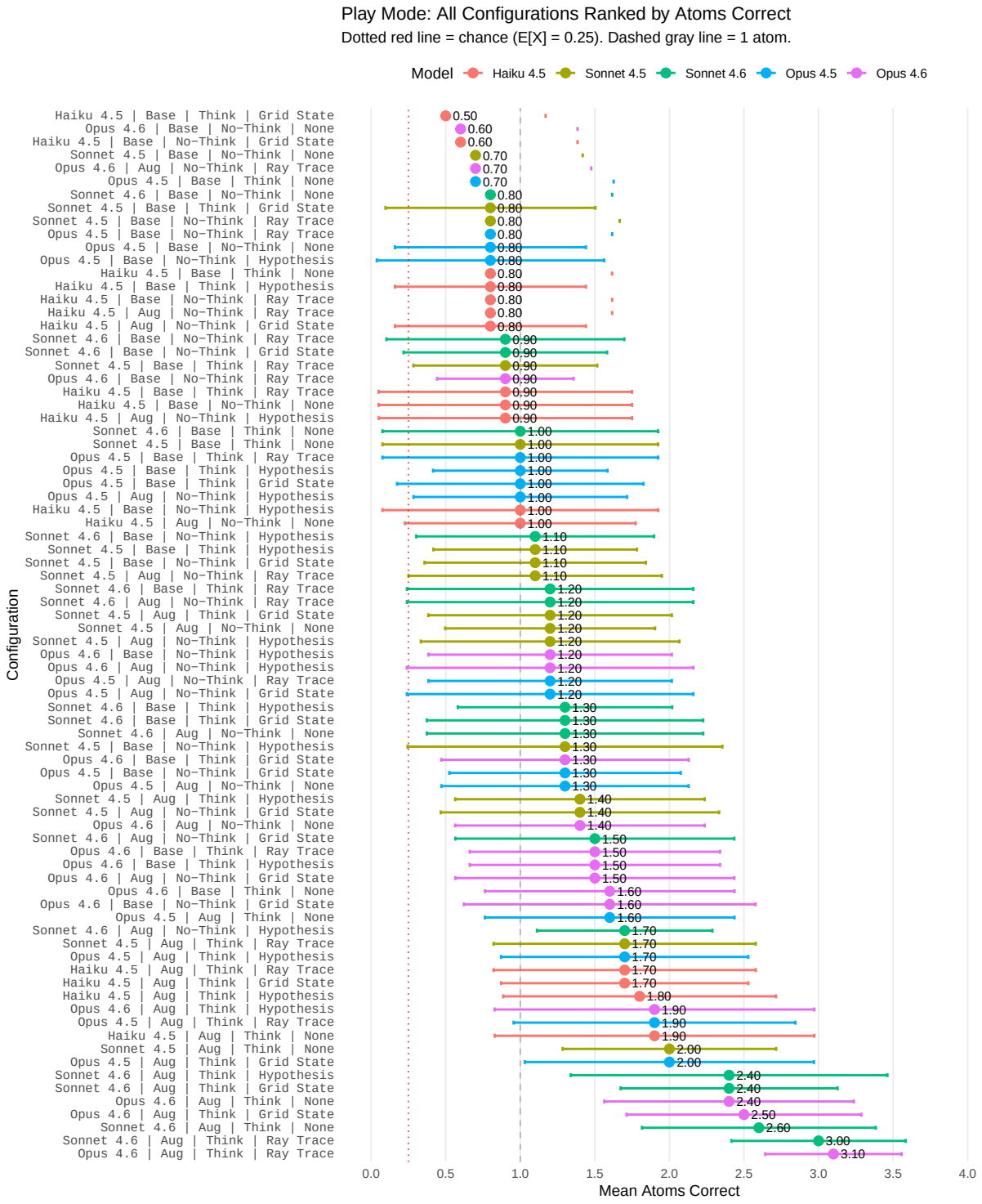
```{.r .cell-code}
# Calculate mean atoms correct and 95% CI for all configurations
play_forest_data <- play_data |>
  mutate(
    # Create readable configuration labels
    config_label = paste0(
      model_name, " | ",
````
````

```
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(enable_thinking), "Think", "No-Think"), " | ",
      gsub("^[A-C]: ", "", as.character(vot_condition))
    )
  ) |>
  group_by(config_label, model_name, prompt_style, enable_thinking, vot_condition) |>
  summarise(
    n = n(),
    mean_correct = mean(atoms_correct),
    se = sd(atoms_correct) / sqrt(n()),
    ci_lower = mean_correct - 1.96 * se,
    ci_upper = mean_correct + 1.96 * se,
    .groups = "drop"
  ) |>
  arrange(desc(mean_correct)) |>
  mutate(
    config_label = factor(config_label, levels = config_label)
  )

# Forest plot
p_play_forest <- ggplot(play_forest_data, aes(x = mean_correct, y = config_label)) +
  geom_point(aes(color = model_name), size = 3) +
  geom_errorbarh(aes(xmin = ci_lower, xmax = ci_upper, color = model_name),
                 height = 0.3, linewidth = 0.8) +
  geom_vline(xintercept = 0.25, linetype = "dotted", color = "red", alpha = 0.7) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "gray50", alpha = 0.5) +
  geom_text(aes(label = sprintf("%.2f", mean_correct)),
            hjust = -0.3, size = 3) +
  labs(
    title = "Play Mode: All Configurations Ranked by Atoms Correct",
    subtitle = "Dotted red line = chance (E[X] = 0.25). Dashed gray line = 1 atom.",
    x = "Mean Atoms Correct",
    y = "Configuration",
    color = "Model"
  ) +
  scale_x_continuous(
    limits = c(0, 4),
    breaks = seq(0, 4, 0.5)
  ) +
  theme_minimal(base_size = 11) +
  theme(
    axis.text.y = element_text(size = 9, family = "mono"),
    panel.grid.major.y = element_blank(),
```

```
    panel.grid.minor = element_blank(),
    legend.position = "top"
  )

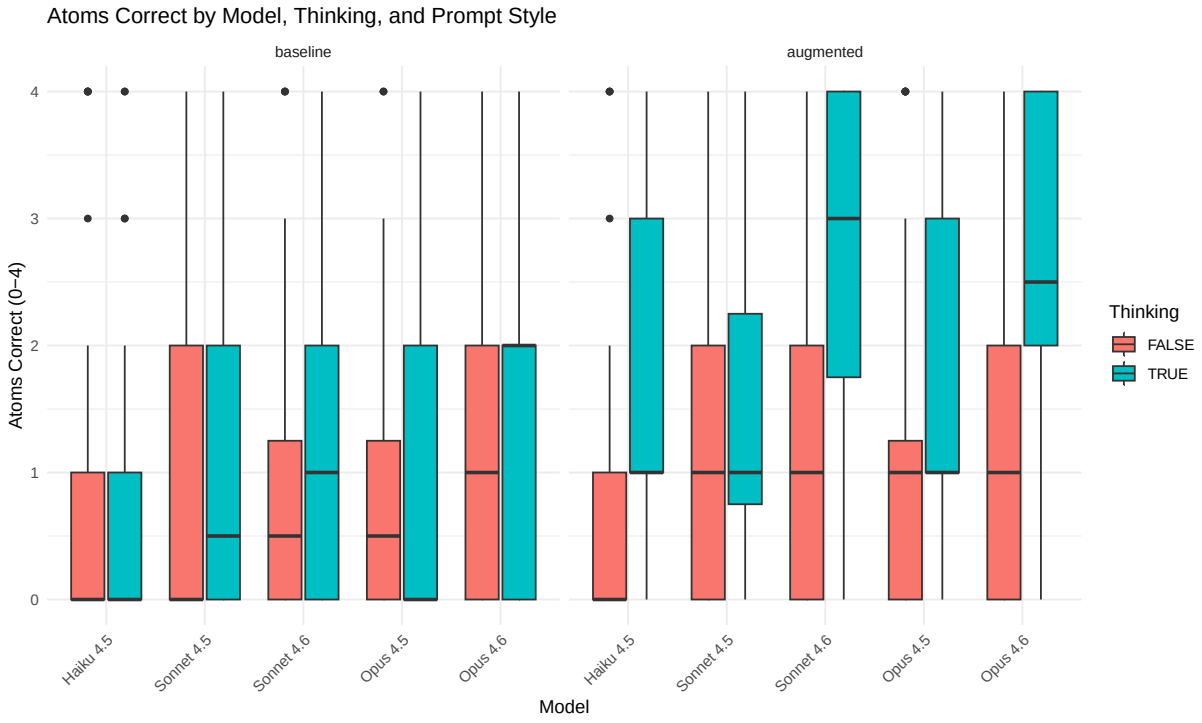
print(p_play_forest)
```


Play Mode: All Configurations Ranked by Atoms Correct
Dotted red line = chance (E[X] = 0.25). Dashed gray line = 1 atom.
Model
Haiku 4.5
Sonnet 4.5
Sonnet 4.6
Opus 4.5
Opus 4.6
Configuration
Mean Atoms Correct
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Haiku 4.5 | Base | Think | Grid State 0.50
Opus 4.6 | Base | No-Think | None 0.60
Haiku 4.5 | Base | No-Think | Grid State 0.60
Sonnet 4.5 | Base | No-Think | None 0.70
Opus 4.6 | Aug | No-Think | Ray Trace 0.70
Opus 4.5 | Base | Think | None 0.70
Sonnet 4.6 | Base | No-Think | None 0.80
Sonnet 4.5 | Base | Think | Grid State 0.80
Sonnet 4.5 | Base | No-Think | Ray Trace 0.80
Opus 4.5 | Base | No-Think | Ray Trace 0.80
Opus 4.5 | Base | No-Think | None 0.80
Opus 4.5 | Base | No-Think | Hypothesis 0.80
Haiku 4.5 | Base | Think | None 0.80
Haiku 4.5 | Base | Think | Hypothesis 0.80
Haiku 4.5 | Base | No-Think | Ray Trace 0.80
Haiku 4.5 | Aug | No-Think | Ray Trace 0.80
Haiku 4.5 | Aug | No-Think | Grid State 0.80
Sonnet 4.6 | Base | No-Think | Ray Trace 0.90
Sonnet 4.6 | Base | No-Think | Grid State 0.90
Sonnet 4.5 | Base | Think | Ray Trace 0.90
Opus 4.6 | Base | No-Think | Ray Trace 0.90
Haiku 4.5 | Base | Think | Ray Trace 0.90
Haiku 4.5 | Base | No-Think | None 0.90
Haiku 4.5 | Aug | No-Think | Hypothesis 0.90
Sonnet 4.6 | Base | Think | None 1.00
Sonnet 4.5 | Base | Think | None 1.00
Opus 4.5 | Base | Think | Ray Trace 1.00
Opus 4.5 | Base | Think | Hypothesis 1.00
Opus 4.5 | Base | Think | Grid State 1.00
Opus 4.5 | Aug | No-Think | Hypothesis 1.00
Haiku 4.5 | Base | No-Think | Hypothesis 1.00
Haiku 4.5 | Aug | No-Think | None 1.00
Sonnet 4.6 | Base | No-Think | Hypothesis 1.10
Sonnet 4.5 | Base | Think | Hypothesis 1.10
Sonnet 4.5 | Base | No-Think | Grid State 1.10
Sonnet 4.5 | Aug | No-Think | Ray Trace 1.10
Sonnet 4.6 | Base | Think | Ray Trace 1.20
Sonnet 4.6 | Aug | No-Think | Ray Trace 1.20
Sonnet 4.5 | Aug | Think | Grid State 1.20
Sonnet 4.5 | Aug | No-Think | None 1.20
Sonnet 4.5 | Aug | No-Think | Hypothesis 1.20
Opus 4.6 | Base | No-Think | Hypothesis 1.20
Opus 4.6 | Aug | No-Think | Hypothesis 1.20
Opus 4.5 | Aug | No-Think | Ray Trace 1.20
Opus 4.5 | Aug | No-Think | Grid State 1.20
Sonnet 4.6 | Base | Think | Hypothesis 1.30
Sonnet 4.6 | Base | Think | Grid State 1.30
Sonnet 4.6 | Aug | No-Think | None 1.30
Sonnet 4.5 | Base | No-Think | Hypothesis 1.30
Opus 4.6 | Base | Think | Grid State 1.30
Opus 4.5 | Base | No-Think | Grid State 1.30
Opus 4.5 | Aug | No-Think | None 1.30
Sonnet 4.5 | Aug | Think | Hypothesis 1.40
Sonnet 4.5 | Aug | No-Think | Grid State 1.40
Opus 4.6 | Aug | No-Think | None 1.40
Sonnet 4.6 | Aug | No-Think | Grid State 1.50
Opus 4.6 | Base | Think | Ray Trace 1.50
Opus 4.6 | Base | Think | Hypothesis 1.50
Opus 4.6 | Aug | No-Think | Grid State 1.50
Opus 4.6 | Base | Think | None 1.60
Opus 4.6 | Base | No-Think | Grid State 1.60
Opus 4.5 | Aug | Think | None 1.60
Sonnet 4.6 | Aug | No-Think | Hypothesis 1.70
Sonnet 4.5 | Aug | Think | Ray Trace 1.70
Opus 4.5 | Aug | Think | Hypothesis 1.70
Haiku 4.5 | Aug | Think | Ray Trace 1.70
Haiku 4.5 | Aug | Think | Grid State 1.70
Haiku 4.5 | Aug | Think | Hypothesis 1.80
Opus 4.6 | Aug | Think | Hypothesis 1.90
Opus 4.5 | Aug | Think | Ray Trace 1.90
Haiku 4.5 | Aug | Think | None 1.90
Sonnet 4.5 | Aug | Think | None 2.00
Opus 4.5 | Aug | Think | Grid State 2.00
Sonnet 4.6 | Aug | Think | Hypothesis 2.40
Sonnet 4.6 | Aug | Think | Grid State 2.40
Opus 4.6 | Aug | Think | None 2.40
Opus 4.6 | Aug | Think | Grid State 2.50
Sonnet 4.6 | Aug | Think | None 2.60
Sonnet 4.6 | Aug | Think | Ray Trace 3.00
Opus 4.6 | Aug | Think | Ray Trace 3.10

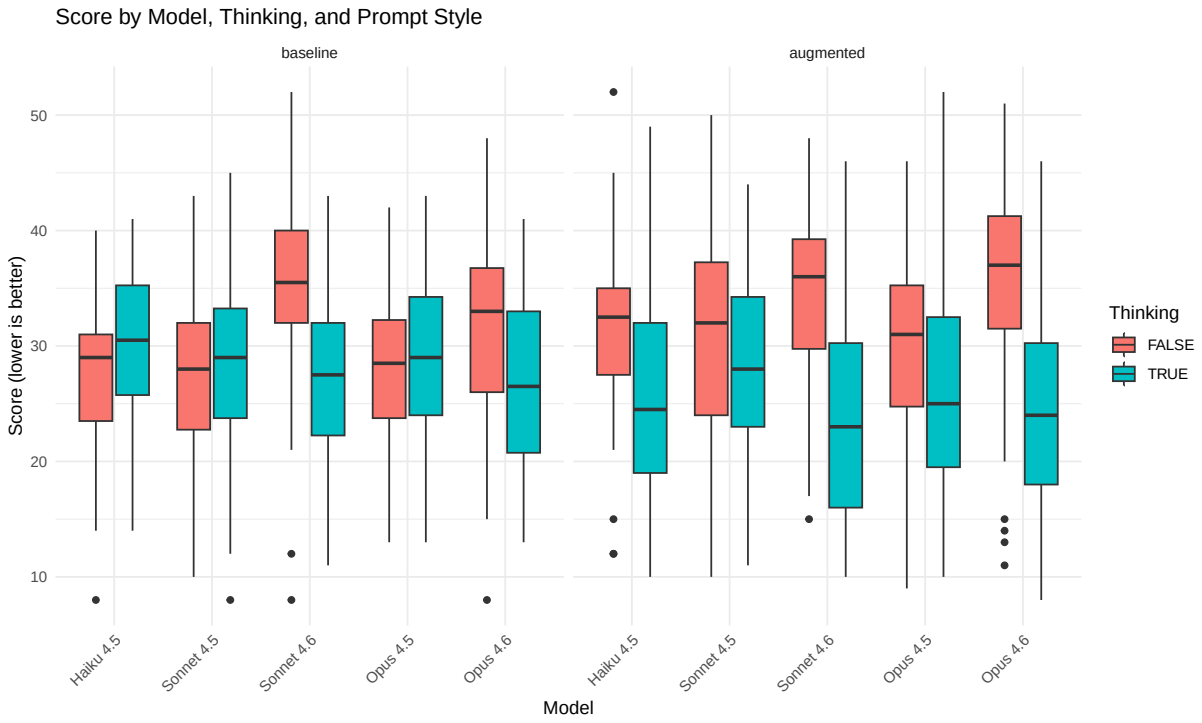
:::

Total configurations: 80 **Best performing**: Opus 4.6 | Aug | Think | Ray Trace (mean = 3.1 atoms correct)

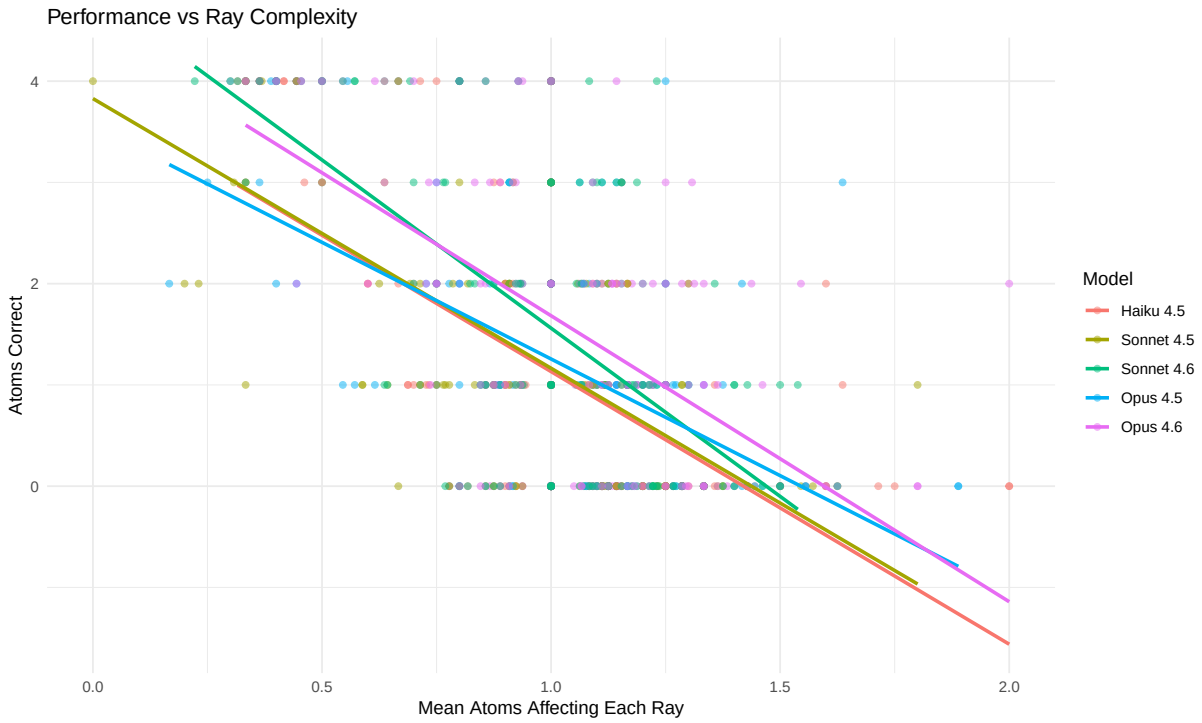
```
# Atoms correct by model and thinking
p1 <- ggplot(play_data, aes(x = model_name, y = atoms_correct, fill = enable_thinking)) +
  geom_boxplot() +
  facet_wrap(~prompt_style) +
  labs(title = "Atoms Correct by Model, Thinking, and Prompt Style",
       x = "Model", y = "Atoms Correct (0-4)", fill = "Thinking") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(p1)
```



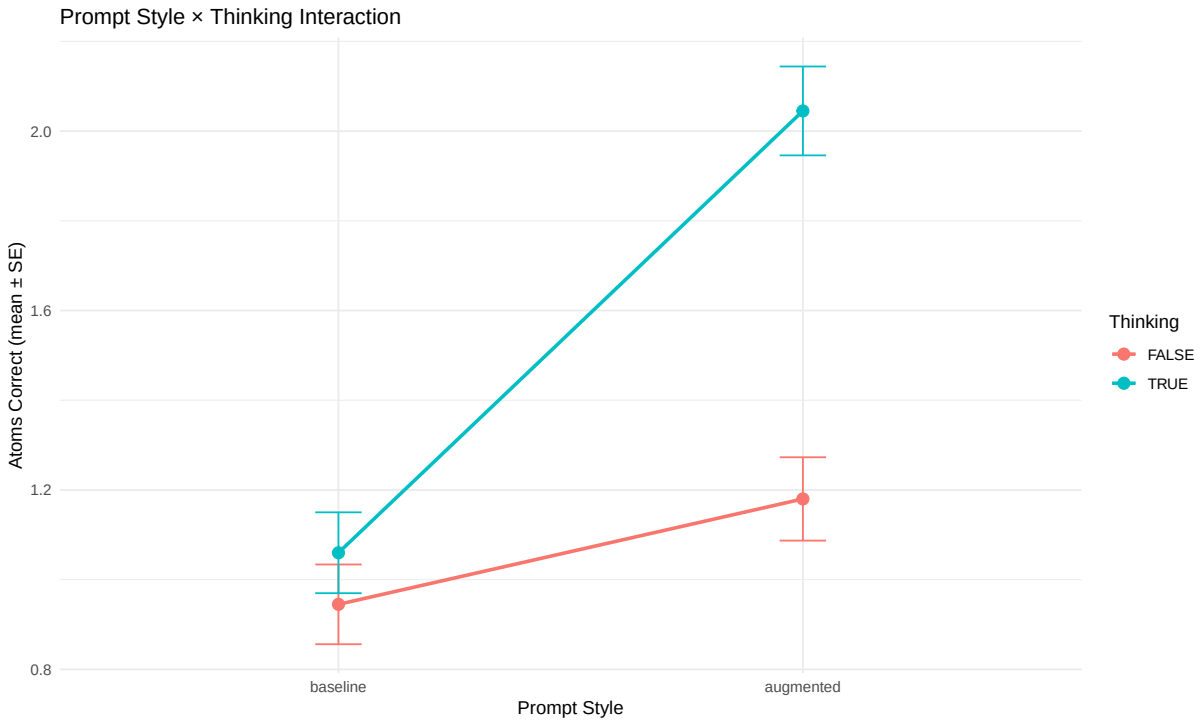

```
# Score distribution
p2 <- ggplot(play_data, aes(x = model_name, y = score, fill = enable_thinking)) +
  geom_boxplot() +
  facet_wrap(~prompt_style) +
  labs(title = "Score by Model, Thinking, and Prompt Style",
       x = "Model", y = "Score (lower is better)", fill = "Thinking") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(p2)
```



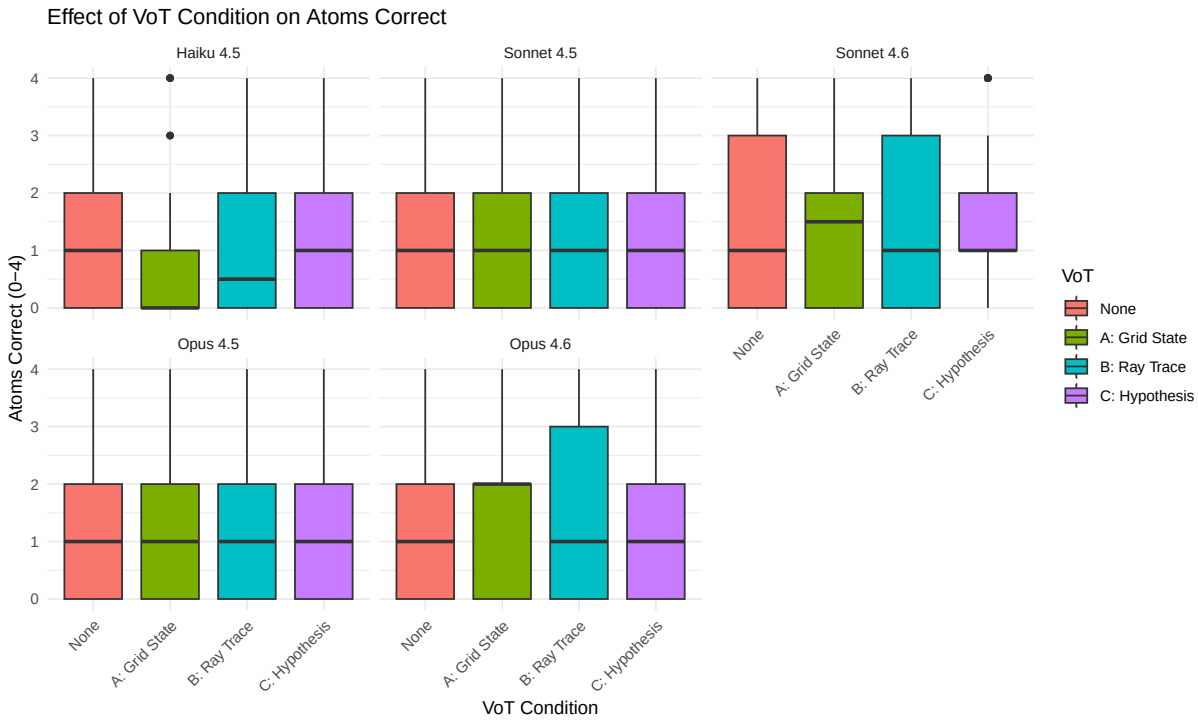

```
# Mean atoms per ray relationship with performance
p3 <- ggplot(play_data, aes(x = mean_atoms_per_ray, y = atoms_correct, color = model_name)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Performance vs Ray Complexity",
       x = "Mean Atoms Affecting Each Ray", y = "Atoms Correct",
       color = "Model") +
  theme_minimal()
print(p3)
```



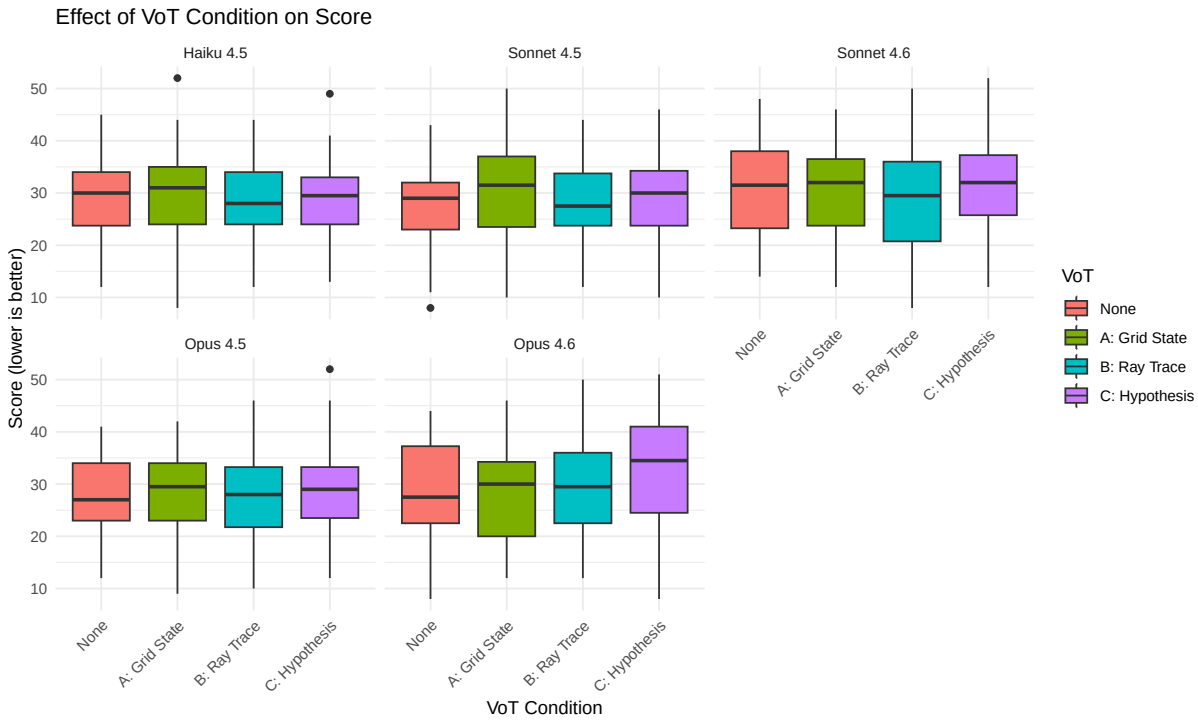

```
# Interaction plot: Prompt Style × Thinking
p4 <- ggplot(play_data, aes(x = prompt_style, y = atoms_correct,
                            color = enable_thinking, group = enable_thinking)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line", linewidth = 1) +
  stat_summary(fun.data = mean_se, geom = "errorbar", width = 0.1) +
  labs(title = "Prompt Style × Thinking Interaction",
       x = "Prompt Style", y = "Atoms Correct (mean ± SE)",
       color = "Thinking") +
  theme_minimal()
print(p4)
```




```
# VoT condition effect on atoms correct
p5 <- ggplot(play_data, aes(x = vot_condition, y = atoms_correct, fill = vot_condition)) +
  geom_boxplot() +
  facet_wrap(~model_name) +
  labs(title = "Effect of VoT Condition on Atoms Correct",
       x = "VoT Condition", y = "Atoms Correct (0-4)", fill = "VoT") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(p5)
```




```
# VoT condition effect on score
p6 <- ggplot(play_data, aes(x = vot_condition, y = score, fill = vot_condition)) +
  geom_boxplot() +
  facet_wrap(~model_name) +
  labs(title = "Effect of VoT Condition on Score",
       x = "VoT Condition", y = "Score (lower is better)", fill = "VoT") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(p6)
```



4.3 Predict vs Play: Cross-Mode Comparison

This section examines whether configuration difficulty and model performance are consistent across the two reasoning modes. Predict mode tests **forward reasoning** (given atoms, predict ray behavior), while Play mode tests **inverse reasoning** (observe ray behavior, infer atom locations). If forward and inverse reasoning rely on shared spatial representations, we expect configuration difficulty and model rankings to correlate across modes.

```
# Predict: accuracy by configuration (aggregated across all conditions)
predict_by_config <- predict_data |>
  mutate(config_index = as.integer(config_index)) |>
  group_by(config_index) |>
  summarise(
    predict_n = n(),
    predict_accuracy = mean(correct),
    predict_se = sqrt(predict_accuracy * (1 - predict_accuracy) / predict_n),
    .groups = "drop"
  )

# Play: performance by configuration (aggregated across all conditions)
play_by_config <- play_data |>
```

```r
  mutate(config_index = as.integer(config_index)) |>
  group_by(config_index) |>
  summarise(
    play_n = n(),
    play_atoms_correct = mean(atoms_correct),
    play_atoms_sd = sd(atoms_correct),
    play_score = mean(score),
    play_score_sd = sd(score),
    .groups = "drop"
  )

# Join predict and play by configuration
cross_mode <- inner_join(predict_by_config, play_by_config, by = "config_index") |>
  mutate(across(where(is.numeric) & !c(config_index, predict_n, play_n),
                \(x) round(x, 3)))

kable(cross_mode,
      col.names = c("Config", "N (Predict)", "Accuracy", "SE",
                    "N (Play)", "Atoms Correct (M)", "Atoms Correct (SD)",
                    "Score (M)", "Score (SD)"),
      caption = "Configuration-Level Performance: Predict vs Play Mode")
```

Table 29: Configuration-Level Performance: Predict vs Play Mode

Config	N (Predict)	Accuracy	SE	N (Play)	Atoms Correct (M)	Atoms Correct (SD)	Score (M)	Score (SD)
1	1840	0.641	0.011	80	1.900	0.836	27.238	6.530
2	1760	0.769	0.010	80	1.913	1.647	23.725	7.386
3	2000	0.786	0.009	80	1.475	1.102	24.875	7.414
4	2240	0.713	0.010	80	0.275	0.595	39.987	6.602
5	1440	0.779	0.011	80	0.812	0.943	33.562	6.063
6	1680	0.824	0.009	80	3.775	0.551	15.787	5.566
7	1920	0.838	0.008	80	0.400	0.851	34.112	7.183
8	1840	0.697	0.011	80	0.875	0.905	30.975	5.994
9	2000	0.787	0.009	80	1.087	0.860	27.012	5.235

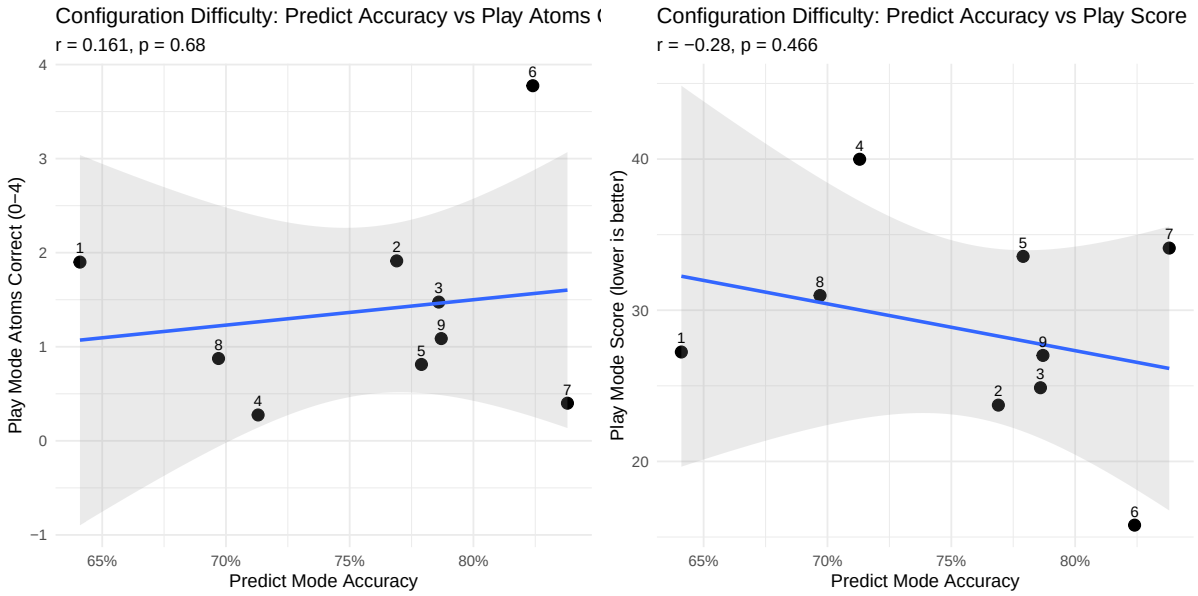
4.3.1 Difficulty Profile Correlation

```r
# Correlation between predict accuracy and play atoms correct
cor_atoms <- cor.test(cross_mode$predict_accuracy, cross_mode$play_atoms_correct)
cor_score <- cor.test(cross_mode$predict_accuracy, cross_mode$play_score)

p_cross1 <- ggplot(cross_mode, aes(x = predict_accuracy, y = play_atoms_correct)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = TRUE, alpha = 0.2) +
  geom_text(aes(label = config_index), vjust = -0.8, size = 3) +
  labs(
    title = "Configuration Difficulty: Predict Accuracy vs Play Atoms Correct",
    subtitle = paste0("r = ", round(cor_atoms$estimate, 3),
                      ", p = ", format.pval(cor_atoms$p.value, digits = 3)),
    x = "Predict Mode Accuracy",
    y = "Play Mode Atoms Correct (0-4)"
  ) +
  scale_x_continuous(labels = scales::percent) +
  theme_minimal()

p_cross2 <- ggplot(cross_mode, aes(x = predict_accuracy, y = play_score)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = TRUE, alpha = 0.2) +
  geom_text(aes(label = config_index), vjust = -0.8, size = 3) +
  labs(
    title = "Configuration Difficulty: Predict Accuracy vs Play Score",
    subtitle = paste0("r = ", round(cor_score$estimate, 3),
                      ", p = ", format.pval(cor_score$p.value, digits = 3)),
    x = "Predict Mode Accuracy",
    y = "Play Mode Score (lower is better)"
  ) +
  scale_x_continuous(labels = scales::percent) +
  theme_minimal()

gridExtra::grid.arrange(p_cross1, p_cross2, ncol = 2)
```



Configuration difficulty shows **weak or no correlation** across modes, suggesting forward and inverse reasoning may rely on different cognitive demands.

4.3.2 Model Rankings Across Modes

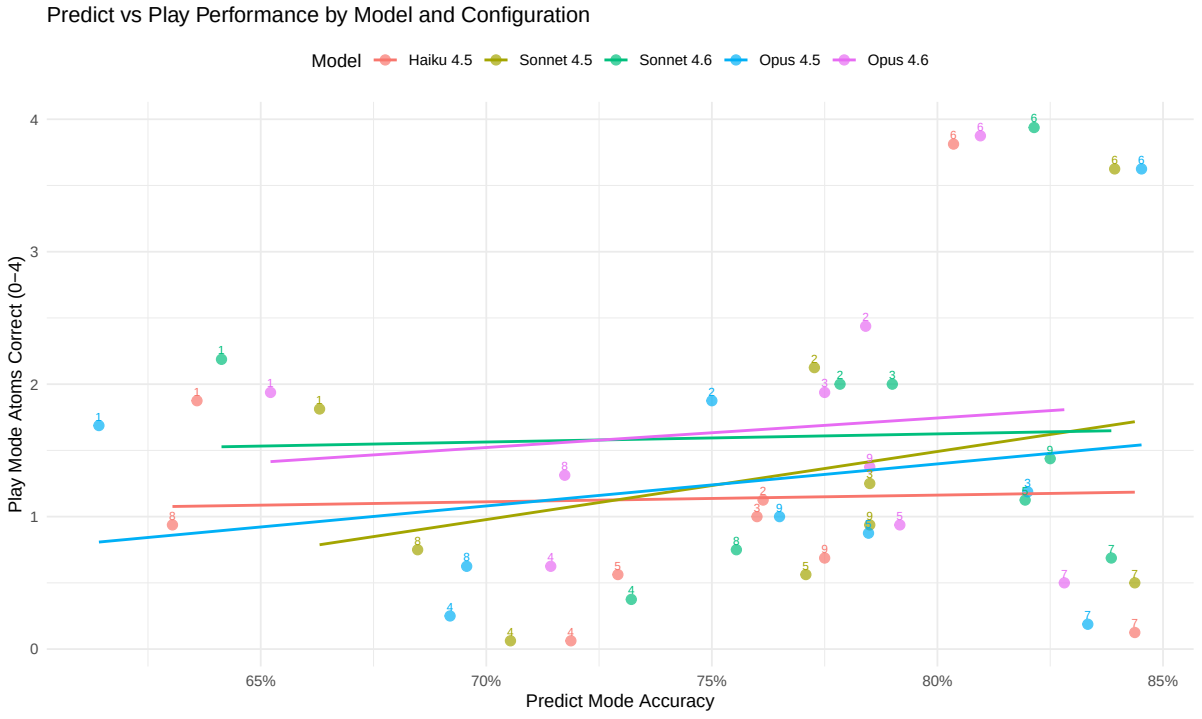
```
# Predict: accuracy by model and configuration
predict_by_model_config <- predict_data |>
  mutate(config_index = as.integer(config_index)) |>
  group_by(model_name, config_index) |>
  summarise(
    predict_accuracy = mean(correct),
    .groups = "drop"
  )

# Play: atoms correct by model and configuration
play_by_model_config <- play_data |>
  mutate(config_index = as.integer(config_index)) |>
  group_by(model_name, config_index) |>
  summarise(
    play_atoms_correct = mean(atoms_correct),
    .groups = "drop"
  )

# Join by model and configuration
```

```r
model_config_cross <- inner_join(predict_by_model_config, play_by_model_config,
                                 by = c("model_name", "config_index"))

# Scatter: predict accuracy vs play atoms correct, colored by model
p_model_cross <- ggplot(model_config_cross,
                        aes(x = predict_accuracy, y = play_atoms_correct,
                            color = model_name)) +
  geom_point(size = 2.5, alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  geom_text(aes(label = config_index), vjust = -0.6, size = 2.5, show.legend = FALSE) +
  labs(
    title = "Predict vs Play Performance by Model and Configuration",
    x = "Predict Mode Accuracy",
    y = "Play Mode Atoms Correct (0-4)",
    color = "Model"
  ) +
  scale_x_continuous(labels = scales::percent) +
  theme_minimal() +
  theme(legend.position = "top")
print(p_model_cross)
```



```r
# Per-model correlations
model_cors <- model_config_cross |>
  group_by(model_name) |>
  summarise(
    n_configs = n(),
    r = cor(predict_accuracy, play_atoms_correct),
    .groups = "drop"
  ) |>
  mutate(r = round(r, 3))

kable(model_cors,
      col.names = c("Model", "N Configs", "r (Predict-Play)"),
      caption = "Within-Model Correlation Between Predict Accuracy and Play Atoms Correct")
```

Table 30: Within-Model Correlation Between Predict Accuracy and Play Atoms Correct

Model	N Configs	r (Predict–Play)
Haiku 4.5	9	0.031
Sonnet 4.5	9	0.302
Sonnet 4.6	9	0.035
Opus 4.5	9	0.230
Opus 4.6	9	0.119

4.3.3 Aggregate Model Performance Comparison

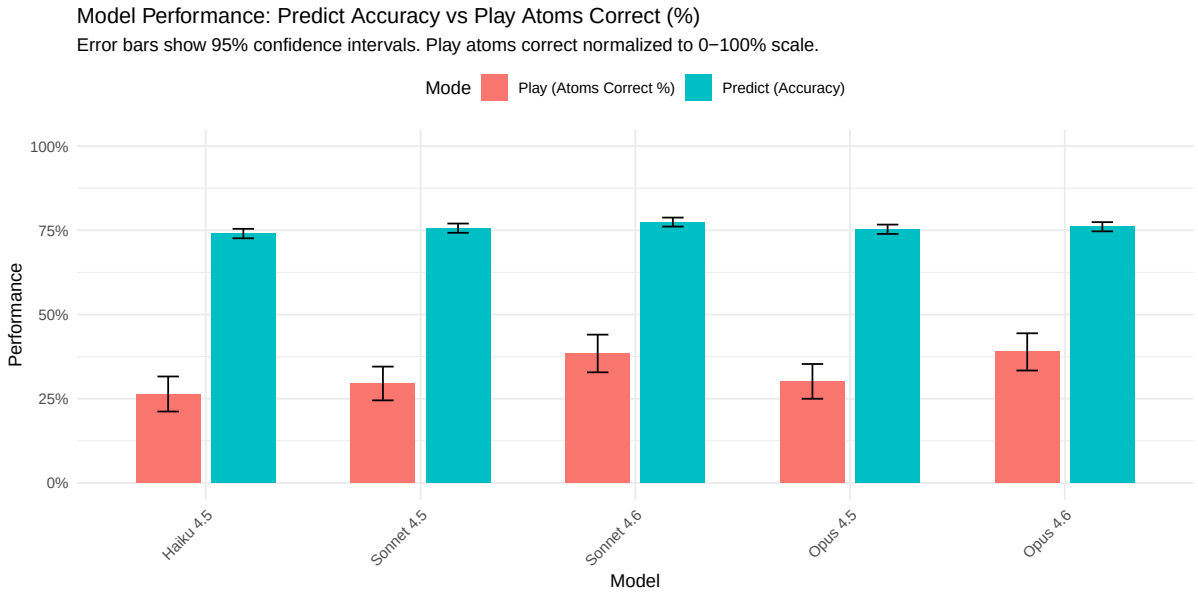
```r
# Model-level aggregates
predict_model_agg <- predict_data |>
  group_by(model_name) |>
  summarise(
    predict_accuracy = mean(correct),
    predict_se = sqrt(predict_accuracy * (1 - predict_accuracy) / n()),
    .groups = "drop"
  )

play_model_agg <- play_data |>
  group_by(model_name) |>
  summarise(
    play_atoms_pct = mean(atoms_correct) / 4,  # normalize to 0-1 scale
    play_atoms_se = sd(atoms_correct) / (4 * sqrt(n())),
```

```
    .groups = "drop"
  )

model_comparison <- inner_join(predict_model_agg, play_model_agg, by = "model_name") |>
  pivot_longer(
    cols = c(predict_accuracy, play_atoms_pct),
    names_to = "mode",
    values_to = "performance"
  ) |>
  mutate(
    se = ifelse(mode == "predict_accuracy", predict_se, play_atoms_se),
    mode = ifelse(mode == "predict_accuracy", "Predict (Accuracy)", "Play (Atoms Correct %)"
  )

p_compare <- ggplot(model_comparison, aes(x = model_name, y = performance, fill = mode)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_errorbar(aes(ymin = performance - 1.96 * se, ymax = performance + 1.96 * se),
                position = position_dodge(width = 0.7), width = 0.2) +
  scale_y_continuous(labels = scales::percent, limits = c(0, 1)) +
  labs(
    title = "Model Performance: Predict Accuracy vs Play Atoms Correct (%)",
    subtitle = "Error bars show 95% confidence intervals. Play atoms correct normalized to 0-
    x = "Model", y = "Performance", fill = "Mode"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "top")
print(p_compare)
```



4.3.4 Forward–Inverse Reasoning Gap

```
# Per-model gap between predict and play
gap_data <- inner_join(predict_model_agg, play_model_agg, by = "model_name") |>
  mutate(
    gap = predict_accuracy - play_atoms_pct,
    gap_pct = paste0(round(gap * 100, 1), " pp")
  )

kable(gap_data |>
        mutate(across(c(predict_accuracy, play_atoms_pct, gap), \(x) round(x, 3))) |>
        select(model_name, predict_accuracy, play_atoms_pct, gap, gap_pct),
      col.names = c("Model", "Predict Accuracy", "Play Atoms %", "Gap", "Gap (pp)"),
      caption = "Forward-Inverse Reasoning Gap by Model (positive = predict outperforms play)
```

Table 31: Forward–Inverse Reasoning Gap by Model (positive = predict outperforms play)

Model	Predict Accuracy	Play Atoms %	Gap	Gap (pp)
Haiku 4.5	0.740	0.264	0.476	47.6 pp
Sonnet 4.5	0.756	0.295	0.461	46.1 pp
Sonnet 4.6	0.774	0.384	0.390	39 pp
Opus 4.5	0.753	0.302	0.452	45.2 pp
Opus 4.6	0.761	0.389	0.372	37.2 pp

Model	Predict Accuracy	Play Atoms %	Gap	Gap (pp)

All models show a **positive gap**: forward reasoning (predict) consistently outperforms inverse reasoning (play), consistent with the theoretical expectation that abductive inference is harder than deductive prediction.

4.4 Chance Performance Baseline

To interpret LLM performance meaningfully, we need a lower bound: what would happen if an agent simply guessed 4 atom positions at random with no information? This chance baseline establishes the floor of the performance corridor. Any agent that extracts meaningful signal from ray observations should substantially exceed this level.

4.4.1 Hypergeometric Derivation

An agent guessing 4 positions uniformly at random from the 64-cell grid, with 4 true atoms hidden, follows a hypergeometric distribution:

$$X \sim \text{Hypergeometric}(N = 64,\ K = 4,\ n = 4)$$

where X counts the number of correctly guessed atoms. The probability mass function is:

$$P(X = k) = \frac{\binom{4}{k}\binom{60}{4-k}}{\binom{64}{4}}, \quad k = 0, 1, 2, 3, 4$$

The expected value is $E[X] = nK/N = 4 \times 4/64 = 0.25$ atoms correct. Since the scoring formula is score $=$ rays_used $+ 5 \times$ atoms_missed, a random guesser who fires zero rays achieves an expected score of $E[\text{score}] = 5 \times (4 - 0.25) = 18.75$. This represents a theoretical ceiling on bad performance—any rational agent should score *below* 18.75 on average.

```
# Exact hypergeometric PMF for random guessing
chance_pmf <- tibble(
  k = 0:4,
  prob = dhyper(k, 4, 60, 4),
  missed = 4 - k,
  penalty = 5 * missed,
  score = penalty  # no rays fired
)
```

```
chance_EX <- sum(chance_pmf$k * chance_pmf$prob)
chance_SD <- sqrt(sum((chance_pmf$k - chance_EX)^2 * chance_pmf$prob))
chance_Escore <- sum(chance_pmf$score * chance_pmf$prob)

kable(chance_pmf |>
        mutate(prob = sprintf("%.4f", prob)) |>
        rename(`Atoms Correct (k)` = k,
               `P(X = k)` = prob,
               `Atoms Missed` = missed,
               `Miss Penalty` = penalty,
               `Score (0 rays)` = score),
      caption = "Hypergeometric PMF for Random Guessing (N=64, K=4, n=4)",
      align = "ccccc") |>
  kable_styling(full_width = FALSE)
```

Table 32: Hypergeometric PMF for Random Guessing (N=64, K=4, n=4)

Atoms Correct (k)	P(X = k)	Atoms Missed	Miss Penalty	Score (0 rays)
0	0.7675	4	20	20
1	0.2154	3	15	15
2	0.0167	2	10	10
3	0.0004	1	5	5
4	0.0000	0	0	0

A random guesser finds zero atoms 76.7% of the time, exactly one atom 21.5% of the time, and two or more atoms only 1.7% of the time. The expected number of correct atoms is 0.25 (SD = 0.47), yielding an expected score of 18.75.

4.4.2 LLM Performance Above Chance

```
# Per-model comparison against chance baseline
chance_comparison <- play_data |>
  group_by(model_name) |>
  summarise(
    n = n(),
    mean_atoms = mean(atoms_correct),
    sd_atoms = sd(atoms_correct),
    mean_score = mean(score),
    atoms_above_chance = mean_atoms - chance_EX,
```

```
    cohens_d = (mean_atoms - chance_EX) / sd_atoms,
    t_stat = (mean_atoms - chance_EX) / (sd_atoms / sqrt(n())),
    p_value = pt(t_stat, df = n() - 1, lower.tail = FALSE),
    score_below_chance = chance_Escore - mean_score,
    .groups = "drop"
  ) |>
  mutate(model_name = factor(as.character(model_name),
                             levels = order_model_names(as.character(model_name))))

kable(chance_comparison |>
        arrange(model_name) |>
        mutate(
          across(c(mean_atoms, sd_atoms, atoms_above_chance, cohens_d, score_below_chance),
                 \(x) round(x, 2)),
          mean_score = round(mean_score, 1),
          p_value = ifelse(p_value < 0.001, "< 0.001", sprintf("%.3f", p_value))
        ) |>
        select(model_name, n, mean_atoms, sd_atoms, mean_score,
               atoms_above_chance, cohens_d, p_value, score_below_chance),
      col.names = c("Model", "N", "Mean Atoms", "SD", "Mean Score",
                    "Atoms Above Chance", "Cohen's d", "p-value",
                    "Score Below Chance"),
      caption = "LLM Performance vs Chance Baseline (E[X] = 0.25 atoms, E[score] = 18.75)",
      align = "lccccccccc") |>
  kable_styling(full_width = FALSE)
```

Table 33: LLM Performance vs Chance Baseline (E[X] = 0.25 atoms, E[score] = 18.75)

Model	N	Mean Atoms	SD	Mean Score	Atoms Above Chance	Cohen's d	p-value	Score B
Haiku 4.5	160	1.06	1.34	28.6	0.81	0.60	< 0.001	
Sonnet 4.5	160	1.18	1.29	28.5	0.93	0.72	< 0.001	
Sonnet 4.6	160	1.54	1.44	30.2	1.29	0.89	< 0.001	-
Opus 4.5	160	1.21	1.33	28.0	0.96	0.72	< 0.001	
Opus 4.6	160	1.56	1.43	29.5	1.31	0.92	< 0.001	-

All models perform **significantly above chance** (all $p < 0.05$). The overall mean of 1.31 atoms correct is 5.2× the chance expectation, confirming that LLMs extract meaningful signal from ray observations.

```r
# Observed LLM distribution
llm_atom_dist <- play_data |>
  count(atoms_correct) |>
  mutate(
    proportion = n / sum(n),
    source = "LLM (Observed)"
  ) |>
  rename(k = atoms_correct)

# Chance distribution
chance_dist <- chance_pmf |>
  mutate(
    n = prob * nrow(play_data),  # scale to same total for visual comparison
    proportion = prob,
    source = "Chance (Hypergeometric)"
  ) |>
  select(k, n, proportion, source)

# Combined for plotting
dist_combined <- bind_rows(llm_atom_dist, chance_dist)

ggplot(dist_combined, aes(x = factor(k), y = proportion, fill = source)) +
  geom_col(position = "dodge", width = 0.7) +
  scale_fill_manual(values = c("Chance (Hypergeometric)" = "#E57373",
                               "LLM (Observed)" = "#4CAF50")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    x = "Atoms Correct",
    y = "Proportion",
    fill = NULL,
    title = "Atoms Correct Distribution: Chance vs LLM",
    subtitle = "Chance baseline concentrates at k = 0; LLMs shift dramatically rightward"
  ) +
  theme_minimal() +
  theme(legend.position = "top")
```

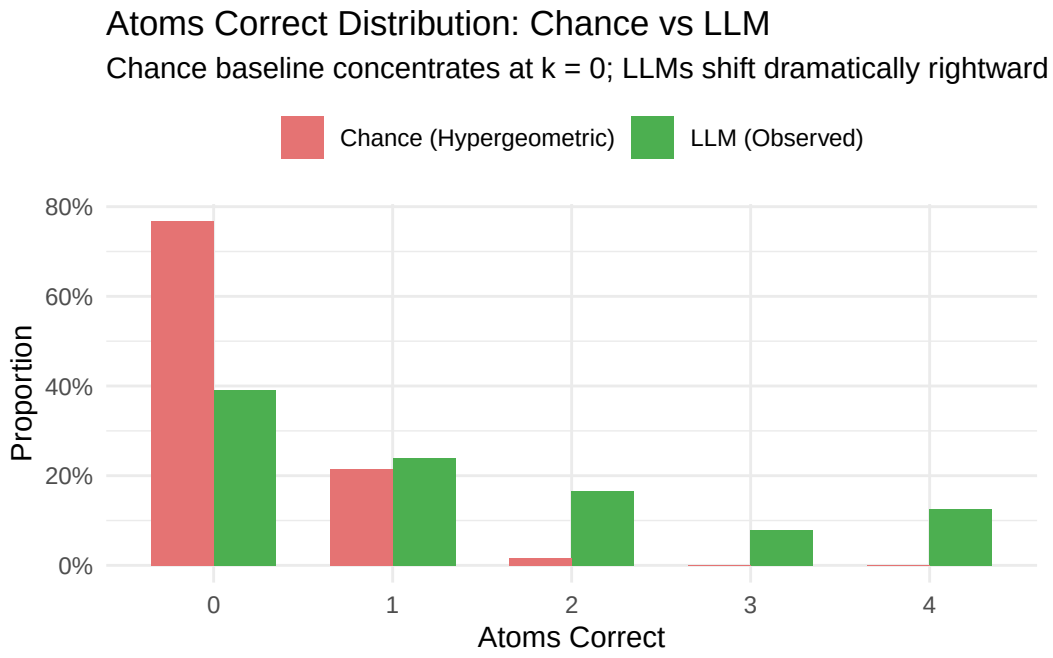


Figure 1: Distribution of Atoms Correct: Chance (Hypergeometric) vs Observed LLM Performance

```
model_score_summary <- play_data |>
  group_by(model_name) |>
  summarise(
    mean_score = mean(score),
    se_score = sd(score) / sqrt(n()),
    ci_lower = mean_score - 1.96 * se_score,
    ci_upper = mean_score + 1.96 * se_score,
    .groups = "drop"
  ) |>
  mutate(model_name = factor(as.character(model_name),
                             levels = order_model_names(as.character(model_name))))

ggplot(model_score_summary, aes(x = model_name, y = mean_score)) +
  geom_hline(yintercept = chance_Escore, linetype = "dotted", color = "red", linewidth = 0.8
  annotate("text", x = -Inf, y = chance_Escore, label = paste0("Chance (", chance_Escore, ")"
           hjust = -0.05, vjust = -0.5, size = 3, color = "red") +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = ci_lower, ymax = ci_upper), width = 0.2, linewidth = 0.8) +
  labs(
    x = "Model",
    y = "Mean Score (lower is better)",
```

```
    title = "Mean Score by Model vs Chance Baseline",
    subtitle = "Error bars show 95% CI. Red dotted line = chance score (no rays, random guess
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```

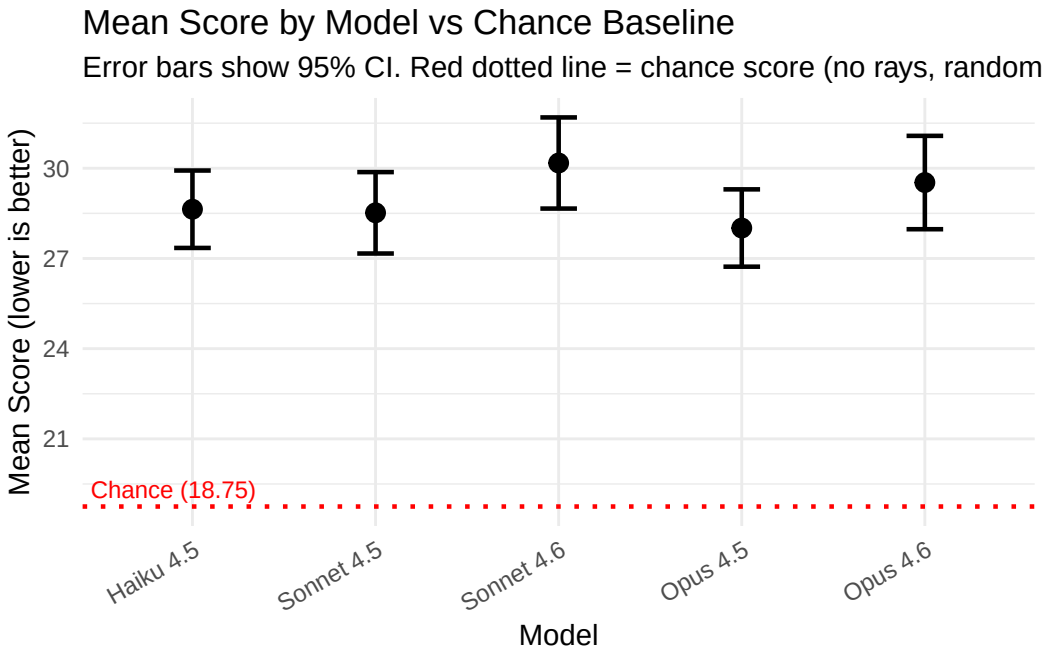


Figure 2: Mean Score by Model with Chance Baseline Reference

The optimal solver benchmark in the next section completes the performance corridor, providing the lower bound (best achievable score) against which both chance and LLM performance can be measured.

4.5 Optimal Solver Benchmark

The greedy optimal solver uses an information-theoretic strategy that minimizes expected remaining candidates at each step over the full C(64,4) = 635,376 candidate space. This provides a principled upper bound on play performance: the fewest rays and lowest score achievable by a perfectly rational agent with unlimited computation.

```
# Load optimal solver benchmark results
optimal_json_path <- "optimal_solver_results.json"
has_optimal <- file.exists(optimal_json_path)
```

```
if (has_optimal) {
  optimal_raw <- fromJSON(optimal_json_path)

  # fromJSON flattens configs into a data frame with score as nested df
  optimal_data <- tibble(
    config_index = optimal_raw$configs$config_index,
    optimal_rays = optimal_raw$configs$optimal_rays,
    optimal_ray_points = optimal_raw$configs$score$ray_points,
    optimal_score = optimal_raw$configs$score$total,
    optimal_solved = optimal_raw$configs$solved
  )

  cat("Optimal solver benchmark loaded:",
      optimal_raw$metadata$n_configs, "configs,",
      "strategy:", optimal_raw$metadata$strategy, "\n")
  cat("Configs hash:", optimal_raw$metadata$configs_hash, "\n")
  cat("Generated:", optimal_raw$metadata$generated_at, "\n")
} else {
  warning("optimal_solver_results.json not found. Run: python blackbox_solver.py benchmark")
}
```

```
Optimal solver benchmark loaded: 10 configs, strategy: greedy_optimal
Configs hash: 44b512d953aa6111
Generated: 2026-03-01T17:14:02.565821+00:00
```

```
# Identify the best condition by lowest mean score (most comparable to optimal solver)
best_condition_lookup <- play_data |>
  mutate(
    condition = paste0(
      model_name, " | ",
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      vot_condition
    )
  ) |>
  group_by(condition, model_name) |>
  summarise(
    score_mean = mean(score),
    rays_mean = mean(rays_used),
    atoms_mean = mean(atoms_correct),
    n = n(),
```

```
    .groups = "drop"
  ) |>
  arrange(score_mean) |>
  slice(1)

best_condition_label <- best_condition_lookup$condition

# Aggregate best condition's data per config_index
best_config_agg <- play_data |>
  mutate(
    condition = paste0(
      model_name, " | ",
      ifelse(prompt_style == "augmented", "Aug", "Base"), " | ",
      ifelse(as.logical(as.character(enable_thinking)), "Think", "No-Think"), " | ",
      vot_condition
    )
  ) |>
  filter(condition == best_condition_label) |>
  group_by(config_index) |>
  summarise(
    best_rays_mean = mean(rays_used),
    best_atoms_mean = mean(atoms_correct),
    best_score_mean = mean(score),
    best_n = n(),
    .groups = "drop"
  )

# Short label for chart legends (just model name)
best_short_label <- paste0("Best (", best_condition_lookup$model_name, ")")
```

```
kable(optimal_data |>
        select(config_index, optimal_rays, optimal_ray_points, optimal_score, optimal_solved
      col.names = c("Config", "Rays", "Ray Points", "Total Score", "Solved"),
      caption = "Optimal Solver Performance by Configuration (greedy strategy, no missed ato
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"))
```

Table 34: Optimal Solver Performance by Configuration (greedy strategy, no missed atoms)

Config	Rays	Ray Points	Total Score	Solved
0	8	12	12	TRUE
1	9	13	13	TRUE

2	7	11	11	TRUE
3	7	9	9	TRUE
4	7	12	12	TRUE
5	9	14	14	TRUE
6	8	12	12	TRUE
7	8	12	12	TRUE
8	8	10	10	TRUE
9	5	8	8	TRUE

The greedy optimal solver solves all 10 configurations in a mean of **7.6 rays** (range: 5–9) with a mean score of **11.3** (range: 8–14). Since it always identifies all 4 atoms, the score consists entirely of ray points with no missed-atom penalty.

4.5.1 LLM vs Optimal Comparison

```
# Aggregate play data by config
play_config_agg <- play_data |>
  group_by(config_index) |>
  summarise(
    n_games = n(),
    llm_rays_mean = mean(rays_used),
    llm_atoms_mean = mean(atoms_correct),
    llm_score_mean = mean(score),
    .groups = "drop"
  )

# Join with optimal data and best condition data
comparison <- inner_join(play_config_agg, optimal_data, by = "config_index") |>
  left_join(best_config_agg, by = "config_index") |>
  mutate(
    rays_ratio = round(llm_rays_mean / optimal_rays, 2),
    score_ratio = round(llm_score_mean / optimal_score, 2),
    best_rays_ratio = round(best_rays_mean / optimal_rays, 2),
    best_score_ratio = round(best_score_mean / optimal_score, 2)
  )

kable(comparison |>
        mutate(across(c(llm_rays_mean, llm_atoms_mean, llm_score_mean,
                        best_rays_mean, best_atoms_mean, best_score_mean), \(x) round(x, 1))) |>
        select(config_index, optimal_rays, best_rays_mean, best_rays_ratio,
```

```
              llm_rays_mean, rays_ratio,
              optimal_score, best_score_mean, best_score_ratio,
              llm_score_mean, score_ratio,
              best_atoms_mean, llm_atoms_mean),
    col.names = c("Config", "Opt Rays", "Best Rays (M)", "Best Ratio",
                  "LLM Rays (M)", "LLM Ratio",
                  "Opt Score", "Best Score (M)", "Best Ratio",
                  "LLM Score (M)", "LLM Ratio",
                  "Best Atoms (M)", "LLM Atoms (M)"),
    caption = paste0("Optimal vs Best Condition vs LLM Mean (Best = ",
                     best_condition_label, "; ratio = agent / optimal)")) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed")) |>
  scroll_box(width = "100%")
```

Table 35: Optimal vs Best Condition vs LLM Mean (Best = Trace; ratio = agent / optimal)

Config	Opt Rays	Best Rays (M)	Best Ratio	LLM Rays (M)	LLM Ratio	Opt Score	Best Score (M
0	8	13	1.62	10.0	1.25	12	1
1	9	13	1.44	11.6	1.29	13	2
2	7	11	1.57	10.4	1.48	11	2
3	7	8	1.14	10.6	1.51	9	1
4	7	12	1.71	12.3	1.76	12	3
5	9	16	1.78	11.9	1.33	14	2
6	8	14	1.75	11.3	1.42	12	1
7	8	7	0.88	10.5	1.31	12	1
8	8	10	1.25	11.8	1.48	10	1
9	5	11	2.20	9.5	1.90	8	2

Across configurations, LLMs use **1.5×** as many rays as the optimal solver on average (range: 1.2×–1.9×) and achieve scores **2.6×** the optimal (range: 1.3×–3.4×). The best condition (Sonnet 4.6 | Aug | Think | B: Ray Trace) narrows the gap: **1.5×** rays (range: 0.9×–2.2×) and **1.9×** score (range: 1.1×–3×). Score ratios above 1.0× reflect both extra ray costs and missed-atom penalties.

```
rays_long <- comparison |>
  select(config_index, optimal_rays, best_rays_mean, llm_rays_mean) |>
  pivot_longer(cols = c(optimal_rays, best_rays_mean, llm_rays_mean),
               names_to = "agent", values_to = "rays") |>
  mutate(agent = case_when(
```

```
    agent == "optimal_rays" ~ "Optimal Solver",
    agent == "best_rays_mean" ~ best_short_label,
    TRUE ~ "LLM (Mean)"
  )) |>
  mutate(agent = factor(agent, levels = c("Optimal Solver", best_short_label, "LLM (Mean)")))

ggplot(rays_long, aes(x = config_index, y = rays, color = agent, group = agent)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = setNames(
    c("#2196F3", "#4CAF50", "#FF9800"),
    c("Optimal Solver", best_short_label, "LLM (Mean)")
  )) +
  scale_x_continuous(breaks = unique(rays_long$config_index)) +
  labs(x = "Configuration", y = "Rays Used", color = NULL,
       title = "Rays Used: Optimal vs Best Condition vs LLM Mean") +
  theme_minimal() +
  theme(legend.position = "top")
```

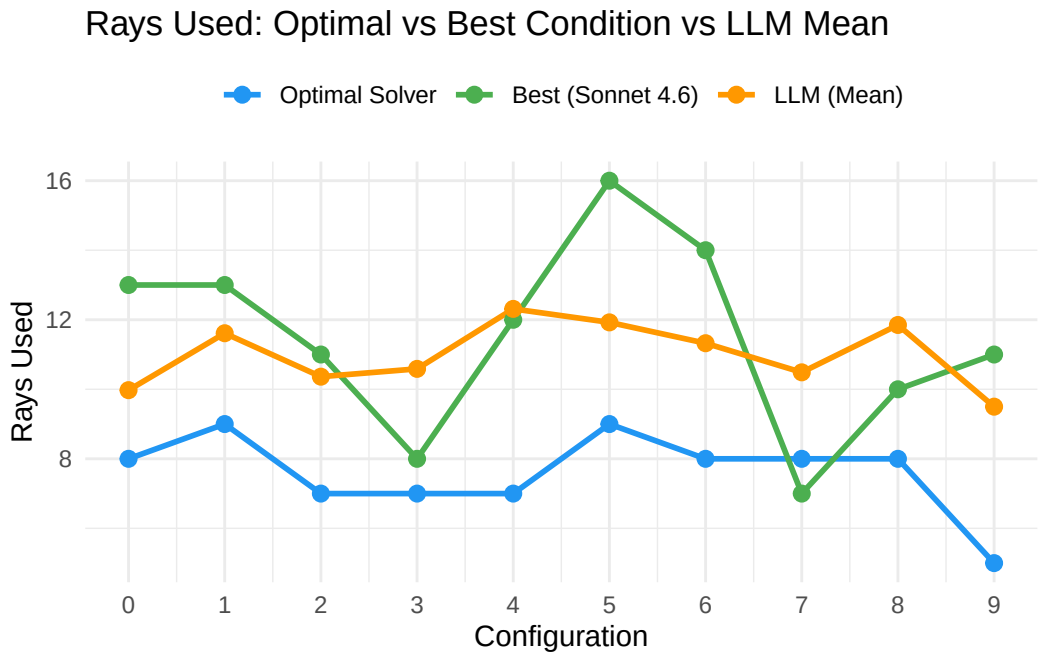


Figure 3: Rays Used: LLM (mean across conditions) vs Optimal Solver by Configuration

```r
score_long <- comparison |>
  select(config_index, optimal_score, best_score_mean, llm_score_mean) |>
  pivot_longer(cols = c(optimal_score, best_score_mean, llm_score_mean),
               names_to = "agent", values_to = "score_val") |>
  mutate(agent = case_when(
    agent == "optimal_score" ~ "Optimal Solver",
    agent == "best_score_mean" ~ best_short_label,
    TRUE ~ "LLM (Mean)"
  )) |>
  mutate(agent = factor(agent, levels = c("Optimal Solver", best_short_label, "LLM (Mean)")))

ggplot(score_long, aes(x = config_index, y = score_val, color = agent, group = agent)) +
  geom_hline(yintercept = 18.75, linetype = "dotted", color = "red", linewidth = 0.8) +
  annotate("text", x = max(score_long$config_index) + 0.3, y = 18.75,
           label = "Chance (18.75)", hjust = 1, vjust = -0.5, size = 3, color = "red") +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = setNames(
    c("#2196F3", "#4CAF50", "#FF9800"),
    c("Optimal Solver", best_short_label, "LLM (Mean)")
  )) +
  scale_x_continuous(breaks = unique(score_long$config_index)) +
  labs(x = "Configuration", y = "Score (lower is better)", color = NULL,
       title = "Score: Optimal vs Best Condition vs LLM Mean") +
  theme_minimal() +
  theme(legend.position = "top")
```

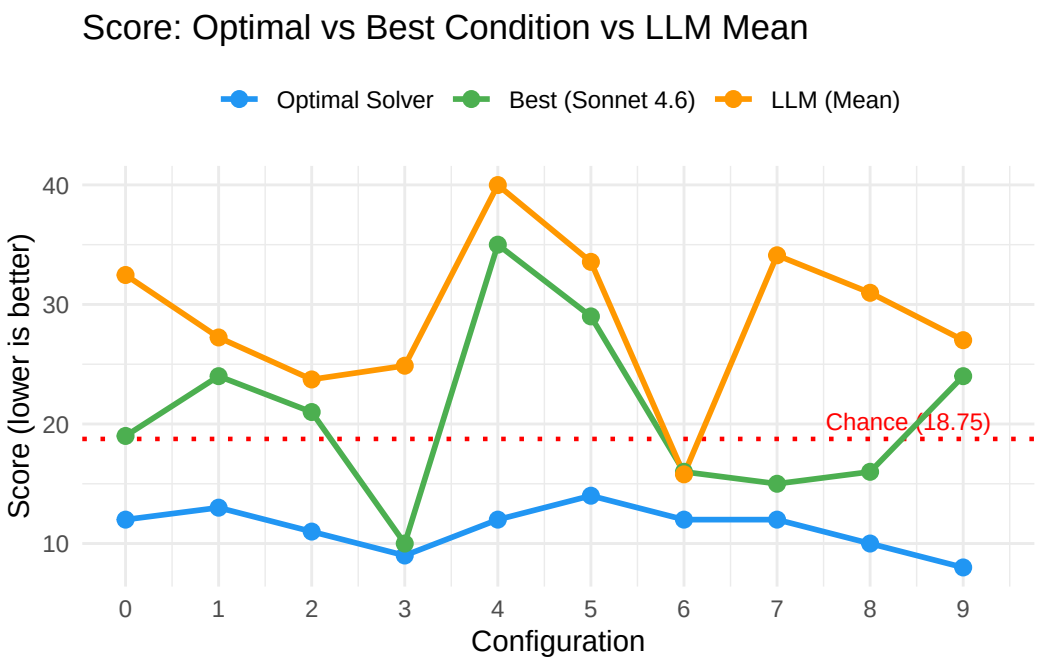


Figure 4: Score: LLM (mean across conditions) vs Optimal Solver by Configuration

```
# Per-model per-config aggregation
play_model_config <- play_data |>
  group_by(model_name, config_index) |>
  summarise(
    llm_score_mean = mean(score),
    llm_rays_mean = mean(rays_used),
    llm_atoms_mean = mean(atoms_correct),
    .groups = "drop"
  ) |>
  inner_join(optimal_data, by = "config_index") |>
  mutate(
    score_ratio = llm_score_mean / optimal_score,
    is_best = model_name == best_condition_lookup$model_name
  )

chance_ratio_data <- optimal_data |>
  mutate(chance_ratio = 18.75 / optimal_score)

ggplot(play_model_config, aes(x = config_index, y = score_ratio, color = model_name, group =
  geom_line(linewidth = 1) +
  geom_point(aes(shape = is_best), size = 2.5) +
```

```
  scale_shape_manual(values = c("TRUE" = 18, "FALSE" = 16), guide = "none") +
  geom_hline(yintercept = 1.0, linetype = "dashed", color = "grey40") +
  annotate("text", x = min(play_model_config$config_index) - 0.3, y = 1.0,
           label = "1.0× (optimal)", hjust = 0, vjust = -0.5, size = 3, color = "grey40") +
  geom_line(data = chance_ratio_data, aes(x = config_index, y = chance_ratio, group = 1),
            inherit.aes = FALSE, linetype = "dotted", color = "red", linewidth = 0.8) +
  annotate("text", x = max(play_model_config$config_index) + 0.3,
           y = mean(chance_ratio_data$chance_ratio),
           label = "Chance", hjust = 1, vjust = -0.5, size = 3, color = "red") +
  scale_x_continuous(breaks = unique(play_model_config$config_index)) +
  labs(x = "Configuration", y = "Score Ratio (LLM / Optimal)",
       color = "Model",
       title = "Score Ratio by Model and Configuration",
       subtitle = paste0("Dashed line = optimal (1.0×). Dotted red = chance. Diamond = best r
                         best_condition_lookup$model_name, ").")) +
  theme_minimal() +
  theme(legend.position = "top")
```

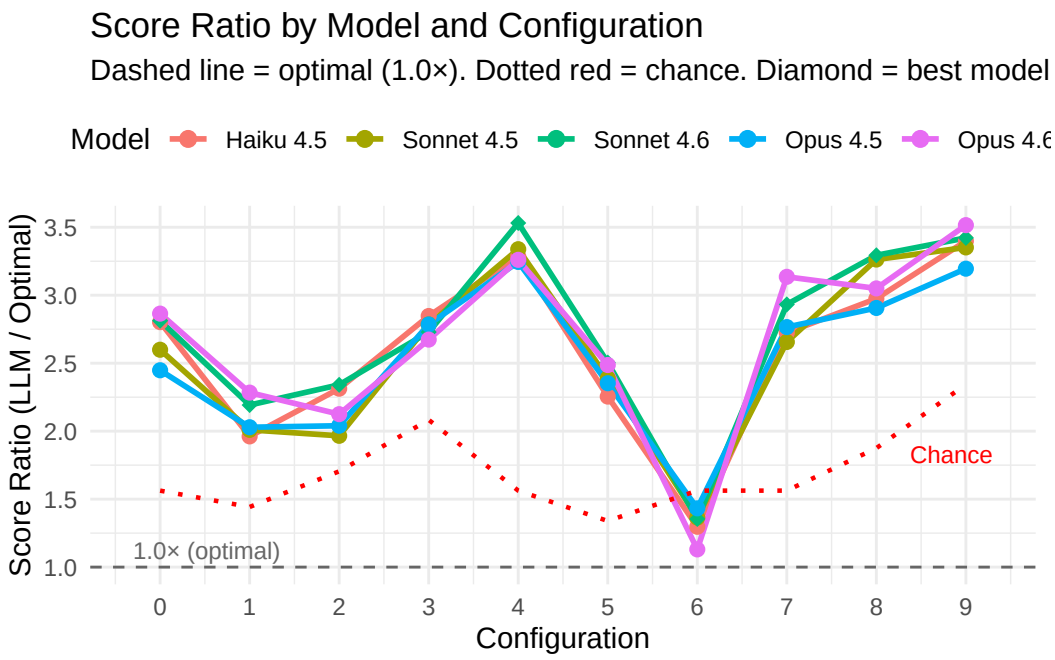


Figure 5: Score Ratio (LLM / Optimal) by Model and Configuration

```
diff_cor <- cor.test(play_model_config$optimal_rays, play_model_config$llm_atoms_mean)

# Join best condition data with optimal rays for scatter overlay
```

```r
best_scatter <- best_config_agg |>
  inner_join(optimal_data |> select(config_index, optimal_rays), by = "config_index")

ggplot(play_model_config, aes(x = optimal_rays, y = llm_atoms_mean, color = model_name)) +
  geom_jitter(width = 0.1, height = 0.05, size = 2.5, alpha = 0.7) +
  geom_point(data = best_scatter, aes(x = optimal_rays, y = best_atoms_mean),
             inherit.aes = FALSE, shape = 18, size = 4.5, color = "#4CAF50") +
  geom_smooth(method = "lm", se = TRUE, aes(group = 1), color = "grey40", linetype = "dashed
  labs(x = "Optimal Solver Rays (configuration difficulty)",
       y = "LLM Atoms Correct (mean)",
       color = "Model",
       title = "Configuration Difficulty vs LLM Atom Identification",
       subtitle = paste0("r = ", round(diff_cor$estimate, 3),
                         ", p = ", format.pval(diff_cor$p.value, digits = 3),
                         ". Green diamonds = best condition (", best_condition_lookup$model_
  scale_y_continuous(limits = c(0, 4)) +
  theme_minimal() +
  theme(legend.position = "top")
```

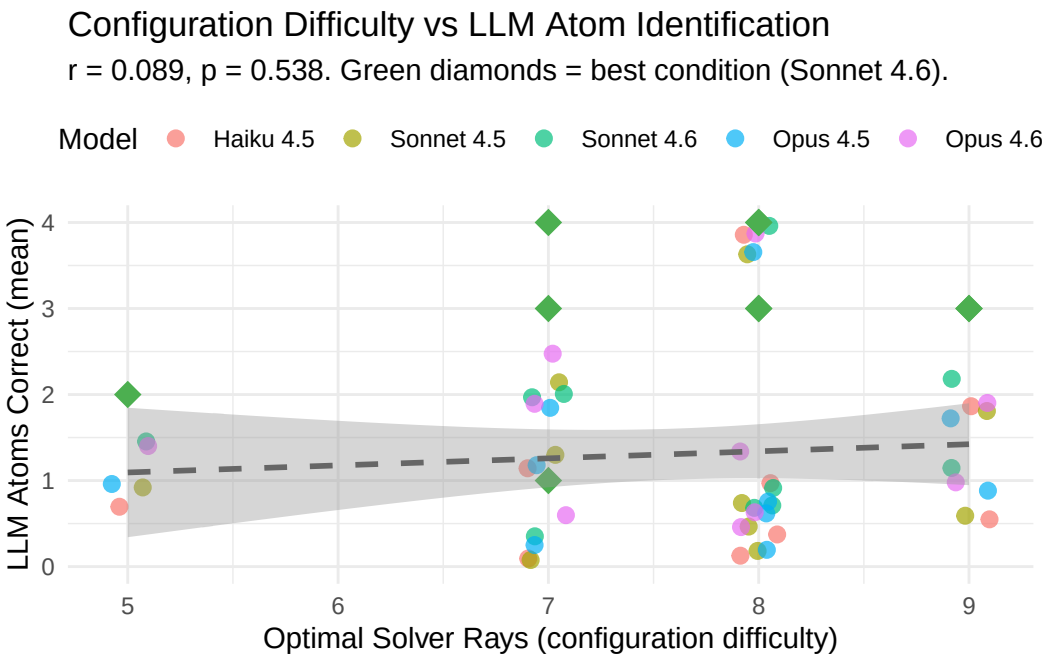


Figure 6: Configuration Difficulty (Optimal Rays) vs LLM Atoms Correct

Best condition summary. The best-performing condition by score is *Sonnet 4.6 | Aug | Think | B: Ray Trace*. Compared to the overall LLM mean, the best condition reduces the

score ratio from 2.6× to 1.9× optimal (a 29% reduction) and the rays ratio from 1.5× to 1.5× optimal. This highlights how much of the gap to optimal is driven by model capability vs. the averaging across weaker conditions.

4.6 Error Analysis

To move beyond the outcome-level confusion matrix above, we classify *which specific ray physics rule broke down* for each incorrect prediction. Since Predict mode is inherently spatial (given atoms, predict ray outcome), all errors are spatial — the interesting question is which rule the model failed to apply correctly.

We use a **deterministic classification** approach: comparing the predicted and actual outcome types (absorbed/reflected/detour) and exit positions to identify the specific failure mechanism. This requires no LLM judge and is 100% reproducible. The classification script (`classify_errors.py --mode predict`) processes all incorrect predictions.

For Play mode, where errors span both spatial and non-spatial domains (experiment design, constraint tracking, belief updating), we use an LLM-as-judge approach with optional gold-standard validation (`classify_errors.py --mode play --api-key KEY --gold-standard`).

```
error_class_file <- "error_classifications.json"
has_error_classifications <- file.exists(error_class_file)

if (has_error_classifications) {
  error_data <- jsonlite::fromJSON(error_class_file)

  # Predict mode: deterministic classifications
  predict_errors <- as_tibble(error_data$predict) |>
    mutate(model_name = model)

  # Play mode: LLM-as-judge classifications (if available)
  has_play_errors <- !is.null(error_data$play) && length(error_data$play) > 0
  if (has_play_errors) {
    play_errors <- as_tibble(error_data$play) |>
      mutate(
        error_label = case_when(
          primary_failure_mode == "ray_path_error" ~ "Ray path error",
          primary_failure_mode == "deflection_geometry_error" ~ "Deflection geometry error",
          primary_failure_mode == "constraint_tracking_error" ~ "Constraint tracking error",
          primary_failure_mode == "experiment_design_error" ~ "Experiment design error",
          primary_failure_mode == "belief_updating_error" ~ "Belief updating error",
          primary_failure_mode == "premature_commitment" ~ "Premature commitment",
          TRUE ~ primary_failure_mode
```

```
        ),
        error_domain = ifelse(is_spatial, "Spatial", "Non-spatial"),
        model_name = model
      )
  }

  # Gold-standard validation (if available)
  has_gold <- !is.null(error_data$play_gold_standard) &&
              length(error_data$play_gold_standard) > 0

  cat("Loaded", nrow(predict_errors), "Predict mode error classifications",
      "(deterministic)\n")
  if (has_play_errors) cat("Loaded", nrow(play_errors),
                           "Play mode classifications (LLM-as-judge)\n")
  if (has_gold) cat("Loaded", length(error_data$play_gold_standard),
                    "gold-standard validations (Opus)\n")
  cat("Classification date:", error_data$metadata$classification_date, "\n")
} else {
  has_play_errors <- FALSE
  has_gold <- FALSE
  cat("Error classifications not found. Run: python classify_errors.py --mode predict\n")
}
```

```
Loaded 2284 Predict mode error classifications (deterministic)
Classification date: 2026-03-08T03:05:12
```

4.6.1 Predict Mode: Ray Physics Rule Failures

In Predict mode, all errors are spatial by definition — the task is purely spatial reasoning. The deterministic classifier identifies which specific ray physics rule the model failed to apply, based on comparing the predicted vs. actual outcome types and exit positions.

Error taxonomy:

Error Category	Pattern	Description
Missed absorption	pred: detour/reflected → actual: absorbed	Failed to check for atom directly in ray's path before checking diagonals
Missed reflection	pred: detour/absorbed → actual: reflected	Failed to recognize reflection condition at entry point

Error Category	Pattern	Description
False absorption	pred: absorbed → actual: detour/reflected	Incorrectly identified absorption; confused diagonal atom with forward atom
False reflection	pred: reflected → actual: detour/absorbed	Incorrectly identified reflection condition
Deflection direction error	pred: detour(side A) → actual: detour(side B)	Deflected ray in wrong direction (e.g., south instead of west)
Off-by-one path error	pred: detour(side-N) → actual: detour(side-N±1)	Coordinate indexing error in cell-by-cell tracing; 97% of same-side errors
Large path tracking error	pred: detour(side-N) → actual: detour(side-M), |N-M| 2	Multiple accumulated errors in ray tracing

```
# Error category distribution - overall
cat_summary <- predict_errors |>
  count(error_label) |>
  arrange(desc(n)) |>
  mutate(pct = round(n / sum(n) * 100, 1))

kable(cat_summary,
      col.names = c("Error Category", "Count", "Percentage"),
      caption = "Predict Mode Error Categories (Deterministic Classification, N = 2,284)")
```

Table 37: Predict Mode Error Categories (Deterministic Classification, N = 2,284)

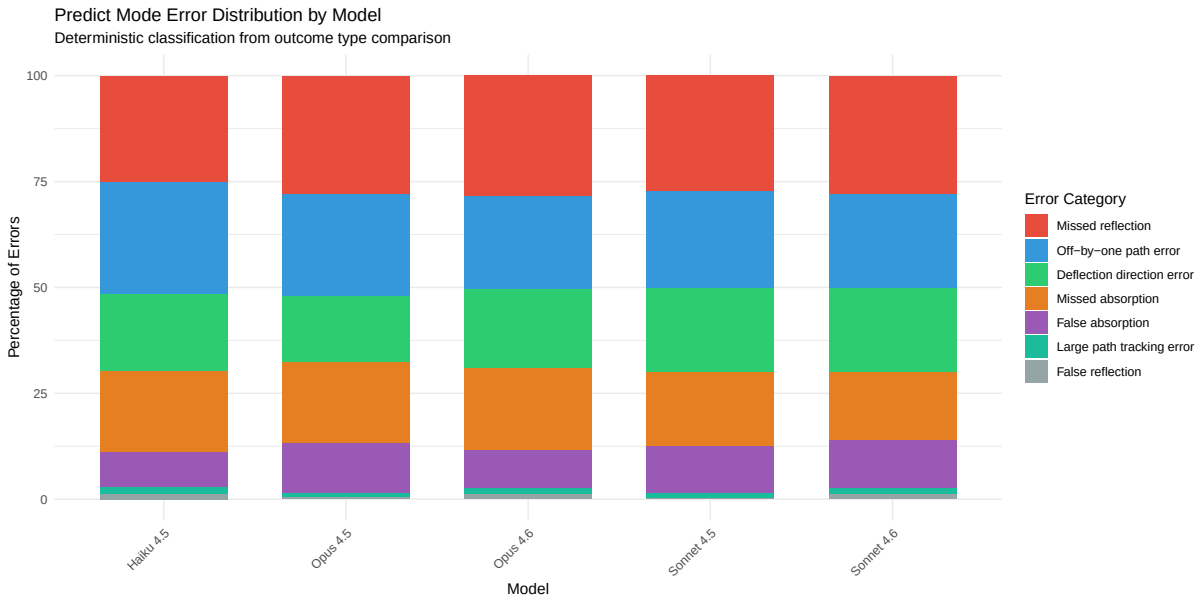
Error Category	Count	Percentage
Missed reflection	625	27.4
Off-by-one path error	536	23.5
Deflection direction error	421	18.4
Missed absorption	416	18.2
False absorption	236	10.3
Large path tracking error	29	1.3
False reflection	21	0.9

```
# Error category by model
error_by_model <- predict_errors |>
  count(model_name, error_label) |>
```

```r
  group_by(model_name) |>
  mutate(pct = n / sum(n) * 100) |>
  ungroup()

# Define order and colors based on error type
cat_order <- c("Missed reflection", "Off-by-one path error",
               "Deflection direction error", "Missed absorption",
               "False absorption", "Large path tracking error",
               "False reflection")
error_by_model$error_label <- factor(error_by_model$error_label, levels = cat_order)

# Color scheme: rule confusion errors in warm tones, path tracking in cool tones
p_cats <- ggplot(error_by_model, aes(x = model_name, y = pct, fill = error_label)) +
  geom_col(position = "stack", width = 0.7) +
  scale_fill_manual(values = c(
    "Missed reflection" = "#e74c3c",
    "Off-by-one path error" = "#3498db",
    "Deflection direction error" = "#2ecc71",
    "Missed absorption" = "#e67e22",
    "False absorption" = "#9b59b6",
    "Large path tracking error" = "#1abc9c",
    "False reflection" = "#95a5a6"
  )) +
  labs(
    title = "Predict Mode Error Distribution by Model",
    subtitle = "Deterministic classification from outcome type comparison",
    x = "Model", y = "Percentage of Errors", fill = "Error Category"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1), legend.position = "right")
print(p_cats)
```



4.6.2 Prompt Effect on Error Patterns

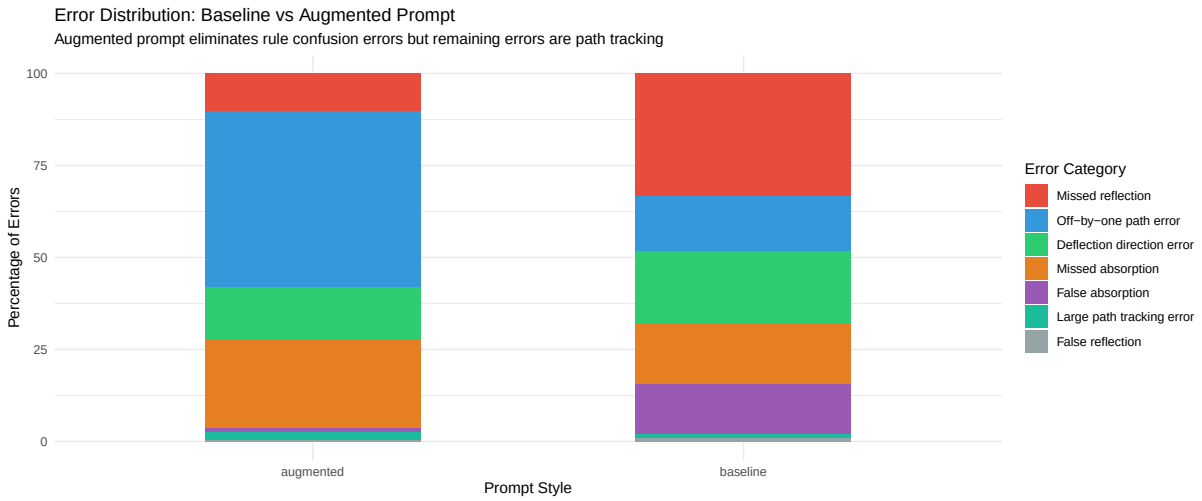
The augmented prompt includes explicit coordinate system explanation, detailed deflection geometry with worked examples, and common error warnings. How does this change the error distribution?

```
# Error categories by prompt style
error_by_prompt <- predict_errors |>
  count(prompt_style, error_label) |>
  group_by(prompt_style) |>
  mutate(pct = n / sum(n) * 100) |>
  ungroup()

error_by_prompt$error_label <- factor(error_by_prompt$error_label, levels = cat_order)

p_prompt <- ggplot(error_by_prompt, aes(x = prompt_style, y = pct, fill = error_label)) +
  geom_col(position = "stack", width = 0.5) +
  scale_fill_manual(values = c(
    "Missed reflection" = "#e74c3c",
    "Off-by-one path error" = "#3498db",
    "Deflection direction error" = "#2ecc71",
    "Missed absorption" = "#e67e22",
    "False absorption" = "#9b59b6",
    "Large path tracking error" = "#1abc9c",
```

```
    "False reflection" = "#95a5a6"
  )) +
  labs(
    title = "Error Distribution: Baseline vs Augmented Prompt",
    subtitle = "Augmented prompt eliminates rule confusion errors but remaining errors are pa
    x = "Prompt Style", y = "Percentage of Errors", fill = "Error Category"
  ) +
  theme_minimal() +
  theme(legend.position = "right")
print(p_prompt)
```




```
# Table: side-by-side comparison
prompt_comparison <- predict_errors |>
  count(prompt_style, error_label) |>
  group_by(prompt_style) |>
  mutate(pct = round(n / sum(n) * 100, 1)) |>
  ungroup() |>
  select(prompt_style, error_label, pct) |>
  pivot_wider(names_from = prompt_style, values_from = pct, values_fill = 0)

kable(prompt_comparison,
      caption = "Error Category Distribution by Prompt Style (% of errors within each style)"
```

Table 38: Error Category Distribution by Prompt Style (% of errors within each style)

error_label	augmented	baseline
Deflection direction error	14.2	19.9
False absorption	1.2	13.6
False reflection	0.3	1.1
Large path tracking error	2.2	0.9
Missed absorption	24.1	16.1
Missed reflection	10.2	33.5
Off-by-one path error	47.8	14.8

```
# Group errors into: rule confusion (missed/false absorption/reflection) vs path tracking
predict_errors_grouped <- predict_errors |>
  mutate(error_group = case_when(
    error_category %in% c("missed_absorption", "missed_reflection",
                          "false_absorption", "false_reflection") ~ "Rule confusion",
    error_category %in% c("deflection_direction_error", "off_by_one_error",
                          "large_path_error") ~ "Path tracking"
  ))

group_by_prompt <- predict_errors_grouped |>
  count(prompt_style, error_group) |>
  group_by(prompt_style) |>
  mutate(pct = round(n / sum(n) * 100, 1)) |>
  ungroup()

kable(group_by_prompt,
      col.names = c("Prompt", "Error Group", "Count", "%"),
      caption = "Rule Confusion vs Path Tracking Errors by Prompt Style")
```

Table 39: Rule Confusion vs Path Tracking Errors by Prompt Style

Prompt	Error Group	Count	%
augmented	Path tracking	384	64.2
augmented	Rule confusion	214	35.8
baseline	Path tracking	602	35.7
baseline	Rule confusion	1084	64.3

4.6.3 Entry Side Asymmetry

```
# Count total trials per side from the full predict dataset (correct + incorrect)
trials_per_side <- predict_data |>
  mutate(entry_side = ray_entry$side) |>
  count(entry_side, name = "trials")

# Error counts by entry side
errors_per_side <- predict_errors |>
  mutate(entry_side = ray_entry$side) |>
  count(entry_side, name = "errors")

# Join to compute error rate normalized by trials
entry_rate <- trials_per_side |>
  left_join(errors_per_side, by = "entry_side") |>
  mutate(
    errors = replace_na(errors, 0),
    error_rate = errors / trials * 100
  ) |>
  arrange(desc(error_rate))

kable(entry_rate,
      col.names = c("Entry Side", "Trials", "Errors", "Error Rate (%)"),
      digits = 1,
      caption = "Error Rate by Ray Entry Side (normalized by trials)")
```

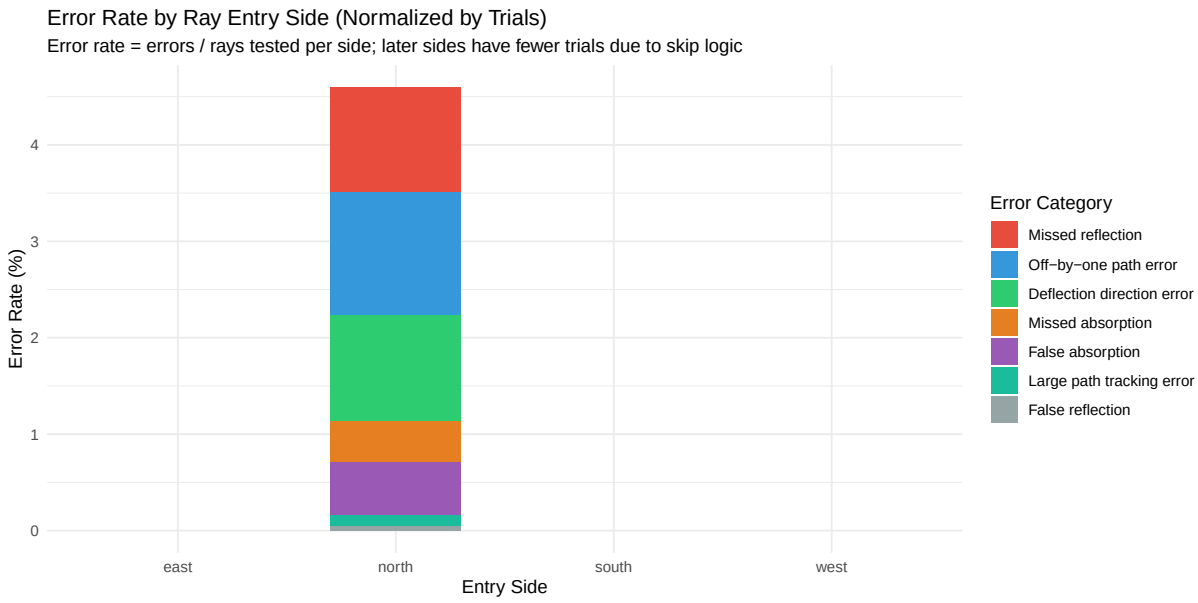
Table 40: Error Rate by Ray Entry Side (normalized by trials)

Entry Side	Trials	Errors	Error Rate (%)
north	18800	864	4.6

```
# Error category distribution by entry side
entry_side <- predict_errors |>
  mutate(entry_side = ray_entry$side) |>
  count(entry_side, error_label) |>
  left_join(trials_per_side, by = "entry_side") |>
  mutate(rate = n / trials * 100)

entry_side$error_label <- factor(entry_side$error_label, levels = cat_order)
```

```r
p_entry <- ggplot(entry_side, aes(x = entry_side, y = rate, fill = error_label)) +
  geom_col(position = "stack", width = 0.6) +
  scale_fill_manual(values = c(
    "Missed reflection" = "#e74c3c",
    "Off-by-one path error" = "#3498db",
    "Deflection direction error" = "#2ecc71",
    "Missed absorption" = "#e67e22",
    "False absorption" = "#9b59b6",
    "Large path tracking error" = "#1abc9c",
    "False reflection" = "#95a5a6"
  )) +
  labs(
    title = "Error Rate by Ray Entry Side (Normalized by Trials)",
    subtitle = "Error rate = errors / rays tested per side; later sides have fewer trials due
    x = "Entry Side", y = "Error Rate (%)", fill = "Error Category"
  ) +
  theme_minimal() +
  theme(legend.position = "right")
print(p_entry)
```



Limitation: In predict mode, rays are tested in a fixed order (North 1–8, South 1–8, East 1–8, West 1–8), and rays whose entry point was already observed as an exit point of a prior ray are skipped. This means later sides (East, West) are tested on fewer rays — and specifically on the subset of positions that were *not* exit points for earlier rays. The table above normalizes error counts by the number of trials actually conducted per side. However, the non-random

subsetting means the tested positions for later sides may be systematically different in difficulty. A stronger design would test all 32 entry points independently, regardless of prior exits.

4.6.4 Play Mode: Spatial vs. Non-Spatial Failure Modes

```
play_cat_summary <- play_errors |>
  count(error_label, error_domain) |>
  arrange(desc(n)) |>
  mutate(pct = round(n / sum(n) * 100, 1))

kable(play_cat_summary,
      col.names = c("Primary Failure Mode", "Domain", "Count", "Percentage"),
      caption = "Play Mode: Primary Failure Mode Distribution (LLM-as-judge)")

# Spatial vs non-spatial by model
play_spatial_by_model <- play_errors |>
  count(model_name, error_domain) |>
  group_by(model_name) |>
  mutate(pct = n / sum(n) * 100) |>
  ungroup()

p_play_spatial <- ggplot(play_spatial_by_model,
                         aes(x = model_name, y = pct, fill = error_domain)) +
  geom_col(position = "stack", width = 0.6) +
  geom_text(aes(label = paste0(round(pct, 0), "%")),
            position = position_stack(vjust = 0.5), size = 3.5) +
  scale_fill_manual(values = c("Spatial" = "#e74c3c", "Non-spatial" = "#3498db")) +
  labs(
    title = "Play Mode: Spatial vs Non-Spatial Primary Failure Mode",
    subtitle = "Classified by LLM-as-judge (Claude Haiku 4.5)",
    x = "Model", y = "Percentage", fill = "Error Domain"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1), legend.position = "top")
print(p_play_spatial)

# Spatial errors per game vs atoms correct
if ("spatial_errors_in_game" %in% names(play_errors)) {
  p_spatial_atoms <- ggplot(play_errors,
                            aes(x = spatial_errors_in_game, y = atoms_correct)) +
    geom_jitter(aes(color = model_name), width = 0.2, height = 0.1, size = 2, alpha = 0.7) +
```

```
    geom_smooth(method = "lm", se = TRUE, color = "grey40", linetype = "dashed") +
    labs(
      title = "Spatial Errors During Game vs Final Accuracy",
      x = "Number of Spatial Reasoning Errors in Game",
      y = "Atoms Correctly Identified (0-4)",
      color = "Model"
    ) +
    theme_minimal() +
    theme(legend.position = "top")
  print(p_spatial_atoms)
}
```

4.6.5 Gold-Standard Validation: Haiku vs Opus Agreement

```
gold <- as_tibble(error_data$play_gold_standard)

agree_rate <- mean(gold$agreement) * 100

cat(paste0("Overall Haiku-Opus agreement: ", round(agree_rate, 1), "% (",
           sum(gold$agreement), "/", nrow(gold), ")\n"))

# Agreement by category
gold_confusion <- gold |>
  count(haiku_primary, primary_failure_mode) |>
  pivot_wider(names_from = primary_failure_mode, values_from = n, values_fill = 0)

kable(gold_confusion,
      caption = "Gold-Standard Confusion Matrix: Haiku (rows) vs Opus (columns)")

# Agreement for spatial vs non-spatial
gold_spatial <- gold |>
  mutate(both_spatial = is_spatial & haiku_is_spatial,
         both_nonspatial = !is_spatial & !haiku_is_spatial,
         domain_agree = (is_spatial == haiku_is_spatial))
domain_agree_rate <- mean(gold_spatial$domain_agree) * 100

cat(paste0("Spatial/non-spatial domain agreement: ", round(domain_agree_rate, 1), "%\n"))
```

5 Discussion

5.1 Summary of Findings

[To be completed after data collection]

5.2 Relation to Literature

5.2.1 Constraint Satisfaction

[Discuss how results relate to ZebraLogic, GenCP findings]

5.2.2 Spatial Reasoning

[Discuss how results relate to PLUGH, spatial reasoning benchmarks]

5.2.3 World Models

[Discuss how results relate to pulley system, text world simulator findings]

5.2.4 Abductive Reasoning

[Discuss how results relate to Occam's Razor, hypothesis generation findings]

5.3 Benchmark Comparison

No single existing benchmark tests the full diagnostic reasoning pipeline that Black Box requires. Established benchmarks typically isolate spatial reasoning, constraint satisfaction, or game-playing individually, whereas Black Box demands all three capabilities operate simultaneously—spatial ray tracing, constraint propagation from accumulated evidence, and abductive inference about hidden state. This section contextualizes Claude's Black Box performance against its published results on benchmarks targeting each component capability. We restrict comparison to benchmarks where Claude-specific performance data is available, avoiding extrapolation from other model families.

```r
benchmark_comparison <- tibble::tribble(
  ~Benchmark, ~`Capability Tested`, ~`Claude Result`, ~`Black Box Analog`,
  "ZebraLogic", "Constraint satisfaction (logic grids)", "33.4% overall; 12.4% on hard puzzle
  "ConstraintBench", "Constraint satisfaction (optimization)", "49.0-50.5% feasibility (Opus
  "LogicGame", "Rule-based planning", "19.15% overall (Sonnet 3.5)", "Multi-step ray selecti
  "CrossWordBench", "Intersecting grid constraints", "0.482 WCR (Sonnet 3.7)", "Multi-ray co
  "BALROG", "Game environments (text/visual)", "Partial success on simple games only", "Perfo
  "PuzzlePlex", "Multi-turn puzzle solving", "Below simple heuristics", "LLMs use ~2x optimal
  "lmgame-Bench", "Game-playing (incl. Sokoban)", "Top non-reasoning tier (Sonnet 3.7)", "Str
  "Black Box Predict", "Spatial + forward reasoning", "[from experiment data]", "Forward reas
  "Black Box Play", "Spatial + constraint + abductive", "[from experiment data]", "Full diagn
)

kable(benchmark_comparison,
      col.names = c("Benchmark", "Capability Tested", "Claude Result", "Black Box Analog"),
      caption = "Claude performance on established benchmarks compared to Black Box analogs.
  kable_styling(bootstrap_options = c("striped", "hover", "condensed")) |>
  pack_rows("Constraint Satisfaction", 1, 4) |>
  pack_rows("Game-Playing & Spatial Reasoning", 5, 7) |>
  pack_rows("Black Box (This Study)", 8, 9)
```

Table 41: Claude performance on established benchmarks compa
results reflect published Claude model evaluations.

Benchmark	Capability Tested	Claude Result
Constraint Satisfaction		
ZebraLogic	Constraint satisfaction (logic grids)	33.4% overall; 12.4% on hard puzzles (Sonnet
ConstraintBench	Constraint satisfaction (optimization)	49.0-50.5% feasibility (Opus 4.5/4.6)
LogicGame	Rule-based planning	19.15% overall (Sonnet 3.5)
CrossWordBench	Intersecting grid constraints	0.482 WCR (Sonnet 3.7)
Game-Playing & Spatial Reasoning		
BALROG	Game environments (text/visual)	Partial success on simple games only
PuzzlePlex	Multi-turn puzzle solving	Below simple heuristics
lmgame-Bench	Game-playing (incl. Sokoban)	Top non-reasoning tier (Sonnet 3.7)
Black Box (This Study)		
Black Box Predict	Spatial + forward reasoning	[from experiment data]
Black Box Play	Spatial + constraint + abductive	[from experiment data]

5.3.1 Consistent Constraint Satisfaction Weakness

Claude's performance on constraint satisfaction benchmarks reveals a systematic limitation that Black Box corroborates. On ZebraLogic, Claude Sonnet 3.5 achieves 33.4% overall accuracy that drops to 12.4% on hard puzzles (Lin et al. 2024); on LogicGame, the same model reaches only 19.15% (Gui et al. 2024); and on CrossWordBench, Claude Sonnet 3.7 achieves a word completion rate of 0.482 (J. Wang et al. 2025). ConstraintBench further confirms this pattern, with Claude Opus models achieving only 49–51% feasibility on constraint satisfaction problems, identifying constraint satisfaction—not objective optimization—as the primary bottleneck (Tso et al. 2026). Black Box's constraint tracking task (ruling out candidate cells based on accumulated ray evidence) is fundamentally a constraint satisfaction problem, and the approximately 2× gap between LLM and optimal solver performance is consistent with these benchmark results. The pattern across benchmarks is clear: performance degrades sharply as constraint complexity increases.

5.3.2 Performance Collapse at Complexity Thresholds

BALROG demonstrates that Claude achieves partial success on simple game environments but fails on complex ones (Paglieri et al. 2024). PuzzlePlex confirms that LLMs perform below simple heuristics on multi-turn puzzles (Authors 2025b). lmgame-Bench shows Claude Sonnet 3.7 performing in the top tier among non-reasoning models but still far below optimal play on structured games like Sokoban (Hu et al. 2025). Black Box exhibits the same pattern: the optimal solver requires 5–9 rays depending on configuration difficulty, LLMs use approximately twice as many, and the gap widens on harder configurations. This aligns with Shojaee et al. (2025)'s observation of "accuracy collapse" at complexity thresholds, where reasoning model performance deteriorates abruptly rather than gradually as problem difficulty increases.

5.3.3 What Black Box Uniquely Reveals

The benchmarks reviewed above each test isolated capabilities—constraint satisfaction, spatial reasoning, or game-playing—in separate task environments. Black Box's contribution is testing the *intersection* of spatial reasoning, constraint satisfaction, abductive inference, active test selection, and sequential belief updating within a single unified task. Its two-mode design (Predict and Play) enables decomposition of the forward–inverse reasoning asymmetry: Predict isolates spatial simulation (given atoms, predict ray behavior), while Play requires the full diagnostic pipeline (observe rays, infer hidden atoms). This decomposition, absent from existing benchmarks, reveals whether failures stem from inability to simulate the physics or from inability to reason backward from observations. Furthermore, the optimal solver provides an information-theoretic performance ceiling, enabling rationality ratio analysis (LLM rays / optimal rays) rather than simple accuracy reporting—a quantitative measure of reasoning efficiency that most benchmarks lack.

5.4 Implications

5.4.1 For LLM Deployment in Diagnostic Domains

[Discuss implications for medical diagnosis, fault detection, and other diagnostic tasks]

5.4.2 For LLM-Modulo Architectures

[Discuss whether tool augmentation addresses the limitations]

5.4.3 For Benchmark Development

[Discuss Black Box as a diagnostic tool for reasoning assessment]

5.5 Limitations

1. **Single game domain**: Results may not generalize to all reasoning tasks
2. **Prompt sensitivity**: Different prompt formulations might yield different results
3. **Model versions**: Rapid model updates may change performance characteristics
4. **Sample size**: [Discuss statistical power]
5. **Predict mode ray selection**: In predict mode, rays are tested in a fixed sequential order (North 1–8, South 1–8, East 1–8, West 1–8), and entry points already observed as exit points of prior rays are skipped. This means later sides (East, West) are tested on fewer rays, and the tested subset is non-random — it excludes positions that served as exits for earlier rays. All 32 entry points should have been tested independently to enable unbiased comparison across entry sides.

5.6 Future Directions

1. **Extended model comparison**: GPT-4, Gemini, open-source models
2. **Tool augmentation study**: Provide ray-tracing verification tool
3. **Difficulty manipulation**: Vary grid size, atom count, atom arrangement
4. **Human baseline**: Compare LLM performance to human players
5. **Fine-tuning study**: Following the approach of Y. Wang et al. (2025), who demonstrated substantial reasoning improvements through game-specific training on Doudizhu and Go, test whether task-specific training on Black Box traces improves performance. Of particular interest is whether such training would generalize to novel atom configurations (in-distribution transfer) and to other spatial reasoning tasks (out-of-distribution transfer), or whether improvements would be limited to memorized patterns—the "catastrophic

forgetting" phenomenon observed in their work when game training interfered with unrelated reasoning tasks

6 Conclusion

[To be completed after data collection]

7 References

8 Appendix A: Prompt Texts

8.1 Baseline Play Prompt

```
You are playing Black Box, a game of hide and seek played on an 8 by 8 grid.

Your opponent has hidden 4 balls within this box. By shooting rays into the
box and observing where they emerge, it is possible to deduce the positions
of the hidden balls.

GRID: 8x8, rows 1-8 (top to bottom), columns 1-8 (left to right).
RAYS: Fire from edge positions - NORTH/SOUTH use columns 1-8, EAST/WEST use rows 1-
8.

There are three possible outcomes for each ray you send into the box:

DETOUR: The ray is deflected and emerges somewhere other than where you sent
it in. Detours are denoted by matching pairs of numbers.

REFLECTION (R): The ray is reflected and emerges in the same place it was sent in.

HIT (H): The ray strikes a ball directly and is absorbed.

[ASCII diagram examples showing deflection, reflection, and hit patterns...]

SCORING:
Your goal is to minimize your score. Lower is better.
- Each ray entry point costs 1 point
- Each ray exit point costs 1 point (detours cost 2 total, reflections cost 1)
- Each missed atom costs 5 points
```

```
RULES:
- You cannot fire from positions already used as entry or exit points.
- Maximum 20 rays.

Respond with JSON only:
{"action": "fire", "side": "north|south|east|west", "position": 1-8, "reasoning": "..."}
{"action": "guess", "atoms": [[row,col], [row,col], [row,col], [row,col]], "reasoning": "...
```

8.2 Baseline Predict Prompt

```
Predict where a ray will exit in Black Box.

GRID: 8x8, rows 1-8 (top to bottom), columns 1-8 (left to right).
EDGES: NORTH/SOUTH use columns 1-8, EAST/WEST use rows 1-8.

As a ray approaches a ball it is deflected ninety degrees. Rays can be
deflected multiple times.

[ASCII diagram examples showing deflection, reflection, and hit patterns...]

Respond with JSON only:
{"exit_side": "north|south|east|west", "exit_position": 1-8, "reasoning": "..."}
OR for absorption: {"absorbed": true, "reasoning": "..."}
OR for reflection: {"reflected": true, "reasoning": "..."}
```

8.3 Augmented Prompt

[Full augmented prompt text to be included]

8.4 Hypothesis-Enabled Play Prompt Extension

When hypotheses are enabled, the response format is extended:

```
Respond with JSON only:
{"action": "fire", "side": "north|south|east|west", "position": 1-8, "reasoning": "..."}
{"action": "mark", "row": 1-8, "col": 1-8, "reasoning": "..."} - mark suspected atom
{"action": "unmark", "row": 1-8, "col": 1-8, "reasoning": "..."} - remove marked position
{"action": "check", "reasoning": "..."} - submit answer (requires exactly 4 marked positions)
```

9 Appendix B: Sample Game Traces

[Representative game traces showing LLM reasoning]

10 Appendix C: Error Taxonomy

[Detailed coding scheme for error analysis]

Authors, Various. 2025a. "Auto-Bench: An Automated Benchmark for Scientific Discovery in LLMs." *arXiv Preprint arXiv:2502.15224*. https://arxiv.org/abs/2502.15224.

———. 2025b. "PuzzlePlex: A Benchmark to Evaluate the Reasoning and Planning of Large Language Models on Puzzles." In *International Conference on Learning Representations (ICLR)*.

Bonlarron, Alexandre, Jean-Charles Régin, and Elisabetta De Maria. 2024. "Combining Constraint Programming Reasoning with Large Language Model Predictions." In *Proceedings of the 30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, 307:25. LIPIcs. Schloss Dagstuhl. https://doi.org/10.4230/LIPIcs.CP.2024.25.

Bos, Nathan. 2024. "Language Models and Spatial Reasoning: What's Good, What Is Still Terrible, and What Is Improving." *Towards Data Science*. https://medium.com/data-science/language-models-and-spatial-reasoning.

Chen, Fangru et al. 2024. "An Empirical Analysis on Spatial Reasoning Capabilities of Large Multimodal Models," 1195.

Freuder, Eugene C. 2024. "Conversational Modeling for Constraint Satisfaction." *Proceedings of the AAAI Conference on Artificial Intelligence* 38 (20): 22592–97. https://doi.org/10.1609/aaai.v38i20.30225.

Giadikiaroglou, Panagiotis, Maria Lymperaiou, Giorgos Filandrianos, and Giorgos Stamou. 2024. "Puzzle Solving Using Reasoning of Large Language Models: A Survey." *arXiv Preprint arXiv:2402.11291*. https://arxiv.org/abs/2402.11291.

Gui, Jiayi, Yiming Liu, Jiale Cheng, Xiaotao Gu, Xiao Liu, Hongning Wang, Yuxiao Dong, Jie Tang, and Minlie Huang. 2024. "LogicGame: Benchmarking Rule-Based Reasoning Abilities of Large Language Models." *arXiv Preprint arXiv:2408.15778*. https://arxiv.org/abs/2408.15778.

Harig, Tobin et al. 2025. "LLM World Models Are Mental: Output Layer Evidence of Brittle World Model Use in LLM Mechanical Reasoning." *arXiv Preprint arXiv:2507.15521*.

Hu, Lanxiang, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P. Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. 2025. "Lmgame-Bench: How Good Are LLMs at Playing Games?" *arXiv Preprint arXiv:2505.15146*. https://arxiv.org/abs/2505.15146.

Kambhampati, Subbarao et al. 2024. "LLMs Can't Plan, but Can Help Planning in LLM-Modulo Frameworks." *arXiv Preprint arXiv:2402.01817*.

Lin, Bill Yuchen et al. 2024. "ZebraLogic: Benchmarking the Logical Reasoning Ability of Language Models." *Hugging Face Blog*. https://huggingface.co/blog/yuchenlin/zebra-logic.

Liu, Emmy, Graham Neubig, and Jacob Andreas. 2024. "An Incomplete Loop: Deductive, Inductive, and Abductive Learning in Large Language Models." *arXiv Preprint arXiv:2404.03028*.

Michailidis, Kostis, Dimosthenis Tsouros, and Tias Guns. 2024. "Constraint Modelling with LLMs Using in-Context Learning." In *Proceedings of the 30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, 307:20. LIPIcs. Schloss Dagstuhl. https://doi.org/10.4230/LIPIcs.CP.2024.20.

Omar, Mahmud, Reem Agbareia, Alon Gorenshtein, Alexander W Charney, Benjamin S Glicksberg, Girish N Nadkarni, and Eyal Klang. 2025. "Large Language Models Chase Zebras: Salient Cues Overrule Base Rates in Clinical Diagnosis." *SSRN Preprint*.

Paglieri, Davide, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, et al. 2024. "BALROG: Benchmarking Agentic LLM and VLM Reasoning on Games." *arXiv Preprint arXiv:2411.13543*. https://arxiv.org/abs/2411.13543.

Quan, Yida et al. 2025. "Benchmarking Spatiotemporal Reasoning in LLMs and Reasoning Models: Capabilities and Challenges." *arXiv Preprint arXiv:2505.11618*.

Rodriguez, Pablo et al. 2025. "Analogical Mappings of Facts and Counterfactuals in the Human Mind and Peirce's Abduction: Limitations in LLMs." *Cognitive Systems Research*. https://doi.org/10.1016/j.cogsys.2025.101389.

Shojaee, Parshin, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. 2025. "The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity." *arXiv Preprint arXiv:2506.06941*.

Tikhonov, Alexey et al. 2024. "PLUGH: A Benchmark for Spatial Understanding and Reasoning in Large Language Models." In *arXiv Preprint*. https://arxiv.org/abs/2408.04648.

Tso, Joseph, Preston Schmittou, Quan Huynh, and Jibran Hutchins. 2026. "Constraint-Bench: Benchmarking LLM Constraint Reasoning on Direct Optimization." *arXiv Preprint arXiv:2602.22465*. https://arxiv.org/abs/2602.22465.

Valmeekam, Karthik, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. 2023. "On the Planning Abilities of Large Language Models: A Critical Investigation." *Advances in Neural Information Processing Systems* 36.

Wang, Jixuan et al. 2025. "CrossWordBench: Evaluating the Reasoning Capabilities of LLMs and LVLMs with Controllable Puzzle Generation." *arXiv Preprint arXiv:2510.13271*. https://arxiv.org/abs/2510.13271.

Wang, Ruoyao, Graham Todd, Xingdi Yuan, et al. 2024. "Can Language Models Serve as Text-Based World Simulators?" In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Wang, Yifan, Xinnan Li, Yifei Niu, Shimin Hu, et al. 2025. "Empowering LLMs in Decision Games Through Algorithmic Data Synthesis." *arXiv Preprint arXiv:2503.13980*. https://arxiv.org/abs/2503.13980.

Wang, Zonglin et al. 2025. "Language Models Do Not Follow Occam's Razor: A Benchmark for Inductive and Abductive Reasoning." *arXiv Preprint arXiv:2509.03345*.

Wen, Andrew, Qiuhao Lu, Yu-Neng Chuang, et al. 2025. "Context Matching Is Not Reasoning When Performing Generalized Clinical Evaluation of Generative Language Models." *Npj*

Digital Medicine. https://doi.org/10.1038/s41746-025-02253-2.

Wikipedia contributors. 2024. "Black Box (Game)." https://en.wikipedia.org/wiki/Black_Box_(game).

Yang, Yunfan et al. 2025. "From Reasoning to Learning: A Survey on Hypothesis Discovery and Rule Learning with Large Language Models." *arXiv Preprint arXiv:2505.21935*.